

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Chí Danh - Lý Trường Nam - Phạm
Tấn Nghĩa - Hà Gia Phúc - Nguyễn Trường
Tân - Trần Nhật Tân

HỆ THỐNG HỖ TRỢ THÔNG TIN HỌC
TẬP CHO SINH VIÊN SỬ DỤNG AI
AGENTS

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng 04/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Chí Danh - 22120048

Lý Trường Nam - 22120218

Phạm Tấn Nghĩa - 22120230

Hà Gia Phúc - 22120272

Nguyễn Trường Tân - 22120326

Trần Nhật Tân - 22120328

**HỆ THỐNG HỖ TRỢ THÔNG TIN HỌC
TẬP CHO SINH VIÊN SỬ DỤNG AI
AGENTS**

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN
TS. Nguyễn Trần Minh Thư

Tp. Hồ Chí Minh, tháng 04/2026

Lời cam đoan

Nhóm xin cam đoan Dự án “Hệ thống hỗ trợ thông tin học tập cho sinh viên sử dụng AI AGENTS” là công trình nghiên cứu độc lập của nhóm, với sự hướng dẫn tận tình của cô Nguyễn Trần Minh Thư.

Công trình được nhóm nghiên cứu và hoàn thành tại Trường Đại học Khoa học Tự nhiên từ tháng 01 năm 2026 đến tháng 6 năm 2026. Các tài liệu tham khảo, các số liệu thống kê phục vụ mục đích nghiên cứu công trình này được sử dụng đúng quy định, không vi phạm quy chế bảo mật của Nhà nước.

Những số liệu, bảng biểu phục vụ cho việc phân tích và dẫn dắt đề tài này được thu thập từ các nguồn tài liệu khác nhau được trích dẫn một cách đầy đủ, đồng thời ghi rõ ràng về nguồn gốc theo quy định của nhà trường.

Ngoài ra, đối với các tài liệu diễn giải để làm rõ thêm các luận điểm đã phân tích và trích dẫn trong phần phụ lục cũng được chú thích nguồn gốc dữ liệu. Nếu những lời cam đoan trên của nhóm không chính xác, nhóm xin chịu hoàn toàn trách nhiệm và chịu mọi hình thức kỷ luật từ khoa và nhà trường.

Lời cảm ơn

Dự án thực tập tốt nghiệp chuyên ngành Hệ Thống Thông Tin trên đánh dấu cột mốc hoàn thành.

Chúng em xin gửi lời cảm ơn chân thành đến Quý Ban Giám hiệu và toàn thể cán bộ giảng viên của Trường Đại học Khoa học Tự nhiên - Đại học Quốc gia TP.HCM, vì đã cung cấp cho chúng em môi trường học tập và nghiên cứu tuyệt vời trong suốt khoảng thời gian học tập và làm việc tại nhà trường.

Và chúng em xin chân thành cảm ơn sâu sắc dành cho cô Nguyễn Trần Minh Thư vì sự tận tâm, kiến thức sâu rộng và sự hỗ trợ quý báu trong suốt quá trình thực hiện đồ án tốt nghiệp. Cô đã dành thời gian và nỗ lực để chỉ dẫn, định hướng và đánh giá công việc của chúng em. Sự tận tâm và kiến thức chuyên môn của cô đã giúp chúng em vượt qua những khó khăn và phát triển kỹ năng nghiên cứu và thực hiện đồ án của mình.

Mặc dù chúng em đã cố gắng hoàn thành dự án thực tập tốt nghiệp với kết quả tốt nhất có thể trong khả năng, nhưng vẫn có những khiếm khuyết, sai sót nhất định do thiếu kinh nghiệm thực tế. Chúng em mong nhận được sự góp ý chân thành và dạy bảo của quý Thầy Cô để có thể hoàn thiện bản thân hơn nữa.

Lời cảm ơn cuối cùng dành cho gia đình, bạn bè và những người thân yêu đã luôn động viên và đồng hành cùng chúng em trong suốt quá trình nghiên cứu và thực hiện đồ án tốt nghiệp. Sự ủng hộ và động viên của họ là nguồn động lực vô cùng quan trọng giúp chúng em vượt qua khó khăn và hoàn thành công việc một cách tốt nhất.

Đề cương chi tiết

ĐỀ CƯƠNG THỰC TẬP DỰ ÁN TỐT NGHIỆP

HỆ THỐNG HỖ TRỢ THÔNG TIN HỌC TẬP CHO SINH VIÊN SỬ DỤNG AI AGENTS

AN AI AGENT-BASED SYSTEM FOR LEARNING INFORMATION SUPPORT FOR STUDENTS

1 Thông tin chung

Người hướng dẫn:

- TS. Nguyễn Trần Minh Thư (Khoa Công nghệ Thông tin)

Nhóm sinh viên thực hiện:

1. Nguyễn Chí Danh (MSSV: 22120048)
2. Lý Trường Nam (MSSV: 22120218)
3. Phạm Tấn Nghĩa (MSSV: 22120230)
4. Hà Gia Phúc (MSSV: 22120272)
5. Nguyễn Trường Tân (MSSV: 22120326)
6. Trần Nhật Tân (MSSV: 22120328)

Loại đề tài: Ứng dụng

Thời gian thực hiện: Từ 01/2026 đến 07/2026

2 Nội dung thực hiện

2.1 Giới thiệu về đề tài

Hiện nay, trong môi trường đại học, quá trình đăng ký môn học và xây dựng lộ trình học tập đóng vai trò vô cùng quan trọng đối với sự phát triển của sinh viên. Tuy nhiên, trên thực tế, sinh viên thường xuyên gặp nhiều khó khăn trong việc lựa chọn các học phần phù hợp với từng học kỳ, cũng như chưa thực sự hiểu rõ mối liên hệ mật thiết giữa các môn học và định hướng nghề nghiệp trong tương lai. Bên cạnh đó, sinh viên còn thường rơi vào tình trạng thiếu thông tin về nội dung cốt lõi, tài liệu tham khảo và quy chế học vụ của nhà trường, dẫn đến việc học tập thiếu định hướng và kém hiệu quả.

Để giải quyết các vấn đề trên, đề tài đề xuất xây dựng một hệ thống hỗ trợ thông tin học tập thông minh dựa trên kiến trúc AI Agents kết hợp với LightRAG nhằm xây dựng và khai thác Đồ thị tri thức (Knowledge Graph). Hệ thống thu thập dữ liệu học vụ từ các nguồn đa dạng bao gồm chương trình đào tạo, đề cương chi tiết môn học và quy chế học vụ của Khoa Công nghệ Thông tin, sau đó cấu trúc hóa toàn bộ thành một mạng lưới tri thức có khả năng suy luận đa bước. Thay vì chỉ tìm kiếm từ khóa đơn thuần, hệ thống có thể hiểu được các mối quan hệ logic giữa môn tiên quyết, môn kế tiếp và các yêu cầu tốt nghiệp, từ đó cung cấp câu trả lời chính xác và có căn cứ rõ ràng hơn so với các chatbot thông thường. Điều này giúp sinh viên tra cứu thông tin học vụ, hiểu rõ mối liên hệ giữa các học phần và lập kế hoạch học tập hiệu quả trong suốt quá trình đào tạo.

2.2 Mục tiêu đề tài

2.2.1 Tại sao cần thực hiện đề tài này?

Trong bối cảnh giáo dục đại học hiện nay, sinh viên thường xuyên đối mặt với sự thiếu liên kết giữa các môn học trong chương trình đào tạo và định hướng học tập thực tế. Khó khăn lớn nhất là việc nhiều sinh viên không thể xác định được môn học nào thực sự cần thiết để đạt mục tiêu của mình, cũng như không nắm rõ các điều kiện tiên quyết và song hành

phức tạp giữa các học phần. Bên cạnh đó, thông tin học vụ hiện tại thường tồn tại dưới dạng các tệp PDF rời rạc hoặc trang web tĩnh, khiến sinh viên phải tự tra cứu thủ công qua nhiều nguồn mà vẫn không có được câu trả lời tổng hợp và chính xác. Các mô hình AI phổ biến đôi khi cung cấp thông tin sai lệch do không được huấn luyện trên dữ liệu đặc thù của nhà trường, gây ra những hậu quả nghiêm trọng cho lộ trình học tập của sinh viên. Chính vì vậy, việc xây dựng một hệ thống hỗ trợ thông minh, có khả năng trả lời chính xác dựa trên kho tri thức học vụ thực tế của Khoa, là vô cùng cấp thiết và có ý nghĩa thực tiễn cao.

2.2.2 Đề tài mang lại được điều gì?

Đề tài hướng đến việc phát triển hệ thống hỗ trợ thông tin học tập thông minh cho sinh viên Khoa Công nghệ Thông tin – Trường Đại học Khoa học Tự nhiên với các khả năng chính:

- Gợi ý môn tiên quyết, môn kế tiếp và lập kế hoạch học tập tối ưu theo từng học kỳ.
- Cung cấp thông tin kiến thức, nội dung, tài liệu của từng môn học, đồng thời giải đáp các thắc mắc về quy chế học vụ và điều kiện tốt nghiệp.
- Ứng dụng AI Agent hỗ trợ học tập cá nhân hóa, theo dõi tiến trình học và gợi ý hướng đi phù hợp với từng sinh viên.
- Cung cấp ứng dụng Mobile (Android) cho phép sinh viên tra cứu thông tin học vụ và đặt câu hỏi với AI Agent mọi lúc, mọi nơi.

2.2.3 Ảnh hưởng và ý nghĩa của kết quả đề tài

Kết quả của đề tài không chỉ là một công cụ hỗ trợ tra cứu đơn thuần mà còn là giải pháp nâng cao tính chủ động và hiệu quả học tập cho sinh viên trong suốt quá trình đào tạo. Đối với giảng viên và nhà trường, hệ thống giúp giảm bớt áp lực trong công tác tư vấn học tập, đồng thời cung cấp một kho tri thức học vụ được số hóa và cấu trúc hóa hoàn chỉnh, tạo cơ sở để cải tiến chương trình đào tạo sát với thực tiễn hơn. Ý nghĩa khoa

học của nghiên cứu còn nằm ở việc đề xuất và hiện thực hóa một quy trình xây dựng Đồ thị tri thức chất lượng cao cho văn bản học thuật tiếng Việt, bao gồm các cơ chế xác thực SHACL và phân giải thực thể, mở rộng hướng ứng dụng của xử lý ngôn ngữ tự nhiên (NLP) và các mô hình ngôn ngữ lớn (LLM) vào bối cảnh giáo dục đại học một cách hiệu quả. Đây là một bước tiến quan trọng trong việc hình thành các hệ thống giáo dục thông minh, góp phần hiện đại hóa phương thức tương tác giữa sinh viên và tri thức. Về lâu dài, dự án hướng tới xây dựng một môi trường học tập kết nối, nơi tri thức học vụ được lưu trữ và cập nhật liên tục, góp phần nâng cao chất lượng đầu ra cho ngành Công nghệ Thông tin.

2.3 Phạm vi của đề tài

Đề tài tập trung nghiên cứu và xây dựng hệ thống hỗ trợ thông tin học tập cho sinh viên Khoa Công nghệ Thông tin – Trường Đại học Khoa học Tự nhiên thông qua các hướng sau:

- Xây dựng quy trình thu thập và tiền xử lý dữ liệu: Triển khai hệ thống cào dữ liệu thông minh (Smart Scraper) từ website Khoa kết hợp với công cụ MinerU để bóc tách nội dung chính xác từ các tài liệu PDF phức tạp, bao gồm tài liệu dạng scan, bảng biểu và công thức toán học.
- Tối ưu hóa truy xuất tri thức với LightRAG: Nghiên cứu và tùy chỉnh pipeline trích xuất thực thể (Entities) và mối quan hệ (Relationships) từ chương trình đào tạo để xây dựng Đồ thị tri thức, kết hợp chiến lược phân đoạn kép (Dual Chunking) và cơ chế xác thực SHACL, phân giải thực thể tự động.
- Triển khai kiến trúc Đa tác nhân (Multi-Agent): Thiết lập luồng điều phối Agentic Workflow sử dụng framework LangGraph, trong đó các tác nhân chuyên biệt phối hợp phân tích câu hỏi, truy xuất tri thức và tổng hợp câu trả lời cho sinh viên.

Đối tượng nghiên cứu: Các mô hình ngôn ngữ lớn (LLM), kỹ thuật GraphRAG (LightRAG), và kiến trúc hệ thống Đa tác nhân (Multi-Agent) trên nền tảng LangGraph.

Các thực thể chính trong hệ thống:

- Môn học: Thực thể trung tâm với các mối quan hệ “tiên quyết”, “song hành”, “thuộc khối kiến thức”.
- Chương trình đào tạo: Nhóm các môn học theo ngành và chuyên ngành.
- Chuẩn đầu ra: Các kỹ năng, kiến thức mà sinh viên cần đạt sau khi hoàn thành môn học.
- Văn bản quy định: Nguồn gốc tri thức bao gồm quy chế học vụ và đề cương chi tiết.

Tập dữ liệu (Dataset):

- Dữ liệu chương trình đào tạo của Khoa Công nghệ Thông tin – Trường Đại học Khoa học Tự nhiên (các khóa từ 2022 đến 2025, bao gồm sáu ngành và khối ngành thuộc Khoa, bao gồm: Cử nhân tài năng, Công nghệ thông tin, Hệ thống thông tin, Khoa học máy tính, Kỹ thuật phần mềm và Trí tuệ nhân tạo).
- Dữ liệu đề cương chi tiết các học phần và quy chế học vụ, quy định xét chuyên ngành.
- Dữ liệu thu thập tự động từ website của Khoa (fit.hcmus.edu.vn) và Trường (hcmus.edu.vn).

Phạm vi dữ liệu: Hệ thống chỉ hỗ trợ dữ liệu tri thức và chương trình đào tạo thuộc Khoa Công nghệ Thông tin – Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM.

Ràng buộc kỹ thuật: Hệ thống yêu cầu kết nối Internet để truy xuất API của mô hình ngôn ngữ lớn, độ chính xác của tư vấn phụ thuộc vào tính cập nhật của dữ liệu chương trình đào tạo được nạp vào Đồ thị tri thức.

Giới hạn chức năng: Hệ thống đóng vai trò hỗ trợ tư vấn và định hướng thông tin học vụ, không thay thế cho các quyết định hành chính chính thức từ phòng đào tạo.

2.4 Cách tiếp cận dự kiến

Các hệ thống sinh văn bản tăng cường truy xuất (Retrieval-Augmented Generation – RAG) truyền thống đã giải quyết được bài toán giao tiếp ngôn ngữ tự nhiên, nhưng vẫn gặp hạn chế lớn do phụ thuộc vào biểu diễn dữ liệu dạng phẳng, dẫn đến việc bỏ sót các mối quan hệ phức tạp giữa các môn học. Các mô hình nâng cao hơn như GraphRAG của Microsoft có khả năng suy luận đa bước nhưng lại phát sinh chi phí tính toán rất lớn và không hỗ trợ cập nhật dữ liệu tăng dần [1]. Để khắc phục những hạn chế trên, đề tài áp dụng kiến trúc LightRAG kết hợp hệ thống Đa tác nhân (Multi-Agent), vừa đảm bảo khả năng suy luận trên Đồ thị tri thức, vừa tối ưu hóa chi phí và hỗ trợ cập nhật tri thức linh hoạt [6].

Với mục tiêu đó, nhóm thực hiện các bước tiếp cận như sau:

- Khảo sát nhu cầu của sinh viên về các tính năng và nghiệp vụ cần quan tâm liên quan đến tư vấn lộ trình học tập, tài liệu môn học để xây dựng một hệ thống tư vấn học tập hoàn chỉnh cho sinh viên.
- Thu thập và tiền xử lý dữ liệu học vụ: triển khai hệ thống thu thập dữ liệu tự động từ website Khoa kết hợp với công cụ trích xuất tài liệu PDF chất lượng cao để đảm bảo đầu vào sạch và đầy đủ cho các bước xử lý tiếp theo.
- Xây dựng Đồ thị tri thức với LightRAG: trích xuất tự động các thực thể và mối quan hệ từ tài liệu học vụ, đảm bảo chất lượng tri thức thông qua cơ chế xác thực và phân giải thực thể trùng lặp.
- Xây dựng hệ thống Đa tác nhân: thiết kế luồng điều phối AI Agent có khả năng phân tích câu hỏi, truy xuất tri thức phù hợp và tổng hợp câu trả lời chính xác, đồng thời tích hợp bộ nhớ để duy trì ngữ cảnh hội thoại.
- Thiết kế và triển khai ứng dụng: xây dựng giao diện web và ứng dụng Mobile thân thiện với người dùng, tích hợp hệ thống xác thực tài khoản và kết nối với tầng AI.
- Đánh giá hệ thống và kiểm thử: đánh giá chất lượng truy xuất và sinh văn bản trên bộ câu hỏi chuẩn, đánh giá chất lượng Đồ thị tri thức, sau

đó tiến hành thử nghiệm thực tế với sinh viên để thu thập phản hồi và cải thiện hệ thống.

2.5 Kết quả dự kiến của đề tài

Hệ thống AI được xây dựng có khả năng đưa ra câu trả lời chính xác, có căn cứ dựa trên kho tri thức học vụ thực tế của Khoa, hạn chế tối đa hiện tượng ảo giác thông tin (hallucination) nhờ kiến trúc GraphRAG và cơ chế xác thực SHACL. Hệ thống hỗ trợ cập nhật tri thức tăng dần mà không cần xây dựng lại toàn bộ Đồ thị, đồng thời tối ưu chi phí gọi API thông qua cơ chế phân cấp truy vấn của LightRAG. Sản phẩm đầu ra bao gồm:

- Website tư vấn học tập tích hợp AI Agent, có khả năng: trả lời câu hỏi về môn tiên quyết, quy chế học vụ và điều kiện tốt nghiệp, cung cấp lộ trình học tập cá nhân hóa, hiển thị câu trả lời theo luồng với trích dẫn nguồn gốc tài liệu rõ ràng.
- Ứng dụng Mobile (Android) cung cấp trải nghiệm tương tự website, cho phép sinh viên tra cứu thông tin học vụ, nhận gợi ý lộ trình học tập và đặt câu hỏi với AI Agent thuận tiện mọi lúc, mọi nơi.
- Tài liệu kỹ thuật hệ thống bao gồm đặc tả yêu cầu, mô hình cơ sở dữ liệu, sơ đồ kiến trúc và báo cáo đánh giá hiệu năng.

Các công trình khoa học liên quan:

- *LightRAG: Simple and Fast Retrieval-Augmented Generation* [6] – Giới thiệu kiến trúc GraphRAG nhẹ tích hợp đồ thị thực thể vào quá trình truy xuất, hỗ trợ cập nhật tri thức tăng dần với chi phí tính toán thấp.
- *From Local to Global: A Graph RAG Approach to Query-Focused Summarization* [1] – Đề xuất kiến trúc GraphRAG phân cấp cộng đồng của Microsoft, cơ sở lý thuyết để so sánh và lựa chọn LightRAG trong đề tài.

- *RAGAS: Automated Evaluation of Retrieval Augmented Generation* [2]
- Framework đánh giá tự động hệ thống RAG theo các chỉ số Faithfulness, Answer Relevance, Context Precision và Context Recall, được áp dụng trực tiếp trong đề tài.

2.6 Kế hoạch thực hiện

STT	Bắt đầu	Kết thúc	Nội dung công việc	Thành viên thực hiện
1	26/10/2025	02/11/2025	Chuẩn bị câu hỏi khảo sát tình trạng của sinh viên khi đăng ký môn học và các vấn đề khi học tập.	Tất cả thành viên
2	02/11/2025	09/11/2025	Thu thập kết quả khảo sát. Phân tích kết quả khảo sát và viết báo cáo. Thu thập tài liệu đào tạo (PDF chương trình đào tạo, đề cương, quy chế học vụ), đánh giá chất lượng dữ liệu đầu vào, xác định phạm vi và yêu cầu hệ thống.	Tất cả thành viên
3	09/11/2025	23/11/2025	Tổng hợp yêu cầu hệ thống, viết đặc tả Use Case, thiết kế kiến trúc tổng thể sơ bộ, phân công nhiệm vụ cho từng thành viên. Đề xuất giải pháp thực hiện ứng dụng.	Tất cả thành viên
4	23/11/2025	14/12/2025	Tìm hiểu GraphRAG và Knowledge Graph. Tìm giải pháp tối ưu hơn GraphRAG cho việc rút trích dữ liệu. Nghiên cứu các công trình liên quan về RAG, GraphRAG và hệ thống Đa tác nhân.	Tất cả thành viên
5	14/12/2025	01/01/2026	Nghiên cứu chuyên sâu về LightRAG.	Tất cả thành viên

STT	Bắt đầu	Kết thúc	Nội dung công việc	Thành viên thực hiện
6	01/01/2026	15/01/2026	Thiết kế UI/UX trên Figma cho ứng dụng web, thiết kế API xác thực (đăng ký, đăng nhập, quên mật khẩu), thiết kế cơ sở dữ liệu PostgreSQL, MongoDB, nghiên cứu LangGraph để xây dựng Agentic Workflow.	Tất cả thành viên
7	15/01/2026	08/02/2026	Triển khai Smart Scraper Engine thu thập dữ liệu từ website Khoa, tích hợp MinerU để tiền xử lý tài liệu PDF phức tạp, xây dựng LightRAG với tùy chỉnh SHACL và Entity Resolution, xây dựng hệ thống Đa tác nhân sơ bộ với LangGraph.	Tất cả thành viên
8	08/02/2026	15/02/2026	Nghiên cứu và thiết kế cơ chế bộ nhớ dài hạn cho Agent (Long-term Memory), tối ưu hóa độ trễ phản hồi và giảm chi phí gọi API LLM.	Tất cả thành viên
9	15/02/2026	15/03/2026	Tích hợp và triển khai toàn bộ hệ thống: backend Spring Boot, frontend Next.js, cơ sở dữ liệu (PostgreSQL, MongoDB, Neo4j, Qdrant, Redis) và hệ thống AI Agent.	Tất cả thành viên
10	15/03/2026	08/04/2026	Tùy chỉnh pipeline trích xuất thực thể, nghiên cứu và triển khai cơ chế xác thực SHACL và phân giải thực thể (Entity Resolution).	Tất cả thành viên

STT	Bắt đầu	Kết thúc	Nội dung công việc	Thành viên thực hiện
11	08/04/2026	29/04/2026	Đánh giá chất lượng Đồ thị tri thức (V-Measure, Groundedness, tỷ lệ nút mờ côi), đánh giá chất lượng RAG với framework RAGAS (Faithfulness, Answer Relevance, Context Recall), kiểm thử và tối ưu hóa hệ thống.	Tất cả thành viên
12	29/04/2026	14/05/2026	Thử nghiệm hệ thống trên môi trường production, tiến hành beta testing với sinh viên Khoa Công nghệ Thông tin.	Tất cả thành viên
13	14/05/2026	20/05/2026	Thu thập và phân tích phản hồi từ người dùng thực, chuẩn bị bộ câu hỏi đánh giá trải nghiệm sử dụng.	Tất cả thành viên
14	20/05/2026	15/06/2026	Cải thiện và hoàn thiện hệ thống dựa trên phản hồi người dùng, viết và hoàn thiện báo cáo tốt nghiệp.	Tất cả thành viên
15	15/06/2026	06/07/2026	Hoàn thiện báo cáo, chuẩn bị tài liệu hướng dẫn triển khai hệ thống và chuẩn bị bảo vệ đồ án tốt nghiệp.	Tất cả thành viên
Phát triển về sau			Nâng cấp và cải tiến hệ thống: mở rộng dữ liệu sang các Khoa khác trong Trường, tích hợp trực tiếp với cổng thông tin đào tạo của Trường.	

Tài liệu

- [1] D. Edge et al., “From local to global: A graph rag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.16130>
- [2] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2023.
- [3] Y. Gao et al., “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023. [Online]. Available: <https://arxiv.org/abs/2312.10997>
- [4] T. Nguyen, N. Vu, and B. Ly, “An approach to constructing a graph data repository for course recommendation based on it career goals in the context of big data,” in *2022 IEEE International Conference on Big Data (Big Data)*, IEEE, 2022, pp. 301–307. DOI: 10.1109/BigData55660.2022
- [5] Z. Xu et al., “Stepchain graphrag: Reasoning over knowledge graphs for multi-hop question answering,” *arXiv preprint arXiv:2510.02827*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.02827>
- [6] H. Zhu et al., “Lightrag: Simple and fast retrieval-augmented generation,” *arXiv preprint arXiv:2410.05779*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.05779>

XÁC NHẬN
CỦA NGƯỜI HƯỚNG DẪN
(Ký và ghi rõ họ tên)

TP. Hồ Chí Minh, ngày 16 tháng 03 năm 2026
NHÓM SINH VIÊN THỰC HIỆN
(Ký và ghi rõ họ tên)

Mục lục

Lời cam đoan	i
Lời cảm ơn	ii
Đề cương chi tiết	iii
Mục lục	xiv
Danh sách viết tắt	xvii
Danh sách hình	xx
Danh sách bảng	xxii
Tóm tắt	xxii
1 Giới thiệu	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu	2
1.3 Phạm vi đề tài	2
1.4 Giải pháp và cách tiếp cận	3
1.5 Đóng góp	3
1.6 Bố cục	4
2 Các công trình liên quan	5
2.1 Tổng quan về hệ thống hỗ trợ học tập thông minh	5
2.1.1 Các hướng tiếp cận truyền thống	5
2.1.2 Ứng dụng AI trong giáo dục hiện đại	6
2.2 Các hệ thống tương tự	7
2.2.1 Các hệ thống tư vấn học vụ quốc tế	7
2.2.2 Các hệ thống tư vấn tại Việt Nam	7

2.3	Mô hình ngôn ngữ lớn (LLMs) trong giáo dục	8
2.3.1	Ứng dụng LLM trong hệ thống hỏi đáp học tập . .	8
2.3.2	Hạn chế của LLM (Hallucination, thiếu context, v.v.)	9
2.4	Retrieval-Augmented Generation (RAG) truyền thống . . .	10
2.5	Các hệ thống RAG dựa trên Đồ thị tri thức	10
2.6	So sánh các giải pháp RAG hiện nay	11
2.7	Lý do lựa chọn LightRAG	14
2.8	Hiện trạng của LightRAG (Phiên bản 1.4.11)	15
2.9	Hệ thống Đa tác nhân (Multi-Agent Systems) và Luồng suy luận (Agentic Workflow)	16
2.9.1	Khái niệm AI Agent	17
2.9.2	Tích hợp LightRAG và Multi-Agent	17
2.10	Đánh giá hệ thống RAG và Multi-Agent	18
2.10.1	Các chỉ số đánh giá RAG (RAGAS)	18
2.10.2	Đánh giá hệ thống đa agent (Multi-Agent Evaluation)	19
2.10.3	Các phương pháp benchmark hiện nay	19
3	Phương pháp đề xuất	21
3.1	Quy trình xử lý dữ liệu	22
3.1.1	Tiền xử lý dữ liệu tri thức	23
3.1.2	Luồng logic xử lý và cơ chế kiểm tra trùng lặp . . .	27
3.1.3	Chiến lược phân đoạn và chuẩn hóa văn bản	30
3.1.4	Cơ chế trích xuất thực thể và xây dựng quan hệ . .	37
3.1.5	Luồng xử lý Prompt linh hoạt cho đa dạng nguồn dữ liệu	38
3.1.6	Kiểm chứng tri thức bằng chuẩn SHACL và đối chiếu bằng chứng xác định	42
3.1.7	Xử lý thực thể trùng lặp trong văn bản học thuật .	44
3.1.8	Quản lý và duy trì bền vững hệ thống	49
3.2	Mô hình hóa trạng thái hệ thống (Agent State)	49
3.2.1	Kiến trúc các tác vụ (Agents)	50
3.2.2	Quy trình điều phối và Luồng dữ liệu	51
3.2.3	Cơ chế truy xuất tri thức (LightRAG Integration) .	52
3.2.4	Thuật toán điều phối (Pseudocode)	55

3.3	Thiết lập thực nghiệm	56
3.3.1	Thiết lập thực nghiệm phân hệ trích xuất (MinerU)	56
3.3.2	Thiết lập thực nghiệm Hệ thống Đồ thị tri thức và Đa tác nhân	57
4	Cài đặt thử nghiệm và kết quả	61
4.1	Đánh giá chất lượng MinerU trên tập tài liệu đầu vào . . .	61
4.2	Đánh giá chất lượng nội tại của Đồ thị tri thức (Knowledge Graph)	62
4.2.1	Phân tích và Nhận xét kết quả	63
4.3	Đánh giá chất lượng truy xuất và sinh văn bản (RAG Metrics)	64
4.4	Đánh giá hiệu năng điều phối đa tác nhân (Agentic Metrics)	65
4.5	Đánh giá tài nguyên triển khai và ước lượng khả năng chịu tải	66
4.5.1	Môi trường triển khai và hiện trạng sử dụng tài nguyên	66
4.5.2	Phương pháp và kết quả ước lượng khả năng chịu tải	67
4.5.3	Nhận xét và đề xuất tối ưu hóa	70
4.6	Đánh giá hệ thống từ phía người dùng cuối	70
4.6.1	Đánh giá trải nghiệm của người dùng	70
4.7	Thiết kế và xây dựng ứng dụng	76
4.7.1	Ý tưởng ứng dụng	76
4.7.2	Phân tích và đặc tả yêu cầu	77
4.7.3	Mô hình Use Case	78
4.7.4	Kiến trúc hệ thống	82
4.7.5	Thiết kế chi tiết các thành phần	85
4.7.6	Thiết kế giao diện	94
5	Kết luận	103
5.1	Kết quả đạt được	103
5.2	Hạn chế của hệ thống	103
5.3	Hướng phát triển trong tương lai	104
	Tài liệu tham khảo	106

Danh sách viết tắt

STT	Viết tắt	Viết đầy đủ	Chú thích
1	AI	Artificial Intelligence	Trí tuệ nhân tạo, liên quan đến tự động hóa hành vi thông minh.
2	API	Application Programming Interface	Là cơ chế giao tiếp giữa backend và frontend bằng một tập hợp các định nghĩa và giao thức.
3	BLEU	Bilingual Evaluation Understudy	Độ đo đánh giá chất lượng văn bản sinh ra bằng máy.
4	CER	Character Error Rate	Tỷ lệ lỗi ký tự, thường dùng đánh giá hệ thống nhận dạng.
5	CSDL	Cơ sở dữ liệu	Tập hợp các dữ liệu được tổ chức có cấu trúc để lưu trữ.
6	HCMUS	Ho Chi Minh City University of Science	Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM.
7	HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản dùng tạo trang web.
8	JSON	JavaScript Object Notation	Định dạng hoán đổi dữ liệu nhẹ, dễ đọc.

STT	Viết tắt	Viết đầy đủ	Chú thích
9	KG	Knowledge Graph	Đồ thị tri thức, biểu diễn dữ liệu dạng thực thể và quan hệ.
10	KV	Key-Value	Mô hình lưu trữ dữ liệu theo từng cặp khóa - giá trị.
11	LLM	Large Language Model	Mô hình ngôn ngữ lớn, được huấn luyện trên dữ liệu khổng lồ.
12	MFR	Mathematical Formula Recognition	Nhận diện và trích xuất công thức toán học từ tài liệu.
13	OCR	Optical Character Recognition	Nhận dạng ký tự quang học từ hình ảnh sang văn bản.
14	PDF	Portable Document Format	Định dạng tài liệu di động phổ biến độc lập nền tảng.
15	RAG	Retrieval-Augmented Generation	Sinh văn bản tăng cường truy xuất kết hợp LLM và tìm kiếm.
16	RAGAS	Retrieval-Augmented Generation Assessment	Khung đánh giá tự động chuyên biệt cho hệ thống RAG.
17	SHACL	Shapes Constraint Language	Ngôn ngữ xác thực các ràng buộc cấu trúc cho đồ thị RDF.

STT	Viết tắt	Viết đầy đủ	Chú thích
18	TEDS	Tree-Edit-Distance-based Similarity	Độ đo tương đồng cấu trúc bảng dựa trên khoảng cách cây.
19	UI/UX	User Interface / User Experience	Giao diện người dùng và Trải nghiệm người dùng.
20	VLM	Vision-Language Model	Mô hình đa phương thức xử lý hình ảnh và ngôn ngữ.
21	WER	Word Error Rate	Tỷ lệ lỗi từ, dùng đánh giá độ chính xác của văn bản.

Danh sách hình

3.1	Quy trình tổng quát từ văn bản thô đến đồ thị tri thức . .	22
3.2	Quy trình Hybrid-Auto-Engine của MinerU[25]	26
3.3	Luồng logic kiểm tra trùng lặp bằng cơ chế băm nội dung .	28
3.4	Quy trình thu thập tri thức từ trang web với Smart Scraper	28
3.5	Luồng logic quy trình trích xuất tri thức và cơ chế Gleaning	38
3.6	Luồng điều phối lời nhắc linh hoạt theo miền tri thức (Dynamic Prompt Flow)	41
3.7	Quy trình kiểm chứng tri thức SHACL và đối chiếu bằng chứng xác định	44
3.8	Kiến trúc hai giai đoạn của module phân giải thực thể. Giai đoạn Candidate Retrieval sử dụng embedding similarity để truy hồi các thực thể ngữ nghĩa tương đồng từ vector store. Giai đoạn LLM Decision chỉ được kích hoạt khi có ứng viên vượt ngưỡng τ , giúp tiết kiệm chi phí gọi LLM cho những thực thể hoàn toàn mới.	45
3.9	Ví dụ cấu trúc knowledge graph sau khi áp dụng phân giải thực thể trên corpus học thuật HCMUS. Các nút viền đậm, màu xám đậm là thực thể dạng chuẩn (<i>canonical</i>); các nút màu trắng là hình thức tên thay thế (<i>alias</i>) được liên kết về nút chuẩn qua quan hệ SAME_AS (đường nét đứt). Quan hệ ngữ nghĩa thông thường (mũi tên nét liền) chỉ xuất phát từ nút chuẩn, đảm bảo tri thức không bị phân tán.	47
3.10	Sơ đồ kiến trúc điều phối Agent tổng thể của trợ lý ảo . .	52
3.11	Cơ chế truy xuất tri thức và lọc ngữ cảnh hai tầng trong LightRAG	54
4.1	Kết quả khảo sát về chất lượng câu trả lời và so sánh hai chế độ phục vụ	72

4.2	Kết quả khảo sát về trải nghiệm hệ thống và mức độ hài lòng tổng thể	74
4.3	Các phản hồi đóng góp ý kiến tự do từ người dùng khảo sát	75
4.4	Sơ đồ Use Case hệ thống	79
4.5	Mô hình kiến trúc hệ thống tổng thể	83
4.6	Mô hình sơ đồ cơ sở dữ liệu quan hệ người dùng	86
4.7	Mô hình khái niệm các loại thực thể và quan hệ chính trong đồ thị tri thức học vụ	88
4.8	Ví dụ subgraph trích xuất từ đề cương môn học trong Knowledge Graph	91
4.9	Tổng quan kiến trúc lưu trữ và triển khai LightRAG . . .	92
4.10	Giao diện Màn hình Đăng ký tài khoản trên hai nền tảng .	94
4.11	Giao diện Màn hình Đăng nhập hệ thống trên hai nền tảng	95
4.12	Giao diện Màn hình Quên mật khẩu trên hai nền tảng . .	97
4.13	Giao diện Màn hình Hồ sơ học tập cá nhân trên hai nền tảng	99
4.14	Giao diện trò chuyện AI Chat trên nền tảng Website . . .	100
4.15	Giao diện Màn hình AI Chatbot trên thiết bị di động . . .	101

Danh sách bảng

2.1	Tỷ lệ chiến thắng (%) đối đầu giữa các mô hình baseline và LightRAG trên bốn tập dữ liệu (trích dẫn từ bài báo gốc LightRAG [10])	13
3.1	Bảng tổng hợp các mô hình thành phần trong MinerU (Version 3.x+)	24
3.2	So sánh hai chiến lược phân đoạn trong hệ thống	34
3.3	So sánh các thành phần thiết kế với công trình liên quan .	48
4.1	Kết quả chi tiết đánh giá MinerU trên các tập tài liệu Chương trình đào tạo (2022-2025)	62
4.2	Kết quả đánh giá nội tại của Đồ thị tri thức	63
4.3	Kết quả thực nghiệm chất lượng truy xuất và sinh văn bản theo khung RAGAS	65
4.4	Hiện trạng sử dụng tài nguyên của hệ thống trên máy chủ VPS	67
4.5	Năng lực phục vụ thực tế của hệ thống theo từng kịch bản cấu hình (đã áp dụng hệ số suy hao 30–40%)	69
4.6	Danh sách yêu cầu chức năng	77
4.7	Yêu cầu phi chức năng	78
4.8	Danh sách Actor	78
4.9	Danh sách Use Case	79
4.13	Danh sách các bảng trong cơ sở dữ liệu quan hệ	86
4.14	Mô tả khóa ngoại của các bảng trong CSDL quan hệ . . .	87
4.15	Danh sách các Collection trong cơ sở dữ liệu hcmus_langgraph	87
4.16	Thuộc tính của node thực thể trong Knowledge Graph (Neo4j)	90
4.17	Thuộc tính của relationship trong Knowledge Graph (Neo4j)	91
4.18	Các thành phần lưu trữ trong kiến trúc triển khai LightRAG	93

Tóm tắt

Nhiều sinh viên của Trường Đại học Khoa học Tự nhiên – ĐHQG TP.HCM gặp khó khăn trong việc tiếp cận và tổng hợp các thông tin học tập do các nguồn thông tin thường phân tán ở nhiều hệ thống và tài liệu khác nhau. Điều này khiến quá trình tìm kiếm, tra cứu và nắm bắt các quy định học vụ, chương trình đào tạo hay thông tin về học phần trở nên mất nhiều thời gian và công sức. Hệ thống hỗ trợ cung cấp thông tin học tập được đề xuất nhằm tập trung và tổ chức các nguồn dữ liệu này thành một nền tảng thống nhất, giúp sinh viên dễ dàng tra cứu và tiếp cận thông tin học tập một cách nhanh chóng, chính xác và thuận tiện.

Hệ thống áp dụng Đồ thị tri thức (Knowledge Graph) để lưu trữ thông tin học vụ, kết hợp Tác tử trí tuệ nhân tạo (AI Agents) để chuyển hóa tri thức thành đồ thị và hỗ trợ tìm kiếm thông tin học tập theo nhu cầu của người dùng. Về mặt phương pháp, nghiên cứu cải tiến quy trình trích xuất đồ thị tri thức bằng mô hình biểu diễn bộ tứ (Quadruple Representation) kết hợp với luồng tối ưu hóa lời nhắc (Prompt Flow) dành riêng cho các tài liệu chuyên biệt. Hệ thống áp dụng kỹ thuật phân mảnh văn bản dựa trên vector ngữ nghĩa (Chunk Vector Section) để duy trì tính toàn vẹn của ngữ cảnh, đồng thời tích hợp cơ chế tự động giải quyết và đồng nhất các từ viết tắt liên quan. Để đánh giá, nghiên cứu tiến hành đánh giá trên hai phương diện: chất lượng Đồ thị tri thức (Knowledge Graph) và hiệu năng của hệ thống Truy xuất tăng cường sinh (Retrieval-Augmented Generation). Quá trình này sử dụng phương pháp mô hình ngôn ngữ lớn làm giám khảo (LLM-as-a-Judge) kết hợp với khung đánh giá RAGAS (Retrieval-Augmented Generation Assessment). Kết quả thực nghiệm trên Đồ thị tri thức cho thấy kiến trúc đề xuất đã cải thiện đáng kể chất lượng liên kết, khắc phục thành công và làm giảm mạnh tỷ lệ nút mồ côi (dangling nodes). Bên cạnh đó, các tiêu chí đánh giá RAG về độ liên quan và độ phủ ngữ cảnh cũng đạt mức cao, chứng minh khả năng cung cấp thông tin học tập chính xác, đầy đủ và hiệu quả trên nền tảng ứng dụng.

Chương 1

Giới thiệu

1.1 Đặt vấn đề

Trong bối cảnh hiện nay, sinh viên ngành Công nghệ thông tin tại Trường Đại học Khoa học Tự nhiên đang phải đối mặt với một lượng kiến thức cực kỳ lớn và rời rạc. Tuy tài liệu có sẵn rất nhiều trên mạng nhưng các tài liệu này thường tồn tại ở dạng các mẫu tin riêng biệt, khiến người học khó có thể tự mình kết nối các nguồn tài liệu này lại thành một thông tin tổng hợp học tập hoàn chỉnh. Các công cụ tìm kiếm thông thường hiện nay chỉ giúp tìm ra thông tin của một môn học cụ thể chứ không thể chỉ ra mối quan hệ giữa các môn hay giải thích tại sao kiến thức này lại cần thiết cho kiến thức kia. Thêm vào đó, các mô hình trí tuệ nhân tạo phổ biến đôi khi vẫn cung cấp thông tin không chính xác hoặc không hiểu rõ các quy định và chương trình học đặc thù của nhà trường [16].

Bên cạnh đó, các hệ thống hỗ trợ học vụ hiện có tại các trường đại học Việt Nam chủ yếu hoạt động dựa trên cổng thông tin tĩnh hoặc chatbot dựa trên quy tắc với bộ từ khóa được lập trình sẵn [36]. Sinh viên phải tự tra cứu thủ công từ nhiều nguồn rời rạc như file PDF đề cương, trang web khoa và hệ thống đào tạo. Điều này dẫn đến tình trạng phân mảnh thông tin, khiến sinh viên khó hình dung bức tranh toàn cảnh của mạng lưới môn học và dễ bị mất phương hướng khi tự lập kế hoạch học tập.

Vì những lý do đó, đề tài này được ra đời nhằm xây dựng một trợ lý học tập thông minh có khả năng hiểu sâu về lộ trình và tài liệu của sinh viên. Việc giải quyết vấn đề này là cần thiết vì định hướng đúng đắn ngay từ đầu sẽ giúp sinh viên tiết kiệm rất nhiều thời gian và công sức trong suốt quá trình theo học tại trường.

1.2 Mục tiêu

Mục tiêu của đề tài là tạo ra một trợ lý ảo có thể trả lời chính xác các thắc mắc về học vụ, đồng thời hiểu được sự liên kết logic giữa các khái niệm khác nhau trong chương trình đào tạo. Để đạt được mục đích đó, nghiên cứu đặt ra ba mục tiêu cụ thể.

Đầu tiên, nghiên cứu xây dựng đồ thị tri thức (Knowledge Graph) từ kho dữ liệu học vụ của Khoa Công nghệ Thông tin bằng hệ thống LightRAG [10], kết hợp cơ chế kiểm chứng cấu trúc đồ thị dựa trên chuẩn SHACL [32] nhằm đảm bảo tính đúng đắn và liên kết chặt chẽ của tri thức được biểu diễn.

Tiếp theo, nghiên cứu phát triển kiến trúc đa tác nhân (Multi-Agent) thông qua nền tảng LangGraph [19] để xử lý các truy vấn học tập phức tạp. Hệ thống đa tác nhân cho phép AI tự động phân tích yêu cầu, lựa chọn công cụ truy xuất phù hợp và phối hợp nhiều bước suy luận nhằm đưa ra câu trả lời toàn diện.

Cuối cùng, nghiên cứu xây dựng pipeline đánh giá tự động kết hợp bộ chỉ số RAGAS [8] và phương pháp LLM-as-a-Judge [37] để đo lường chất lượng truy xuất, độ trung thực của câu trả lời và hiệu năng điều phối đa tác nhân một cách khách quan và có hệ thống.

1.3 Phạm vi đề tài

Nghiên cứu này tập trung chủ yếu vào sinh viên Công nghệ thông tin tại Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh. Dữ liệu được sử dụng bao gồm chương trình đào tạo của các ngành thuộc Khoa Công nghệ Thông tin, đề cương chi tiết các môn học cơ sở và chuyên ngành, cùng các quy chế học vụ hiện hành. Phạm vi hệ thống giới hạn ở việc hỗ trợ trả lời các câu hỏi về cách học, mối quan hệ giữa các môn học và lộ trình phát triển chuyên môn trong khuôn khổ chương trình đào tạo của Khoa, chưa mở rộng ra toàn trường hay các khoa khác.

1.4 Giải pháp và cách tiếp cận

Đề tài lựa chọn hướng giải quyết thông qua việc kết hợp giữa kiến trúc đa tác nhân (Multi-Agent) và phương pháp tổ chức dữ liệu theo dạng đồ thị tri thức (Knowledge Graph). Cụ thể, quy trình xử lý bao gồm bốn giai đoạn chính. Giai đoạn đầu tiên là tiền xử lý tài liệu, trong đó các tài liệu PDF đề cương và quy chế học vụ được số hóa và trích xuất nội dung thông qua hệ thống MinerU [25]. Giai đoạn thứ hai là xây dựng đồ thị tri thức bằng LightRAG [10], nơi các thực thể (môn học, kỹ năng, khối kiến thức) và mối quan hệ giữa chúng được trích xuất và biểu diễn dưới dạng đồ thị, kết hợp cơ chế kiểm chứng cấu trúc bằng chuẩn SHACL [32]. Giai đoạn thứ ba là truy vấn đa cấp, trong đó hệ thống thực hiện song song việc khớp từ khóa, tìm kiếm vector và truy vấn đồ thị để bao quát ngữ cảnh từ chi tiết đến tổng thể. Giai đoạn cuối cùng là điều phối đa tác nhân thông qua LangGraph [19], nơi các AI Agent tự động phân tích câu hỏi, lựa chọn chiến lược truy xuất và tổng hợp câu trả lời phù hợp nhất.

Bằng cách xây dựng một bản đồ kiến thức chặt chẽ, hệ thống sẽ khắc phục được tình trạng trả lời thiếu chiều sâu hoặc thông tin không liên quan mà các phương pháp truyền thống thường gặp phải.

1.5 Đóng góp

Nghiên cứu này mang lại ba đóng góp chính, hướng đến việc tạo ra một nền tảng tri thức tin cậy, nơi mọi thông tin học tập được tổng hợp và kết nối một cách logic nhất.

Đầu tiên, nghiên cứu xây dựng Đồ thị tri thức (Knowledge Graph) từ kho dữ liệu học vụ của Khoa Công nghệ Thông tin thông qua hệ thống LightRAG. Điểm nổi bật của nghiên cứu là tích hợp cơ chế kiểm chứng dựa trên chuẩn SHACL nhằm đảm bảo tính đúng đắn của cấu trúc đồ thị và sự liên kết giữa các thực thể, đồng thời đề xuất giải pháp xử lý thực thể trùng lặp (Entity Resolution) phù hợp với ngữ cảnh tài liệu học thuật tiếng Việt.

Tiếp theo, nghiên cứu đề xuất và triển khai kiến trúc đa tác nhân (Multi-Agent) trên nền tảng LangGraph, cho phép hệ thống tự động phân

tích yêu cầu của người dùng, lựa chọn công cụ truy xuất phù hợp và phối hợp nhiều tác nhân để xử lý các truy vấn liên quan đến thông tin học vụ một cách hiệu quả.

Cuối cùng, xây dựng khung đánh giá tự động kết hợp bộ chỉ số RAGAS (đo lường độ trung thực, độ liên quan, độ chính xác và độ bao phủ của ngữ cảnh) với phương pháp LLM-as-a-Judge, cho phép đo lường và tinh chỉnh chất lượng hệ thống một cách khách quan mà không phụ thuộc hoàn toàn vào đánh giá thủ công.

1.6 Bố cục

Nội dung của báo cáo được tổ chức thành năm chương như sau.

Chương 1: Giới thiệu. Trình bày bối cảnh và động lực nghiên cứu, xác định mục tiêu, phạm vi, giải pháp đề xuất cùng những đóng góp chính của đề tài.

Chương 2: Các công trình liên quan. Tổng quan về các hệ thống hỗ trợ học tập thông minh, phân tích các hệ thống tương tự đã triển khai, cơ sở lý thuyết về Mô hình ngôn ngữ lớn (LLM), kiến trúc RAG, GraphRAG, Knowledge Graph, hệ thống LightRAG và kiến trúc đa tác nhân (Multi-Agent).

Chương 3: Phương pháp đề xuất. Trình bày chi tiết kiến trúc tổng thể của hệ thống và các phương pháp được đề xuất, bao gồm quy trình xử lý dữ liệu, chiến lược phân mảnh văn bản (Chunking Strategy), phương pháp xây dựng Đồ thị tri thức (Knowledge Graph), cơ chế kiểm chứng bằng SHACL, giải pháp xử lý thực thể trùng lặp (Entity Resolution) và kiến trúc điều phối đa tác nhân (Multi-Agent).

Chương 4: Cài đặt thử nghiệm và kết quả. Trình bày kết quả đánh giá chất lượng xử lý dữ liệu, chất lượng đồ thị tri thức, hiệu năng truy xuất và sinh văn bản, hiệu năng điều phối đa tác nhân, đánh giá từ người dùng cuối, cùng thiết kế và xây dựng ứng dụng hoàn chỉnh.

Chương 5: Kết luận. Tổng kết các kết quả đạt được, nêu ra những hạn chế còn tồn tại và định hướng phát triển trong tương lai.

Chương 2

Các công trình liên quan

2.1 Tổng quan về hệ thống hỗ trợ học tập thông minh

Trong môi trường giáo dục đại học, khối lượng kiến thức và cấu trúc chương trình đào tạo ngày càng trở nên phức tạp. Việc xây dựng một lộ trình học tập tối ưu không chỉ dừng lại ở việc hoàn thành các tín chỉ, mà còn phải đáp ứng đúng định hướng nghề nghiệp và năng lực cốt lõi của từng cá nhân. Các hệ thống hỗ trợ học tập thông minh (Intelligent Tutoring/Learning Support Systems) ra đời nhằm giải quyết bài toán này, đóng vai trò như một cầu nối thông tin giữa nhà trường, chương trình đào tạo và sinh viên [36].

2.1.1 Các hướng tiếp cận truyền thống

Trước khi có sự bùng nổ của Trí tuệ Nhân tạo và các Mô hình Ngôn ngữ Lớn (LLMs), các hệ thống hỗ trợ học vụ tại các trường đại học chủ yếu hoạt động dựa trên các quy tắc tĩnh (rule-based) và kiến trúc dữ liệu phẳng (flat data representations). Một số đặc điểm và hạn chế của các hệ thống này có thể được phân tích qua ba khía cạnh chính. Một trong những điểm yếu rõ rệt nhất nằm ở các hệ thống cổng thông tin tĩnh (Static Portals). Tại đây, thông tin về chương trình đào tạo, đề cương chi tiết (syllabus), môn học tiên quyết và song hành thường được cung cấp dưới dạng các tệp văn bản hoặc PDF rời rạc. Do sinh viên phải tự tra cứu thủ công, điều này dẫn đến tình trạng phân mảnh thông tin và khiến họ khó hình dung được bức tranh toàn cảnh của mạng lưới môn học. Bên cạnh đó, các chatbot dựa trên quy tắc (Rule-based Chatbots) thế hệ cũ thường chỉ hoạt động dựa trên bộ từ khóa (keywords) và các kịch bản hội thoại được lập trình sẵn. Do đó, hệ thống này gặp hạn chế lớn trong việc xử lý ngôn ngữ tự nhiên, hoàn toàn không hiểu được ngữ cảnh tổng thể hay các truy vấn phức tạp

của sinh viên. Ngoài ra, các phương pháp truyền thống còn thiếu khả năng mô hình hóa các mối quan hệ đa chiều, dẫn đến hạn chế trong việc liên kết dữ liệu. Việc kết nối giữa điểm số, sở thích cá nhân và môn học hầu như không được tự động hóa. Hậu quả là làm mất đi tính cá nhân hóa, khiến sinh viên dễ bị mất phương hướng khi tự lập kế hoạch học tập.

2.1.2 Ứng dụng AI trong giáo dục hiện đại

Sự phát triển của Trí tuệ Nhân tạo (AI), đặc biệt là Xử lý Ngôn ngữ Tự nhiên (NLP) và Học máy (Machine Learning), đã tạo ra những bước nhảy vọt trong việc phát triển các hệ thống giáo dục thông minh. Thay vì chỉ cung cấp thông tin thụ động, các hệ thống hiện đại đã chuyển dịch sang hướng tương tác và cá nhân hóa sâu sắc:

Tiêu biểu là các hệ thống gợi ý và cá nhân hóa (Recommender Systems in Education). Tại đây, AI cho phép phân tích dữ liệu lịch sử học tập (điểm số, tiến độ) kết hợp với dữ liệu phi cấu trúc (nguyện vọng, mục tiêu nghề nghiệp) nhằm đưa ra các gợi ý môn học phù hợp nhất. Nhờ đó, các thuật toán này giúp sinh viên vạch ra lộ trình học tập linh hoạt theo từng học kỳ, bám sát với định hướng chuyên môn. Song song đó là sự chuyển đổi mạnh mẽ từ Chatbot sang AI Agents (Tác nhân AI). Trái ngược với chatbot thông thường chỉ phản hồi truy vấn đơn lẻ, AI Agents (đặc biệt là kiến trúc Multi-Agent) được trang bị khả năng tự động hóa chuỗi suy luận, giúp giải quyết các yêu cầu phức tạp hơn. Hơn nữa, việc ứng dụng Đồ thị hóa tri thức giáo dục (Educational Knowledge Graphs) cũng đang trở thành xu hướng. Nhằm khắc phục tình trạng AI tạo ra thông tin sai lệch (“ảo giác” - hallucination) và thiếu ngữ cảnh, các nghiên cứu hiện đại đang tích cực kết hợp AI với Đồ thị tri thức [13]. Vì việc số hóa và biểu diễn cấu trúc chương trình đào tạo dưới dạng các thực thể và mối quan hệ sẽ giúp hệ thống có khả năng truy xuất thông tin đa chiều, AI từ đó có thể giải thích rõ ràng lý do tại sao một môn học lại cần thiết.

2.2 Các hệ thống tương tự

Để xác định rõ khoảng trống nghiên cứu và vị trí đóng góp của đề tài, phần này phân tích một số hệ thống hỗ trợ tư vấn học vụ tiêu biểu đã được triển khai trên thế giới và tại Việt Nam.

2.2.1 Các hệ thống tư vấn học vụ quốc tế

Pounce là chatbot AI tiên phong trong lĩnh vực tư vấn học vụ đại học, được Georgia State University (Mỹ) phát triển từ năm 2016 cùng nền tảng Mainstay (trước đây là AdmitHub) [26]. Hệ thống tự động nhắc nhở sinh viên về các mốc quan trọng như nộp hồ sơ tài chính (FAFSA), đăng ký ký túc xá và đăng ký môn học. Nghiên cứu đã chứng minh Pounce giúp giảm đáng kể tỷ lệ “summer melt” (sinh viên được nhận nhưng không nhập học) và cải thiện tỷ lệ duy trì học tập. Tuy nhiên, Pounce hoạt động chủ yếu dựa trên kịch bản hội thoại có sẵn và dữ liệu hành chính, không xây dựng đồ thị tri thức về mối quan hệ giữa các môn học nên không có khả năng suy luận về lộ trình học tập hay giải thích tại sao một môn học lại cần thiết cho môn khác.

Hệ thống tư vấn học thuật tại UAEU (United Arab Emirates University, 2024) đại diện cho hướng tiếp cận hiện đại hơn, sử dụng cơ sở dữ liệu đồ thị Neo4j để mô hình hóa mối quan hệ giữa các môn học, điều kiện tiên quyết và tiến trình học tập của sinh viên [2]. Mô hình ngôn ngữ lớn (LLM) chuyển đổi câu hỏi bằng ngôn ngữ tự nhiên thành truy vấn Cypher để trích xuất thông tin chính xác từ đồ thị. Kết quả thử nghiệm cho thấy lộ trình do hệ thống gợi ý đạt độ tương đồng lên tới 80% so với lộ trình thực tế của các sinh viên đã tốt nghiệp thành công. Mặc dù vậy, hệ thống này chưa tích hợp kiến trúc đa tác nhân (Multi-Agent) để xử lý các truy vấn phức tạp đòi hỏi phối hợp nhiều bước suy luận, đồng thời chưa có cơ chế kiểm chứng tự động cấu trúc đồ thị nhằm đảm bảo tính nhất quán của tri thức.

2.2.2 Các hệ thống tư vấn tại Việt Nam

UEH AI Chatbot (Đại học Kinh tế TP.HCM) là một trong những chatbot đại học đầu tiên tại Việt Nam, được triển khai nhằm hỗ trợ giải

đáp thắc mắc về ngành nghề, quy trình học tập và thông tin tuyển sinh phục vụ 24/7 cho sinh viên [1]. Hệ thống hoạt động chủ yếu theo mô hình hỏi đáp dựa trên kỹ thuật RAG đơn giản, truy xuất thông tin từ kho tài liệu nội bộ của trường. Tuy nhiên, UEH AI Chatbot chưa có khả năng suy luận đa bước về mối liên kết giữa các môn học trong chương trình đào tạo, chưa xây dựng đồ thị tri thức để biểu diễn mạng lưới kiến thức và chưa hỗ trợ cá nhân hóa theo tiến trình học tập cụ thể của từng sinh viên.

Ngoài ra, phần lớn các trường đại học tại Việt Nam hiện nay, bao gồm cả Trường Đại học Khoa học Tự nhiên (HCMUS), chủ yếu cung cấp thông tin chương trình đào tạo, đề cương môn học và quy chế học vụ thông qua các cổng thông tin trực tuyến dạng tĩnh. Sinh viên phải tự tra cứu thủ công từ nhiều nguồn rời rạc như file PDF, trang web của khoa và hệ thống quản lý đào tạo. Mô hình này hoàn toàn thiếu cơ chế tương tác, không có khả năng suy luận hay tổng hợp thông tin tự động, khiến sinh viên gặp nhiều khó khăn trong việc hình dung bức tranh tổng thể về lộ trình học tập.

2.3 Mô hình ngôn ngữ lớn (LLMs) trong giáo dục

2.3.1 Ứng dụng LLM trong hệ thống hỏi đáp học tập

Với khả năng hiểu và sinh ngôn ngữ tự nhiên, LLMs đã nhanh chóng được tích hợp vào các hệ thống giáo dục, đặc biệt là trong các hệ thống hỏi đáp (Q&A):

Một trong những ứng dụng nổi bật là khả năng giao tiếp bằng ngôn ngữ tự nhiên (Natural Language Interface). Thay vì phải sử dụng các từ khóa cứng nhắc hay câu lệnh phức tạp, sinh viên hoàn toàn có thể đặt câu hỏi theo cách giao tiếp thông thường (ví dụ: “Nếu em trượt môn Cơ sở dữ liệu thì kỳ sau em có được học Hệ quản trị cơ sở dữ liệu không?”). Nhờ vào LLMs, hệ thống có thể hiểu chính xác ý định và đưa ra câu trả lời trực tiếp. Đồng thời, LLMs còn đóng vai trò đắc lực trong việc hỗ trợ giải thích và tóm tắt kiến thức. Với khả năng phân tích các đề cương môn học phức tạp, mô hình có thể tóm tắt lại những kỹ năng cốt lõi mà sinh viên sẽ đạt được. Hơn thế nữa, mô hình có thể đóng vai trò như một người hướng dẫn,

giải thích các khái niệm học thuật bằng nhiều cách diễn đạt khác nhau. Ngoài ra, tính năng hỗ trợ sinh tự động (Generative Assistance) cũng là một điểm cộng lớn. LLMs có thể hỗ trợ sinh viên trong việc phác thảo kế hoạch học tập tạm thời, gợi ý cách viết email trao đổi học thuật với giảng viên, hoặc thậm chí sinh ra các câu hỏi trắc nghiệm để sinh viên tự ôn tập dựa trên tài liệu được cung cấp.

2.3.2 Hạn chế của LLM (Hallucination, thiếu context, v.v.)

Mặc dù sở hữu những khả năng vượt trội, việc triển khai trực tiếp các LLMs thuần túy (vanilla LLMs) vào hệ thống tư vấn học vụ gặp phải những rủi ro và rào cản, đặc biệt là trong môi trường đại học yêu cầu tính chính xác tuyệt đối:

Nguy cơ nghiêm trọng nhất là hiện tượng ảo giác AI (Hallucination). Khi không có đủ thông tin hoặc câu hỏi vượt quá dữ liệu huấn luyện, LLM có xu hướng bịa ra các câu trả lời nghe có vẻ rất hợp lý nhưng thực chất lại sai sự thật [16]. Nếu hệ thống AI tự sáng tác ra một môn học tiên quyết không tồn tại hoặc cung cấp sai quy chế học vụ, điều này sẽ dẫn đến hậu quả rất nghiêm trọng, làm sai lệch toàn bộ lộ trình học tập của sinh viên. Một hạn chế khác đến từ việc thiếu ngữ cảnh chuyên biệt và tính cập nhật (Lack of Domain-specific Context & Up-to-date Knowledge). Do LLMs được huấn luyện trên dữ liệu đại trà tính đến một thời điểm cắt (cut-off date) nhất định, các mô hình này không thể tự động nắm bắt những thay đổi mới nhất trong chương trình đào tạo, quy định chuẩn đầu ra, hay danh sách giảng viên phụ trách của nhà trường. Thêm vào đó, LLMs thuần túy bộc lộ sự yếu kém trong suy luận đa bước phức tạp (Complex Multi-hop Reasoning). Để vạch ra một lộ trình học tập, hệ thống cần phải duyệt qua một mạng lưới chằng chịt các điều kiện ràng buộc (ví dụ: môn A là tiên quyết của môn B, môn B lại học song hành với môn C). Vì thế, nếu thông tin chỉ được biểu diễn dưới dạng văn bản phẳng, mô hình sẽ gặp rất nhiều rào cản trong việc xâu chuỗi các mối quan hệ đa tầng này. Một rào cản nữa là sự khó khăn trong việc xác minh nguồn gốc (Lack of Traceability). Các câu trả lời được LLMs sinh ra tự động thường không đi kèm với nguồn trích dẫn rõ ràng, khiến người dùng không biết thông tin được lấy từ văn

bản quy chế nào. Hệ quả là điều này làm giảm đi đáng kể độ tin cậy của hệ thống trong mắt sinh viên và giảng viên.

2.4 Retrieval-Augmented Generation (RAG) truyền thống

Kiến trúc sinh văn bản tăng cường truy xuất (Retrieval-Augmented Generation - RAG) [20] là một bước đột phá giúp giải quyết các điểm yếu lớn của mô hình ngôn ngữ lớn như hiện tượng ảo giác và sự thiếu hụt tri thức cập nhật ngoài tập dữ liệu huấn luyện. RAG hoạt động bằng cách bổ sung một bộ nhớ ngoài phi tham số chứa dữ liệu thực tế, giúp mô hình ngôn ngữ lớn truy xuất ngữ cảnh chính xác trước khi tạo lập câu trả lời. Trong mô hình RAG truyền thống (Naive RAG), quy trình được chia thành ba giai đoạn chính bao gồm lập chỉ mục, truy xuất và sinh văn bản.

Trải qua quá trình đánh giá thực tiễn, mô hình RAG truyền thống sở hữu một số ưu điểm đáng ghi nhận. Nổi bật nhất là khả năng giảm thiểu hiện tượng ảo giác nhờ ép buộc mô hình ngôn ngữ lớn chỉ trả lời dựa trên tài liệu được cung cấp. Bên cạnh đó, giải pháp này có chi phí bảo trì tương đối thấp và không đòi hỏi tinh chỉnh lại mô hình khi dữ liệu thay đổi. Đồng thời, khả năng xác minh nguồn gốc thông tin cũng được đảm bảo nhờ việc hệ thống hỗ trợ trích dẫn trực tiếp các đoạn văn bản nguồn. Tuy nhiên, kiến trúc này vẫn tồn tại những hạn chế cốt lõi. Đầu tiên, việc biểu diễn dữ liệu phẳng làm chia cắt các mối liên kết logic phức tạp trong tài liệu thành các khối rời rạc. Tiếp theo, hệ thống thường gặp thất bại trong suy luận đa bước khi câu hỏi yêu cầu tổng hợp thông tin liên kết chéo giữa nhiều tài liệu khác nhau. Cuối cùng, hiện tượng bỏ sót thông tin vẫn diễn ra nếu từ khóa truy vấn của người dùng không khớp về mặt ngữ nghĩa cục bộ với các khối dữ liệu trong cơ sở dữ liệu vector.

2.5 Các hệ thống RAG dựa trên Đồ thị tri thức

Để khắc phục các hạn chế của RAG truyền thống trong việc xử lý các tài liệu có cấu trúc liên kết phức tạp, phương pháp tích hợp Đồ thị tri thức

(Knowledge Graph - KG) đã được phát triển, mở ra thể hệ RAG dựa trên đồ thị (Graph-based RAG) [13]. Đồ thị tri thức mô hình hóa dữ liệu dưới dạng mạng lưới gồm các thực thể và mối quan hệ giữa chúng, giúp bảo toàn trọn vẹn ngữ nghĩa liên kết đa chiều của tài liệu.

Khi so sánh mô hình biểu diễn dữ liệu (được mô tả trực quan trong sơ đồ kiến trúc so sánh của bài báo gốc LightRAG [10]), sự khác biệt về mặt kỹ thuật giữa các giải pháp RAG hiện nay được thể hiện qua cách thức tổ chức và truy xuất thông tin. Với RAG truyền thống, các phân đoạn văn bản được coi là các vector độc lập trong không gian. Khi thực hiện truy xuất, hệ thống chỉ tìm kiếm các khối văn bản rời rạc có độ tương đồng ngữ nghĩa cao nhất, dẫn đến việc mất mát các kết nối logic và ngữ cảnh tổng quát. Với hệ thống Microsoft GraphRAG [6], các khối văn bản được gom cụm thành các cộng đồng thực thể phân cấp. Hệ thống tiến hành tóm tắt tri thức ở cấp độ từng cộng đồng để phục vụ truy vấn. Phương pháp này giúp nắm bắt thông tin toàn cục rất tốt thông qua các báo cáo cộng đồng, nhưng lại bỏ lỡ các chi tiết cụ thể ở mức độ thực thể đơn lẻ và chịu chi phí tính toán cực kỳ lớn. Đối với hệ thống LightRAG [10], dữ liệu được biểu diễn dưới dạng đồ thị thực thể phẳng kết hợp các mối quan hệ đa chiều. Quy trình truy xuất được thiết kế theo cơ chế hai tầng đọc đảo. Ở tầng thấp, hệ thống truy xuất các thông tin chi tiết về thực thể và các nút lân cận. Ở tầng cao, hệ thống truy xuất và tổng hợp thông tin từ góc độ các mối quan hệ và chủ đề vĩ mô trên toàn đồ thị. Nhờ đó, LightRAG đạt được sự cấu trúc cân bằng tối ưu giữa việc bảo toàn chi tiết cục bộ và bao quát ngữ cảnh toàn cục.

2.6 So sánh các giải pháp RAG hiện nay

Để minh chứng cho các lập luận lý thuyết, nghiên cứu trích dẫn trực tiếp các số liệu thực nghiệm được công bố trong bài báo gốc của LightRAG [10].

Trong bài báo gốc, LightRAG được đặt trong mối tương quan đối đầu trực tiếp với bốn kiến trúc RAG tiên tiến đại diện cho các phương pháp tiếp cận và trường phái thiết kế khác nhau nhằm chứng minh tính hiệu quả vượt trội của mình: Mô hình RAG truyền thống (Naive RAG) đại diện

cho trường phái tìm kiếm dựa trên cơ sở dữ liệu vector phẳng cơ bản. Quy trình vận hành chính bao gồm việc chia văn bản thô thành các đoạn văn ngắn cố định, chuyển đổi chúng thành vector số học bằng mô hình nhúng và lưu trữ vào cơ sở dữ liệu vector. Khi nhận được truy vấn từ người dùng, hệ thống mã hóa câu hỏi thành vector và dùng các thuật toán đo khoảng cách (như độ tương đồng Cosine) để tìm các đoạn văn bản tương đồng nhất. Mặc dù cấu trúc này có hiệu suất tìm kiếm nhanh và chi phí thấp, việc phân đoạn phẳng vô tình cắt đứt liên kết ngữ nghĩa đa chiều trong tài liệu và khiến hệ thống gặp thất bại khi trả lời các câu hỏi suy luận đa bước. Mô hình RQ-RAG (Reasoning Filters for Query-Augmented RAG) tiếp cận bằng cách tối ưu hóa và tinh chỉnh câu hỏi truy vấn của người dùng trước khi tiến hành tìm kiếm trong cơ sở dữ liệu. Nhận thấy câu hỏi gốc từ người dùng thường mang tính mơ hồ, quá ngắn hoặc chứa thông tin không đầy đủ, RQ-RAG sử dụng một mô hình ngôn ngữ lớn làm bộ lọc suy luận để thực hiện các thao tác viết lại câu hỏi, phân rã một truy vấn phức tạp thành nhiều câu hỏi phụ đơn giản hơn, và loại bỏ các yếu tố đa nghĩa. Hệ thống sau đó tiến hành truy xuất ngữ cảnh song song cho các câu hỏi phụ này và tổng hợp thông tin mang về, giúp cải thiện độ chính xác của tài liệu được trả về. Mô hình HyDE (Hypothetical Document Embeddings) giải quyết bài toán không tương đồng không gian vector giữa câu hỏi của người dùng và câu trả lời trong cơ sở dữ liệu. Thông thường, một câu hỏi ngắn và một câu trả lời chi tiết sẽ nằm cách xa nhau trong không gian nhúng. HyDE khắc phục điều này bằng cách sử dụng mô hình ngôn ngữ lớn để sinh ra một tài liệu giả định (chứa câu trả lời dự đoán) dựa trên truy vấn đầu vào. Mặc dù tài liệu giả định này có thể chứa thông tin không chính xác về mặt thực tế, nó lại mang văn phong và phân bố ngữ nghĩa rất gần với các tài liệu thật trong cơ sở dữ liệu. Vector của tài liệu giả định này sau đó được sử dụng làm truy vấn chính để tìm kiếm tài liệu thật trong Vector Database, giúp cải thiện đáng kể hiệu năng tìm kiếm ngữ nghĩa. Mô hình Microsoft GraphRAG đại diện cho hướng đi tăng cường đồ thị tri thức kết hợp tóm tắt phân cấp. Quy trình bắt đầu bằng việc sử dụng mô hình ngôn ngữ lớn để trích xuất toàn bộ các thực thể, mối quan hệ và các đoạn mô tả tương ứng từ văn bản nguồn, xây dựng nên một đồ thị tri thức chi tiết. Hệ thống sau đó áp dụng các thuật toán phát hiện cộng đồng để

phân cụm đồ thị này thành các cụm cộng đồng phân cấp từ vĩ mô đến vi mô, đồng thời dùng mô hình ngôn ngữ lớn viết báo cáo tóm tắt cho từng cộng đồng. Khi người dùng truy vấn, GraphRAG duyệt qua các báo cáo tóm tắt cộng đồng này thay vì các khối dữ liệu thô, giúp trả lời các câu hỏi toàn cục rất tốt nhưng phải đánh đổi bằng chi phí xây dựng chỉ mục vô cùng đắt đỏ và số lượng token tiêu tốn cực kỳ lớn.

Kết quả đánh giá đối đầu (Win Rate

Bảng 2.1: Tỷ lệ chiến thắng (%) đối đầu giữa các mô hình baseline và LightRAG trên bốn tập dữ liệu (trích dẫn từ bài báo gốc LightRAG [10])

Tiêu chí đánh giá	Agriculture		CS		Legal		Mix	
	Baseline	LightRAG	Baseline	LightRAG	Baseline	LightRAG	Baseline	LightRAG
<i>So sánh với Naive RAG</i>								
Comprehensiveness	32,4%	67,6%	38,4%	61,6%	16,4%	83,6%	38,8%	61,2%
Diversity	23,6%	76,4%	38,0%	62,0%	13,6%	86,4%	32,4%	67,6%
Empowerment	32,4%	67,6%	38,8%	61,2%	16,4%	83,6%	42,8%	57,2%
Overall	32,4%	67,6%	38,8%	61,2%	15,2%	84,8%	40,0%	60,0%
<i>So sánh với RQ-RAG</i>								
Comprehensiveness	31,6%	68,4%	38,8%	61,2%	15,2%	84,8%	39,2%	60,8%
Diversity	29,2%	70,8%	39,2%	60,8%	11,6%	88,4%	30,8%	69,2%
Empowerment	31,6%	68,4%	36,4%	63,6%	15,2%	84,8%	42,4%	57,6%
Overall	32,4%	67,6%	38,0%	62,0%	14,4%	85,6%	40,0%	60,0%
<i>So sánh với HyDE</i>								
Comprehensiveness	26,0%	74,0%	41,6%	58,4%	26,8%	73,2%	40,4%	59,6%
Diversity	24,0%	76,0%	38,8%	61,2%	20,0%	80,0%	32,4%	67,6%
Empowerment	25,2%	74,8%	40,8%	59,2%	26,0%	74,0%	46,0%	54,0%
Overall	24,8%	75,2%	41,6%	58,4%	26,4%	73,6%	42,4%	57,6%
<i>So sánh với Microsoft GraphRAG</i>								
Comprehensiveness	45,6%	54,4%	48,4%	51,6%	48,4%	51,6%	50,4%	49,6%
Diversity	22,8%	77,2%	40,8%	59,2%	26,4%	73,6%	36,0%	64,0%
Empowerment	41,2%	58,8%	45,2%	54,8%	43,6%	56,4%	50,8%	49,2%
Overall	45,2%	54,8%	48,0%	52,0%	47,2%	52,8%	50,4%	49,6%

Nhằm phục vụ cho việc nhận xét và đánh giá chính xác các kết quả trong Bảng 2.1, các tiêu chí so sánh đối đầu được định nghĩa chi tiết theo tiêu chuẩn của bài báo gốc. Tiêu chí Tính toàn diện (Comprehensiveness) đo lường mức độ bao quát tri thức, tức là khả năng câu trả lời giải quyết đầy đủ tất cả các góc cạnh và chi tiết được đặt ra trong câu hỏi của người dùng. Tiêu chí Tính đa dạng (Diversity) đánh giá sự phong phú về góc nhìn và thông tin bổ trợ, đảm bảo câu trả lời cung cấp nhiều khía cạnh tri thức khác nhau thay vì chỉ tập trung vào một luận điểm duy nhất. Tiêu chí Tính trao quyền (Empowerment) phản ánh mức độ hữu ích thực tiễn, tức là khả năng hỗ trợ người đọc hiểu rõ bản chất vấn đề để đưa ra các quyết định hoặc hành động chính xác dựa trên bằng chứng cụ thể. Tiêu chí Đánh giá tổng thể (Overall) là kết quả tích hợp từ ba chiều đo lường trên nhằm xác định câu trả lời nào mang lại chất lượng phục vụ tốt nhất cho

người dùng. Đối với cách thức diễn giải thông số trong bảng, tỷ lệ chiến thắng (Win Rate

Kết quả thực nghiệm cho thấy mô hình LightRAG đạt tỷ lệ chiến thắng áp đảo so với RAG truyền thống (Naive RAG) và các mô hình dựa trên câu hỏi/tài liệu giả định (RQ-RAG, HyDE). Điển hình là trên tập dữ liệu Pháp luật, tỷ lệ thắng của LightRAG so với Naive RAG lần lượt là 83,6% về tính toàn diện, 86,4% về tính đa dạng và 83,6% về tính trao quyền. Khi so sánh với đối thủ mạnh nhất là Microsoft GraphRAG, LightRAG vẫn duy trì lợi thế vượt trội về tính đa dạng (ví dụ đạt tỷ lệ thắng 73,6% so với 26,4% của GraphRAG ở tập dữ liệu Pháp luật) trong khi đạt kết quả tương đương ở tính toàn diện và tính trao quyền, nhưng với chi phí lập chỉ mục và tài nguyên gọi API thấp hơn rất nhiều.

2.7 Lý do lựa chọn LightRAG

Từ các kết quả thực nghiệm và đối chiếu lý thuyết nêu trên, nghiên cứu quyết định lựa chọn LightRAG làm giải pháp cốt lõi để xây dựng hệ thống tư vấn học tập vì các nguyên nhân chính: Đầu tiên là về khả năng truy xuất đa cấp độ, sự kết hợp hiệu quả giữa truy xuất cục bộ và truy xuất toàn cục giúp đáp ứng linh hoạt các dạng câu hỏi khác nhau của sinh viên từ chi tiết môn học đến vạch lộ trình học tập tổng thể. Tiếp theo là về chi phí vận hành và lập chỉ mục thấp, việc trích xuất thực thể phẳng giúp giảm thiểu tối đa số lượng token gửi đến mô hình ngôn ngữ lớn trong quá trình xây dựng đồ thị, tiết kiệm đáng kể chi phí API so với Microsoft GraphRAG. Thêm vào đó là khả năng cập nhật tri thức tăng dần, khi LightRAG hỗ trợ thêm mới tài liệu và tích hợp trực tiếp vào đồ thị hiện hành mà không cần lập chỉ mục lại toàn bộ hệ thống từ đầu, rất phù hợp với môi trường học vụ luôn biến động theo từng học kỳ. Cuối cùng là sự phù hợp đặc thù với cấu trúc tài liệu đào tạo, khác với các miền tri thức phẳng thông thường, tài liệu chương trình đào tạo đại học (được trình bày chi tiết tại mục dữ liệu thực nghiệm trong Chương 3) mang đặc tính liên kết mạng lưới rất cao (bao gồm sơ đồ tiên quyết giữa các môn học, ràng buộc về tiến độ học tập và chuẩn đầu ra kỹ năng). Ưu điểm cấu trúc đồ thị của LightRAG cho phép lưu trữ và ánh xạ chính xác mạng lưới liên kết

phức tạp này, giúp hệ thống không bị đứt gãy ngữ cảnh khi lập luận đa bước về lộ trình học tập của sinh viên.

2.8 Hiện trạng của LightRAG (Phiên bản 1.4.11)

Nhằm hiểu rõ hơn về tiềm năng và khả năng ứng dụng thực tế của LightRAG trong ngữ cảnh hiện tại, việc đánh giá chi tiết cả những ưu điểm nổi trội lẫn các hạn chế còn tồn đọng là vô cùng cần thiết. Hiện tại, nền tảng đang được xem xét dựa trên phiên bản phát hành 1.4.11.

Xét về ưu điểm, khác với cấu trúc phân cụm phức tạp gây tốn kém tài nguyên của GraphRAG hay mô hình lược đồ cứng nhắc của OG-RAG, LightRAG sử dụng cấu trúc đồ thị thực thể và mối quan hệ dạng phẳng kết hợp với cơ chế truy xuất đa cấp độ. Hệ thống thực hiện song song việc khớp từ khóa, tìm kiếm vector và truy vấn đồ thị cục bộ lẫn toàn cục. Nhờ chiến lược truy xuất theo hai luồng riêng biệt này, LightRAG không chỉ bao quát được toàn bộ ngữ cảnh từ diện rộng đến chi tiết mà còn đạt tốc độ phản hồi cực kỳ nhanh với độ trễ tối ưu chỉ khoảng tám mươi mili-giây [10]. Một lợi thế cạnh tranh mang tính quyết định khác của hệ thống nằm ở khả năng cập nhật tri thức linh hoạt. Trong môi trường học thuật, nơi quy chế đào tạo thay đổi thường xuyên, LightRAG hỗ trợ cập nhật tăng dần một cách mượt mà bằng cách trích xuất tự động và nối thêm các thực thể vào mạng lưới đồ thị hiện hữu mà không làm phá vỡ tổng thể. Sự linh hoạt này đi kèm với khả năng tương thích cao trên nhiều nền tảng lưu trữ như Neo4j hay PostgreSQL.

Bên cạnh những điểm mạnh vượt trội, hệ thống hiện tại (phiên bản 1.4.11) vẫn bộc lộ nhiều hạn chế cần được nghiên cứu khắc phục. Thứ nhất, quá trình trích xuất thông tin để xây dựng đồ thị phụ thuộc hoàn toàn vào chất lượng của mô hình ngôn ngữ lớn làm nền tảng. Nếu mô hình không hiểu sâu sát văn phong học vụ, hệ thống dễ nhận diện sai hoặc bỏ sót mối quan hệ. Thứ hai, cơ chế trích xuất nội dung văn bản mặc định của LightRAG chủ yếu dựa trên các thư viện đọc trực tiếp (như `pypdf`, `python-docx`, hoặc `docling`) để xử lý các tài liệu PDF, Word. Phương pháp này bộc lộ yếu điểm lớn khi áp dụng vào kho tài liệu thực tế của trường đại học, vốn chứa rất nhiều tài liệu dạng ảnh, các file PDF được

scan từ văn bản giấy, hoặc các slide bài giảng có cấu trúc phức tạp. Do thiếu cơ chế Nhận dạng Ký tự Quang học (OCR) chuyên sâu được tối ưu hóa cho tài liệu Tiếng Việt dạng này, hệ thống không thể đọc được hoặc đọc sai lệch nội dung từ các hình ảnh, dẫn đến việc mất mát lượng lớn thông tin quan trọng trước khi đưa vào mô hình học.

Thứ ba, LightRAG hiện đang thiếu hụt một hệ thống thu thập dữ liệu tự động từ các nguồn trực tuyến. Việc không có sẵn các công cụ cào dữ liệu từ các trang web của trường hay khoa khiến quá trình cập nhật thông tin trở nên thủ công và mất thời gian. Thứ tư, các câu lệnh định hướng (prompt) mặc định của hệ thống được phân bổ quá chung chung. Các câu lệnh này hoàn toàn không phù hợp để xử lý các tài liệu đặc thù của trường đại học, vốn chứa đầy đặc các từ viết tắt, thuật ngữ chuyên ngành và các quy tắc học vụ riêng biệt. Thứ năm, cơ chế hợp nhất mặc định của LightRAG chỉ nhận diện hai thực thể là cùng nút khi tên của hai thực thể này khớp nhau sau khi chuẩn hóa về chữ thường. Điều này có nghĩa HCMUS và Trường Đại học Khoa học Tự nhiên sẽ tồn tại như hai nút hoàn toàn riêng biệt trong đồ thị. Hậu quả trực tiếp là tri thức về cùng một đối tượng bị phân tán, các quan hệ thuộc về một hình thức tên không được kết nối với các quan hệ thuộc hình thức tên khác, và hệ thống truy vấn không thể tổng hợp thông tin một cách đầy đủ. Cuối cùng, cấu trúc Đồ thị tri thức hiện tại của hệ thống vẫn còn thiếu các cơ chế ràng buộc hoặc kiểm chứng tính đúng đắn. Việc không có các quy tắc xác thực chặt chẽ khiến đồ thị dễ bị nhiễu bởi các thực thể sai lệch sinh ra từ quá trình trích xuất tự động, từ đó làm giảm đi độ tin cậy của toàn bộ hệ thống tư vấn. Trong môi trường corpus học thuật tiếng Việt, một thực thể trong thế giới thực thường xuất hiện với nhiều hình thức biểu diễn ngôn ngữ khác nhau tùy theo ngữ cảnh của từng tài liệu.

2.9 Hệ thống Đa tác nhân (Multi-Agent Systems) và Luồng suy luận (Agentic Workflow)

Mặc dù sự kết hợp giữa LLM và LightRAG đã giải quyết được bài toán truy xuất thông tin học vụ chính xác, bản thân mô hình ngôn ngữ vẫn

chỉ đóng vai trò thụ động (nhận câu hỏi và trả lời). Để hệ thống có thể tự động lập kế hoạch, tự kiểm tra lỗi và sử dụng các công cụ một cách linh hoạt, việc áp dụng kiến trúc AI Agent (Tác nhân AI) là thiết yếu.

2.9.1 Khái niệm AI Agent

AI Agent (Tác nhân AI) là một hệ thống sử dụng Mô hình ngôn ngữ lớn (LLM) làm trung tâm để điều khiển quá trình suy luận và đưa ra quyết định [35]. Khác với LLM truyền thống, một Agent hoàn chỉnh bao gồm 4 thành phần cốt lõi: Nền tảng của một Agent bắt đầu từ Hồ sơ (Persona/Profile), đóng vai trò thiết yếu nhằm định hình tính cách và nhiệm vụ cụ thể của tác nhân (ví dụ như: Cố vấn học tập, Trợ giảng). Song song với hồ sơ, hệ thống không thể thiếu Bộ nhớ (Memory). Thành phần này đảm bảo khả năng lưu trữ ngữ cảnh ngắn hạn từ các đoạn hội thoại gần nhất, đồng thời duy trì kiến thức dài hạn liên quan đến thông tin, sở thích và điểm số của sinh viên. Để có thể hành động thực tế, Agent được trang bị các Công cụ (Tools). Nhờ đó, tác nhân có khả năng tương tác trực tiếp với môi trường bên ngoài, ví dụ như thực hiện gọi API để truy xuất dữ liệu từ LightRAG, hay kích hoạt các công cụ tìm kiếm web. Yếu tố làm nên sự khác biệt cốt lõi của Agent chính là năng lực Lập kế hoạch (Planning). Khi đối mặt với một truy vấn phức tạp từ người dùng, Agent có khả năng tự động phân tích và chia nhỏ vấn đề thành nhiều bước xử lý tuần tự để giải quyết một cách triệt để.

2.9.2 Tích hợp LightRAG và Multi-Agent

Sự kết hợp giữa Multi-Agent và LightRAG tạo ra một hệ thống hỗ trợ học tập toàn diện. Trong đó, LightRAG đóng vai trò là “Bộ nhớ kiến thức dài hạn” (Long-term Knowledge Memory) với độ chính xác cao và chi phí tối ưu, còn Multi-Agent đóng vai trò là “Trung tâm xử lý logic” (Reasoning Engine). Tác nhân AI sẽ tự động quyết định khi nào cần sử dụng truy xuất cấp độ thấp (chỉ tìm thông tin 1 môn học) và khi nào cần truy xuất cấp độ cao (tìm lộ trình tổng thể) trên nền tảng LightRAG, từ đó mang lại trải nghiệm tư vấn cá nhân hóa và sát với thực tế nhất cho sinh viên Khoa Công nghệ Thông tin.

2.10 Đánh giá hệ thống RAG và Multi-Agent

Sau khi thiết kế và triển khai hệ thống thông minh dựa trên kiến trúc LightRAG và Multi-Agent, việc thiết lập một quy trình đánh giá chuẩn xác là vô cùng cần thiết. Đặc thù của dữ liệu giáo dục đại học yêu cầu hệ thống không chỉ trả lời tự nhiên mà phải đảm bảo tính chính xác tuyệt đối, không gây ảnh hưởng sai lệch đến quyết định đăng ký môn học của sinh viên.

2.10.1 Các chỉ số đánh giá RAG (RAGAS)

Để đánh giá chất lượng của kiến trúc RAG một cách tự động và định lượng, RAGAS (Retrieval Augmented Generation Assessment) [8] hiện là framework tiêu chuẩn được cộng đồng nghiên cứu sử dụng rộng rãi. RAGAS đánh giá hệ thống thông qua hai khía cạnh chính: khả năng truy xuất (Retrieval) và khả năng sinh văn bản (Generation), với 4 chỉ số cốt lõi: Chỉ số trọng tâm nhất là Độ trung thực (Faithfulness), nhằm đo lường mức độ câu trả lời của AI bám sát vào ngữ cảnh (context) đã được hệ thống truy xuất. Giả sử hệ thống bịa ra một quy định chuẩn đầu ra Tiếng Anh hoàn toàn không có trong tài liệu của trường, chỉ số này sẽ lập tức bị đánh giá thấp, qua đó phơi bày tình trạng “ảo giác” (hallucination). Bên cạnh tính trung thực, Độ liên quan của câu trả lời (Answer Relevance) cũng cần được xem xét kỹ lưỡng. Tiêu chí này dùng để kiểm định xem câu trả lời do AI sinh ra có đi thẳng vào trọng tâm vấn đề mà sinh viên thắc mắc hay không, hay chỉ đang phản hồi một cách vòng vo và dư thừa thông tin. Mặt khác, đối với module truy xuất, Độ chính xác của ngữ cảnh (Context Precision) đóng vai trò định lượng chất lượng tìm kiếm. Chỉ số này cho biết liệu các tài liệu hoặc các node trên đồ thị tri thức quan trọng nhất có được thuật toán ưu tiên xếp hạng ở những vị trí đầu tiên hay không. Bổ sung cho độ chính xác là Độ bao phủ của ngữ cảnh (Context Recall). Đây là thước đo đánh giá xem bộ truy xuất có thu thập “đủ” các thông tin cần thiết để phục vụ cho câu hỏi hay không. Nếu sinh viên hỏi một vấn đề đòi hỏi sự kết hợp giữa đề cương môn học và quy chế học vụ, chỉ số này sẽ phản ánh việc hệ thống có bỏ sót bất kỳ tài liệu quan trọng nào không.

2.10.2 Đánh giá hệ thống đa agent (Multi-Agent Evaluation)

Việc đánh giá một hệ thống Đa tác nhân (Multi-Agent) phức tạp hơn RAG thông thường, bởi quy trình này không chỉ là đầu vào - đầu ra (input/output) mà là một quy trình suy luận gồm nhiều bước (Agentic workflow). Các tiêu chí đánh giá tập trung vào: Thước đo quan trọng hàng đầu là Tỷ lệ hoàn thành nhiệm vụ (Task Completion Rate). Chỉ số này tập trung đo lường tần suất các Agent có thể phối hợp thành công để giải quyết trọn vẹn một luồng công việc, ví dụ như đưa ra một kế hoạch học tập hoàn chỉnh cho sinh viên. Ngoài việc hoàn thành đúng nhiệm vụ, Hiệu suất và Chi phí (Efficiency & Cost) cũng là bài toán thực tế cần lời giải. Nhóm nghiên cứu cần tiến hành đánh giá chi tiết về độ trễ trung bình của hệ thống (Latency) cũng như chi phí tài nguyên (dựa trên số lượng API Token tiêu thụ cho mỗi phiên hội thoại). Nhờ vào sự tối ưu của kiến trúc, việc tích hợp LightRAG được kỳ vọng sẽ giảm thiểu đáng kể chỉ số chi phí này so với các mô hình GraphRAG truyền thống.

2.10.3 Các phương pháp benchmark hiện nay

Để tiến hành đo lường các chỉ số trên, nhóm dự kiến áp dụng các phương pháp benchmark (chấm điểm chuẩn) sau: Nền móng của toàn bộ quá trình đánh giá là việc xây dựng tập dữ liệu chuẩn (Golden Dataset / Ground Truth). Để thực hiện điều này, nhóm sẽ tự biên soạn một bộ khoảng 100-200 cặp câu hỏi - câu trả lời chuẩn xác, bao quát các kịch bản thực tế như đăng ký môn tiên quyết, tư vấn hướng chuyên ngành, hay quy định xét tốt nghiệp. Toàn bộ câu trả lời trong tập dữ liệu này sẽ được trích xuất thủ công và chính xác từ các văn bản chính thức của Khoa Công nghệ Thông tin, nhằm làm mốc tham chiếu để đối chiếu tự động với kết quả đầu ra của hệ thống. Để tự động hóa quá trình chấm điểm quy mô lớn, phương pháp LLM-as-a-Judge (Sử dụng LLM mạnh hơn để chấm điểm) sẽ được áp dụng. Thay vì phải tốn công sức chấm điểm thủ công hàng ngàn câu trả lời, nhóm sẽ sử dụng các mô hình ngôn ngữ tiên tiến nhất hiện nay (như GPT-4 hoặc Claude 3.5) để đóng vai trò là “Giám khảo” [37]. Vị giám khảo này sẽ có nhiệm vụ đọc câu hỏi, đối chiếu câu trả lời chuẩn (Ground Truth) với câu trả lời của hệ thống, từ đó chấm điểm các tiêu chí

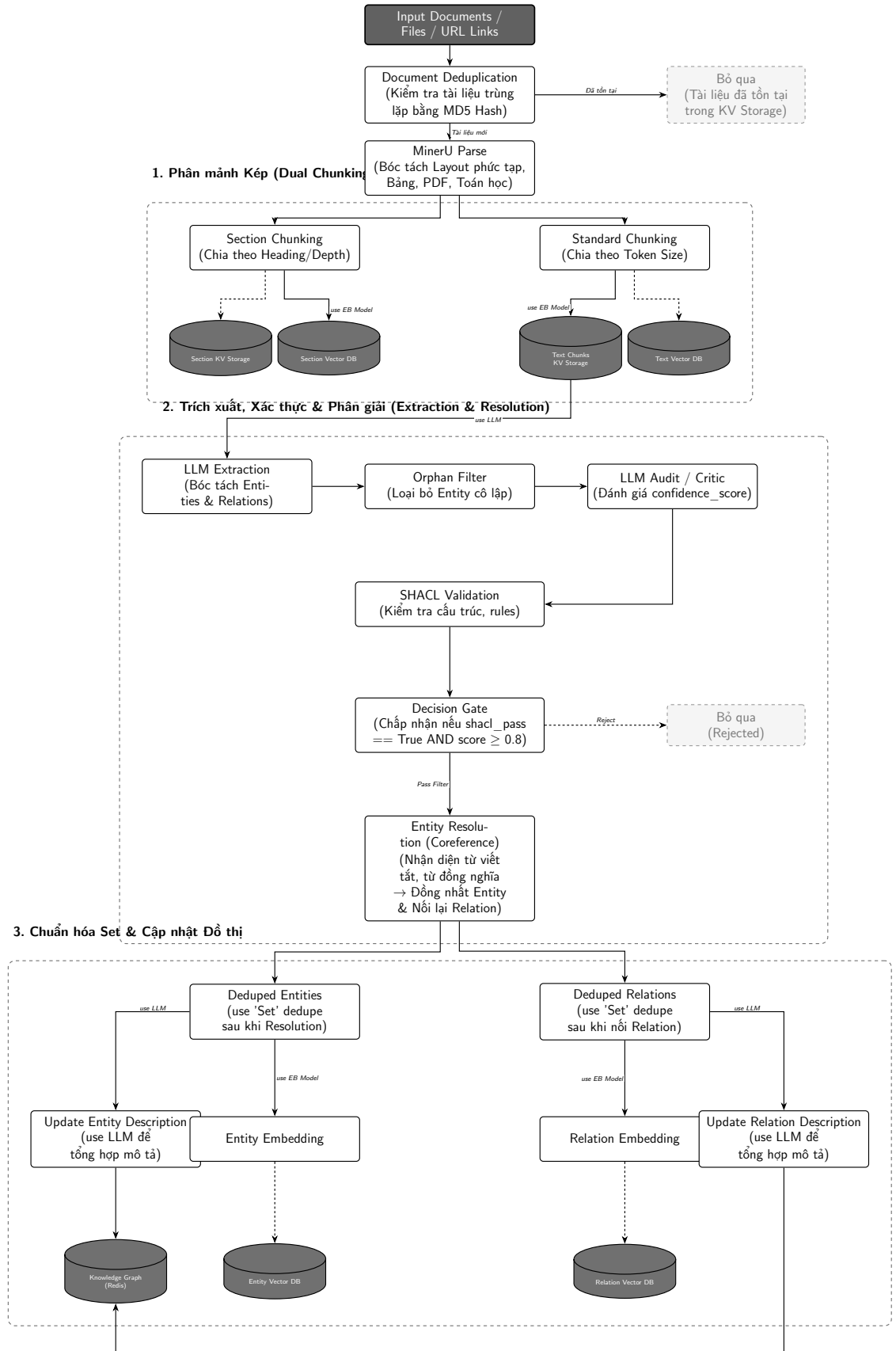
RAGAS một cách khách quan và nhất quán. Bước kiểm chứng cuối cùng và cũng mang tính quyết định nhất chính là Đánh giá từ người dùng thực tế (Human-in-the-loop Evaluation). Trong giai đoạn này, hệ thống sẽ được triển khai thử nghiệm (Beta testing) cho một nhóm sinh viên và giảng viên cố vấn học tập của Khoa. Thông qua các khảo sát về trải nghiệm người dùng (UX) và mức độ hữu ích của thông tin, những phản hồi thu thập được sẽ là cơ sở thực tiễn vô giá để nhóm tiếp tục tinh chỉnh các Prompt cũng như luồng hoạt động của Agent.

Chương 3

Phương pháp đề xuất

Trên cơ sở phân tích lý thuyết và yêu cầu thực tiễn, đề xuất kiến trúc hệ thống hỗ trợ học tập thông minh dựa trên LightRAG [10], một mã nguồn mở được phát triển từ GraphRAG [6] và LangGraph [19].

3.1 Quy trình xử lý dữ liệu



Hình 3.1: Quy trình tổng quát từ văn bản thô đến đồ thị tri thức

Đây là quy trình xử lý thông tin, đóng vai trò "nguyên liệu đầu vào" cho toàn bộ hệ thống. Hệ thống không chỉ lưu trữ dữ liệu thô mà còn thực hiện các phép biến đổi vector hóa phức tạp. Quản lý nguồn dữ liệu đa dạng bao gồm hồ sơ người học và các học liệu phi cấu trúc như bài giảng, tài liệu tham khảo.

Đồ thị tri thức (Knowledge Graph) [12]: đây là thành phần quan trọng nhất, lưu trữ tri thức dưới dạng cấu trúc mạng lưới. Các nút đại diện cho các khái niệm và các cạnh biểu diễn mối quan hệ sự phạm. Điều này giúp hệ thống hiểu được "vị trí" và "tầm quan trọng" của một khái niệm trong tổng thể chương trình học.

3.1.1 Tiền xử lý dữ liệu tri thức

3.1.1.1 Hệ thống MinerU

Cơ sở lựa chọn công cụ và vai trò trong hệ thống như trong môi trường học thuật, nguồn dữ liệu đầu vào thường rất đa dạng và phức tạp, bao gồm các tệp PDF được scan từ văn bản giấy, tài liệu chứa nhiều biểu đồ, hình ảnh, và đặc biệt là các công thức toán học. Các công cụ trích xuất văn bản truyền thống thường xuyên gặp thất bại trong việc xử lý các tài liệu này, dẫn đến việc mất thông tin quan trọng hoặc làm xáo trộn cấu trúc bài giảng.

Để giải quyết triệt để thách thức này, hệ thống áp dụng công cụ MinerU [25] (từ phiên bản 3.2.3 trở lên) làm giải pháp lõi cho quy trình tiền xử lý dữ liệu. Đây là một công cụ trích xuất dữ liệu đa phương thức chất lượng cao, tích hợp các mô hình tiên tiến như Vision-Language Model (VLM) cho việc phân tích bố cục (Layout Analysis), mô hình OCR để nhận diện văn bản trên ảnh, và mô hình Mathematical Formula Recognition (MFR) để chuyển đổi công thức toán học sang chuẩn LaTeX. Sự kết hợp này giúp MinerU không chỉ trích xuất chính xác văn bản mà còn bảo toàn hoàn hảo cấu trúc logic của tài liệu gốc, đặc biệt phù hợp với các giáo trình, bài báo khoa học và tài liệu lưu trữ của trường đại học.

Để đạt hiệu quả tối ưu trong việc bóc tách các tài liệu học thuật phức tạp, MinerU phiên bản 3.x+ tích hợp một chuỗi các mô hình học sâu chuyên biệt cho từng loại tác vụ trong luồng xử lý hybrid. Cơ chế này giúp

phân rã và xử lý chính xác từng phần của trang văn bản thông qua các mô hình được liệt kê chi tiết trong Bảng 3.1.

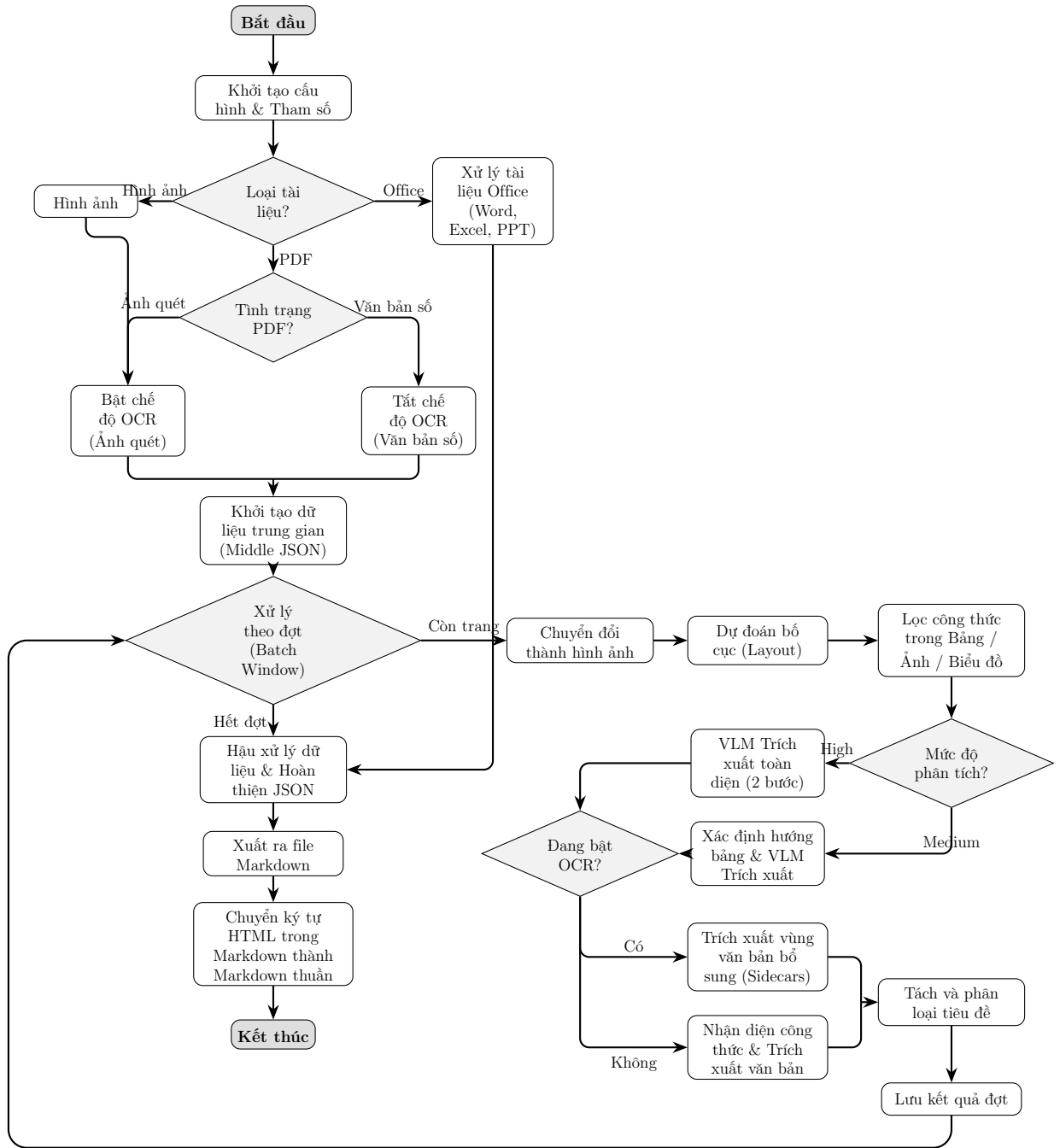
Bảng 3.1: Bảng tổng hợp các mô hình thành phần trong MinerU (Version 3.x+)

Loại tác vụ (Task Type)	Tên mô hình (Model Name)	Vai trò và đặc điểm
Phân tích bố cục (Layout Detection)	PPDocLayoutV2	Quét toàn bộ trang, nhận diện cấu trúc và phân loại 24 loại vùng dữ liệu khác nhau (văn bản, tiêu đề, ảnh, bảng biểu...).
Nhận diện công thức (Formula Rec)	UniMERNet (Mặc định) hoặc PP-FormulaNet Plus M	Dịch các vùng ảnh chứa công thức (cả inline và block) sang mã chuẩn LaTeX. Bản Plus M bổ sung tối ưu cho công thức kèm ký tự tiếng Trung.
Nhận diện chữ (OCR)	PytorchPaddleOCR (paddleocr_torch)	Trích xuất ký tự văn bản thuần với độ phủ lên tới 109 ngôn ngữ.
Phân loại bảng (Table Cls)	PaddleTableCls (dựa trên PP-LCNet)	Phân loại xem bảng đó thuộc dạng có viền (wired) hay không viền (wireless) để điều phối qua mô hình cấu trúc phù hợp.
Cấu trúc bảng (Table Parsing)	SlanetPlus (cho bảng không viền) hoặc Unet (cho bảng có viền)	Bóc tách cấu trúc hàng/cột, xử lý các ô gộp phức tạp và chuyển đổi thành định dạng HTML/Markdown.
Phát hiện hướng (Orientation)	PaddleOrientationCls (PP-LCNet)	Tự động phát hiện và xoay thẳng lại các trang hoặc bảng biểu bị đặt ngược hoặc nghiêng góc.

Bên cạnh các mô hình chuyên biệt nêu trên, trong các phiên bản cập nhật mới nhất (như dòng MinerU 2.5 Pro trở lên), hệ thống hỗ trợ tích hợp trực tiếp mô hình ngôn ngữ lớn đa phương thức Vision-Language Model (VLM) đóng vai trò là động cơ trích xuất duy nhất hoặc phối hợp (Hybrid). Khi hoạt động trong vai trò VLM chính của đường ống (pipeline), mô hình

MinerU 2.5 Pro VLM tiếp nhận trực tiếp ảnh chụp trang tài liệu để thực hiện đồng thời các tác vụ nhận diện bố cục, nhận diện chữ (OCR), và dịch công thức toán học/bảng biểu chỉ qua một lượt suy luận duy nhất (single-turn inference). Giải pháp này giúp giảm bớt sự phụ thuộc vào chuỗi nhiều mô hình nhỏ kế tiếp nhau, hạn chế tối đa sai số tích lũy giữa các bước phân tích bố cục và nhận diện chữ, qua đó duy trì cấu trúc tài liệu một cách tự nhiên và chính xác hơn.

Luồng xử lý tự động (Hybrid-Auto-Engine) Chế độ `-b hybrid-auto-engine` trong MinerU thực thi một quy trình xử lý khép kín, kết hợp giữa mô hình đa phương thức (VLM) để bóc tách cấu trúc và các mô hình truyền thống (OCR, MFR) để nhận dạng văn bản. Dưới đây là sơ đồ chi tiết về luồng hoạt động này:



Hình 3.2: Quy trình Hybrid-Auto-Engine của MinerU[25]

Quy trình xử lý được thực hiện tuần tự qua các giai đoạn. Đầu tiên, hệ thống tiến hành phân loại và phân tích định dạng của tệp đầu vào để xác định là văn bản gốc hay bản quét. Dựa trên kết quả này, cơ chế nhận dạng ký tự quang học sẽ tự động được kích hoạt nếu cần thiết. Tiếp theo, hệ thống đánh giá tài nguyên phần cứng hiện có, cụ thể là dung lượng bộ nhớ đồ họa VRAM, để tính toán tỷ lệ phân lô phù hợp, đảm bảo tối ưu hóa hiệu suất mà không gây tràn bộ nhớ. Sau quá trình khởi tạo, tài liệu được phân chia thành các đoạn nhỏ để xử lý dần. Tại mỗi phân đoạn, hệ

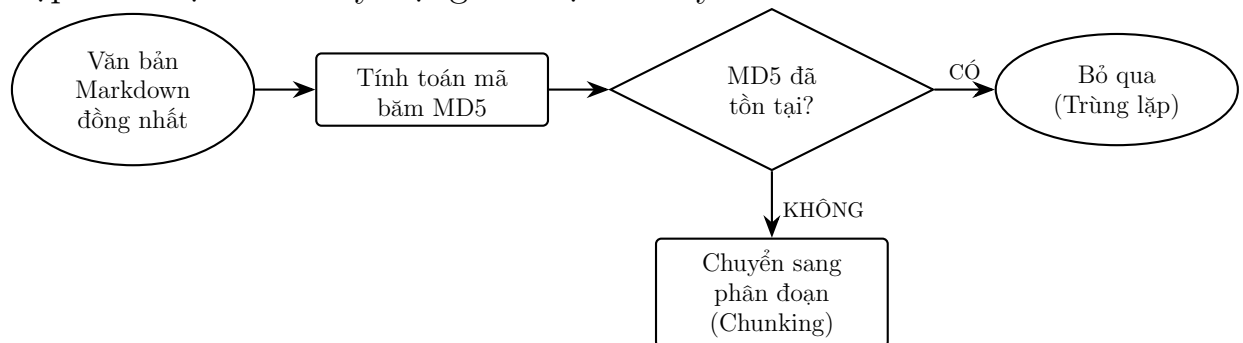
thống sử dụng một mô hình đa phương thức VLM mạnh mẽ để bóc tách cấu trúc tổng thể, nhận diện các khối văn bản, bảng biểu, hình ảnh và công thức. Đối với các vùng chứa hình ảnh và bảng biểu, hệ thống sẽ tiến hành làm mờ nhằm không gây nhiễu cho các mô hình phía sau. Đồng thời, các công thức toán học nằm trong dòng văn bản sẽ được mô hình MFR chuyên biệt nhận diện và dịch đổi sang định dạng mã chuẩn LaTeX. Các khối chữ chưa xử lý được chuyển qua mô hình nhận diện ký tự quang học OCR. Nhằm đảm bảo độ chính xác, các kết quả có độ tin cậy thấp sẽ bị hệ thống tự động lọc bỏ. Cuối cùng, ở giai đoạn tổng hợp, tọa độ của các khối thông tin được chuẩn hóa về một hệ quy chiếu chung. Tất cả các dữ liệu trích xuất từ mô hình đa phương thức, mô hình nhận diện chữ và mô hình công thức được gộp chung vào một cấu trúc dữ liệu trung gian. Quy trình này lặp lại cho đến khi xử lý hết các phân đoạn của tài liệu, sau đó hệ thống sẽ giải phóng tài nguyên bộ nhớ và hoàn tất việc tạo ra một văn bản đầu ra đồng nhất và giữ nguyên cấu trúc logic gốc. Nhờ đường ống Hybrid này, MinerU khắc phục hoàn toàn những nhược điểm của việc đọc tệp bằng `textextract` hay `pypdf` thông thường, cung cấp dòng dữ liệu đầu vào sắc nét và có độ tin cậy tuyệt đối cho mô hình ngôn ngữ phía sau.

3.1.2 Luồng logic xử lý và cơ chế kiểm tra trùng lặp

Một trong những thành phần then chốt giúp hệ thống vận hành hiệu quả và tiết kiệm chi phí là cơ chế kiểm soát dữ liệu tại cổng vào. Sau khi văn bản được trích xuất từ các nguồn đa dạng (PDF, Office, Web) và đưa về định dạng Markdown đồng nhất, hệ thống thực hiện quy trình kiểm tra trùng lặp (Deduplication) dựa trên mã băm nội dung.

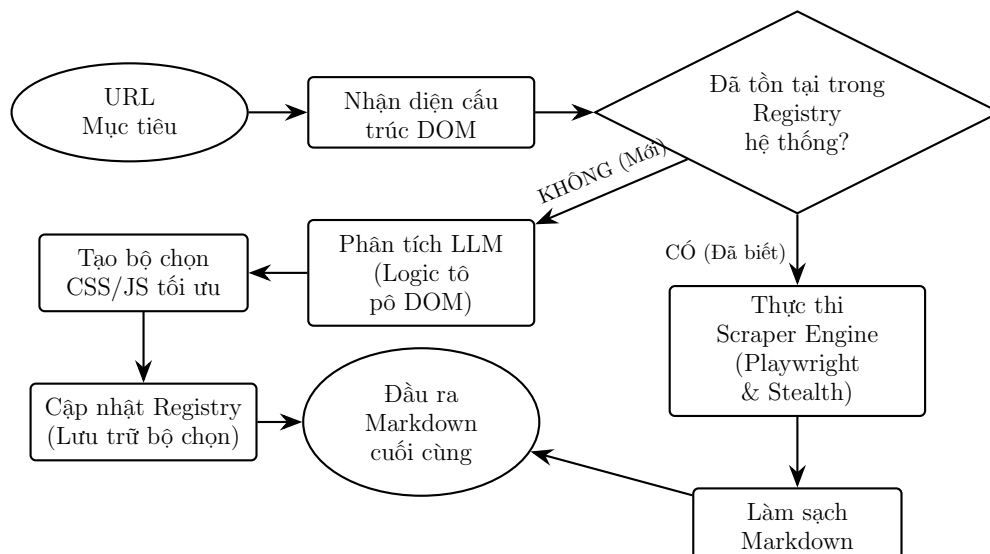
Cụ thể, hệ thống sử dụng thuật toán băm MD5 để tạo ra một "dấu vân tay" kỹ thuật số duy nhất cho toàn bộ nội dung văn bản. Mã băm này sau đó được đối chiếu với cơ sở dữ liệu tri thức hiện tại, dẫn đến hai kịch bản xử lý chính. Trong trường hợp trùng khớp hoàn toàn, nếu mã MD5 của tài liệu mới đã tồn tại, hệ thống sẽ ngay lập tức dừng quy trình xử lý nhằm ngăn chặn việc nạp lại dữ liệu dư thừa, bảo vệ tính toàn vẹn của đồ thị tri thức và tiết kiệm tối đa tài nguyên tính toán. Ngược lại, đối với trường hợp nội dung mới hoặc có sự thay đổi, tài liệu được coi là nguồn tri thức

mới và được chuyển tiếp sang giai đoạn phân đoạn. Ngay cả khi tài liệu chỉ có một thay đổi nhỏ, hệ thống vẫn sẽ thực hiện lại quy trình để đảm bảo tính cập nhật, trong khi sự trùng lặp về mặt thực thể sẽ được tự động hợp nhất tại bước xây dựng đồ thị sau này.



Hình 3.3: Luồng logic kiểm tra trùng lặp bằng cơ chế băm nội dung

3.1.2.1 Hệ thống thu thập tri thức từ trang web



Hình 3.4: Quy trình thu thập tri thức từ trang web với Smart Scraper

Hệ thống cào web thông minh hoạt động theo quy trình tự động hóa từ việc nhận diện cấu trúc trang đến trích xuất dữ liệu. Logic cốt lõi dựa trên công cụ Smart Scraper Engine với cơ chế vận hành qua bốn bước chính: lấy dấu vân tay, đối chiếu kho lưu trữ, phân tích cấu trúc bằng mô hình ngôn ngữ và thực thi bộ phân giải.

Quy trình bắt đầu bằng việc tạo ra một bản tóm tắt cấu trúc cây DOM của trang web, tập trung vào các thuộc tính định danh như thẻ HTML, tên lớp và ID. Hệ thống trích xuất một bộ khung xương kỹ thuật của trang

web để làm cơ sở cho việc nhận diện. Sau đó, bản tóm tắt này được đối chiếu với kho lưu trữ các bộ phân giải đã biết. Mỗi mục trong kho lưu trữ chứa các thông tin về dấu hiệu nhận biết, bộ chọn nội dung chính, các thành phần cần loại bỏ và chiến lược chờ đợi phù hợp cho từng loại công nghệ web như React, Vue, hay các hệ quản trị nội dung truyền thống.

Trong trường hợp gặp một trang web mới không có trong kho lưu trữ, hệ thống sẽ gửi một phần cấu trúc DOM tiêu biểu cho mô hình ngôn ngữ lớn. Mô hình này có nhiệm vụ phân tích logic phân bố nội dung và tự động sinh ra mã lệnh điều hướng. Mã lệnh này bao gồm các bộ chọn CSS tối ưu để trích xuất văn bản có ý nghĩa và các quy tắc loại bỏ những thành phần gây nhiễu như trình đơn điều hướng, biểu ngữ quảng cáo hoặc các đoạn mã kịch bản. Bộ phân giải mới sinh ra sẽ được kiểm tra và lưu lại vào kho lưu trữ để sử dụng cho các lần truy cập sau, giúp tối ưu hóa hiệu suất và giảm thiểu chi phí gọi API.

Đối với các tên miền mặc định như `fit.hcmus.edu.vn` và `hcmus.edu.vn`, hệ thống đã được cấu hình sẵn các bộ phân giải chuyên biệt cho nền tảng DotNetNuke. Do đặc thù của nền tảng này thường chứa nhiều nội dung lồng ghép trong các khung phức tạp, hệ thống sử dụng các bộ chọn mục tiêu như `.dnn_contentPane` và `#dnn_ContentPane` để định vị chính xác vùng chứa thông tin học thuật. Quy trình trích xuất văn bản tại đây không chỉ lấy chữ thuần túy mà còn thực hiện chuyển đổi các liên kết quan trọng sang định dạng Markdown để bảo toàn thông tin tham chiếu.

Đặc biệt, đối với các trang có tính tương tác cao như danh mục môn học hoặc đề cương chi tiết, hệ thống áp dụng chế độ tương tác vết càn. Chế độ này sử dụng thư viện Playwright để tự động thực hiện các hành động như nhấn vào các nút mở rộng, cuộn trang để tải thêm dữ liệu và chờ đợi các tiến trình AJAX hoàn tất. Điều này đảm bảo rằng toàn bộ tri thức về chương trình đào tạo, mô tả môn học và thông tin giảng viên được thu thập đầy đủ nhất, ngay cả khi các thông tin này nằm sau các thành phần giao diện động. Dữ liệu sau khi trích xuất sẽ được làm sạch và chuẩn hóa trước khi đưa vào các bước phân đoạn và xây dựng đồ thị tri thức tiếp theo.

3.1.3 Chiến lược phân đoạn và chuẩn hóa văn bản

Quá trình xử lý khởi đầu bằng việc làm sạch dữ liệu đầu vào, loại bỏ các ký tự lỗi và đảm bảo văn bản tuân thủ chuẩn mã hóa quốc tế Unicode (UTF-8). Bước tiền xử lý này không chỉ chuẩn hóa định dạng mà còn loại bỏ các thẻ HTML dư thừa, khoảng trắng không cần thiết và các ký tự điều khiển có thể gây nhiễu cho mô hình ngôn ngữ ở các bước xử lý tiếp theo.

Về mặt kỹ thuật, hệ thống áp dụng song song hai chiến lược phân đoạn chính nhằm phục vụ hai mục tiêu khác biệt trong kiến trúc tổng thể: chiến lược phân đoạn theo kích thước cố định phục vụ trích xuất đồ thị tri thức và chiến lược phân đoạn phân cấp đa phương thức phục vụ truy xuất ngữ cảnh chính xác.

Chiến lược thứ nhất, phân đoạn theo kích thước cố định (fixed-size chunking), được thiết kế chuyên biệt để phục vụ quy trình trích xuất thực thể và quan hệ nhằm xây dựng đồ thị tri thức trong LightRAG [11]. Phương pháp này chia toàn bộ văn bản đã chuẩn hóa thành các đoạn có độ dài xác định trước được thiết lập cố định là 1200 token theo nghiên cứu và khuyến nghị của các tác giả, đi kèm với vùng chồng lấp (overlap window) cố định là 100 token giữa hai đoạn liên tiếp.

Cơ chế chồng lấp này đóng vai trò như một “cầu nối ngữ cảnh” (context bridge): khi mô hình ngôn ngữ xử lý đoạn thứ i , mô hình đã có sẵn phần đuôi của đoạn thứ $i - 1$ làm nền tảng, từ đó tránh được hiện tượng cắt đứt giữa chừng một câu hoặc một luận điểm quan trọng. Cụ thể, với kích thước đoạn mặc định là 1200 token và vùng chồng lấp 100 token, hệ thống đảm bảo rằng các thực thể và mối quan hệ nằm gần ranh giới giữa hai đoạn liên tiếp vẫn được mô hình ngôn ngữ lớn nhận diện đầy đủ trong ít nhất một đoạn.

Chiến lược phân đoạn kích thước cố định được lựa chọn cho tác vụ trích xuất đồ thị do quy trình trích xuất thực thể và quan hệ trong LightRAG yêu cầu mỗi đoạn văn phải đủ dài để chứa ngữ cảnh phong phú, giúp mô hình ngôn ngữ lớn nhận diện chính xác các thực thể và suy luận được mối liên hệ giữa các thực thể này. Các đoạn quá ngắn sẽ thiếu ngữ cảnh, dẫn đến việc bỏ sót các quan hệ ngầm định giữa các câu không liền kề.

Chiến lược thứ hai, phân đoạn phân cấp (hierarchical chunking), được

thiết kế nhằm bảo toàn tối đa cấu trúc và ngữ nghĩa của tài liệu gốc, phục vụ cho việc truy xuất ngữ cảnh chính xác khi hệ thống nhận truy vấn từ người dùng. Chiến lược này tham khảo từ MultiDocFusion [31], một hệ thống phân đoạn phân cấp được đề xuất cho các tài liệu dài và phức tạp, đã chứng minh hiệu quả vượt trội trong việc cải thiện độ chính xác truy xuất từ 8–15% so với các phương pháp phân đoạn truyền thống.

Khác với phân đoạn kích thước cố định vốn xử lý văn bản như một chuỗi ký tự thuần túy, chiến lược phân cấp đa phương thức nhận diện rằng tài liệu học thuật không chỉ chứa văn bản tuyến tính mà còn bao gồm nhiều thành phần có cấu trúc và ngữ nghĩa khác nhau. Do đó, quy trình phân đoạn không thể chỉ dựa trên số lượng token mà cần xem xét ba loại cấu trúc: (i) cấu trúc phân cấp (structural), tức mối quan hệ cha-con giữa các tiêu đề, mục, đoạn văn; (ii) cấu trúc bố cục (layout), tức vị trí vật lý, cột, vùng bao đóng của các thành phần trên trang; và (iii) cấu trúc ngữ nghĩa (semantic), tức mối quan hệ về nghĩa giữa các đoạn nội dung dù không kề nhau về vị trí vật lý.

Quy trình phân đoạn phân cấp đa phương thức trong hệ thống được triển khai qua bốn giai đoạn tuần tự, tương ứng với bốn bước chính trong kiến trúc MultiDocFusion [31]:

Giai đoạn 1: Phân tích cấu trúc tài liệu bằng thị giác máy tính (Vision-based Document Parsing). Hệ thống VMiner (đã trình bày ở mục trước) sử dụng các mô hình học sâu chuyên biệt để phân tích cấu trúc hình học của từng trang tài liệu. Quá trình này tự động phát hiện và phân loại các vùng nội dung thành các loại thành phần riêng biệt: vùng tiêu đề (heading), khối văn bản (text block), bảng biểu (table), hình ảnh và biểu đồ (figure/chart), danh sách (list), chú thích (footnote) và công thức toán học. Mỗi vùng được xác định bởi một hộp bao đóng (bounding box) chứa tọa độ chính xác trên trang, làm cơ sở cho việc sắp xếp thứ tự đọc hợp lý và phân loại nội dung ở các bước tiếp theo.

Giai đoạn 2: Trích xuất nội dung và phân loại thành phần đa phương thức. Sau khi nhận diện các vùng nội dung, hệ thống tiến hành trích xuất nội dung tương ứng từ từng vùng. Đối với các vùng văn bản thuần túy, nội dung được trích xuất bằng các công cụ nhận dạng ký tự quang học (OCR) đã tối ưu hóa cho tiếng Việt. Đối với bảng biểu, hệ thống tái tạo cấu trúc

dữ liệu dưới dạng Markdown hoặc HTML, bảo toàn nguyên vẹn thông tin hàng, cột và tiêu đề. Đối với hình ảnh và biểu đồ, hệ thống sinh mô tả văn bản tự động thông qua mô hình ngôn ngữ thị giác (Vision Language Model, VLM), kết hợp với chú thích (caption) gốc và văn bản ngữ cảnh xung quanh hình ảnh trong tài liệu. Bước phân loại này đảm bảo mỗi thành phần được xử lý bởi chiến lược phù hợp nhất với đặc trưng dữ liệu của thành phần đó.

Giai đoạn 3: Xây dựng cây cấu trúc phân cấp (Hierarchical Structure Parsing). Đây là giai đoạn cốt lõi phân biệt chiến lược phân cấp với các phương pháp phân đoạn truyền thống. Hệ thống tái cấu trúc toàn bộ tài liệu thành một cây phân cấp (hierarchical tree) phản ánh tổ chức logic của nội dung, trong đó gốc cây là tài liệu, các nút trung gian là các mục (section) và tiểu mục (sub-section), còn các nút lá là các đơn vị nội dung cụ thể như đoạn văn, bảng, hình ảnh. Cấu trúc phân cấp này được suy luận từ hệ thống tiêu đề (heading hierarchy) trong tài liệu, cụ thể các cấp tiêu đề H1, H2, H3 tương ứng với các mức độ lồng nhau khác nhau trong cây. Trong trường hợp tài liệu không có tiêu đề rõ ràng hoặc cấu trúc tiêu đề bị nhiễu, hệ thống bổ sung cơ chế phân tích dựa trên mô hình ngôn ngữ lớn (tương tự kỹ thuật DSHP-LLM trong MultiDocFusion [31]) để suy luận ranh giới ngữ nghĩa giữa các phần. Mỗi nút trong cây được gán một đường dẫn phân cấp (breadcrumb path) dạng: [Tài liệu] > [Chương X] > [Mục X.Y] > [Tiểu mục X.Y.Z], cung cấp ngữ cảnh vị trí chính xác cho từng đơn vị nội dung.

Giai đoạn 4: Gom nhóm và sinh phân đoạn theo chiến lược DFS (DFS-based Grouping). Giai đoạn cuối cùng thực hiện việc duyệt cây phân cấp theo chiều sâu (Depth-First Search) để gom nhóm các nút lá thành các phân đoạn hoàn chỉnh, tuân thủ giới hạn kích thước token tối đa đã được cấu hình. Thuật toán duyệt DFS đảm bảo rằng các nút con thuộc cùng một nút cha sẽ được ưu tiên gom vào cùng một phân đoạn, từ đó bảo toàn tính liên kết ngữ nghĩa theo cấu trúc phân cấp gốc. Cụ thể, quy tắc gom nhóm được áp dụng dựa trên từng loại thành phần nội dung.

Đối với thành phần là một đoạn văn ngắn (dưới 100 token), gộp với các đoạn văn tiếp theo thuộc cùng mục (section) cho đến khi đạt ngưỡng kích thước tối thiểu.

Đối với thành phần là một đoạn văn dài (vượt quá giới hạn token), áp dụng kỹ thuật tách nhận biết ranh giới câu (sentence-aware splitting) để tránh cắt ngang ý tưởng.

Đối với thành phần là bảng biểu, giữ nguyên toàn bộ bảng như một phân đoạn độc lập kèm theo chú thích bảng và ngữ cảnh mục chứa bảng. Đối với bảng quá lớn vượt giới hạn token, áp dụng chiến lược tách theo nhóm hàng kèm giữ nguyên hàng tiêu đề.

Đối với thành phần là hình ảnh hoặc biểu đồ, tạo phân đoạn hình ảnh chứa mô tả sinh bởi VLM, chú thích (caption) gốc và đoạn văn bản ngữ cảnh trước/sau hình ảnh, thiết lập liên kết chéo giữa các thành phần này.

Đối với thành phần là tài liệu trình chiếu (slide), giữ nguyên toàn bộ nội dung slide gồm tiêu đề, khung văn bản, danh sách và ghi chú diễn giả làm một phân đoạn; tách khi nội dung vượt giới hạn.

Mỗi phân đoạn được tạo ra sẽ được nhúng (inject) thêm thông tin đường dẫn phân cấp (breadcrumb context) vào đầu nội dung, theo định dạng:

```
[Document: {tên_tài_liệu}] > [{đường_dẫn_mục}] {nội_dung_gốc}
```

Kỹ thuật nhúng ngữ cảnh phân cấp này giúp mô hình ngôn ngữ lớn có đủ thông tin định vị ngay cả khi phân đoạn bị tách khỏi tài liệu gốc. Breadcrumb được nhúng vào đầu nội dung nhưng không được tính vào số lượng token của phần nội dung chính, nhằm không ảnh hưởng đến chất lượng biểu diễn vector nhúng (embedding) của nội dung thực.

Bảng 3.2: So sánh hai chiến lược phân đoạn trong hệ thống

Tiêu chí	Chiến lược 1: Kích thước cố định	Chiến lược 2: Phân cấp đa phương thức
Mục đích chính	Trích xuất thực thể và quan hệ cho đồ thị tri thức	Truy xuất ngữ cảnh chính xác cho RAG
Đơn vị phân đoạn	Cửa sổ token cố định (1200 token)	Đơn vị ngữ nghĩa tự nhiên (đoạn văn, bảng, hình)
Bảo toàn cấu trúc	Không, xử lý văn bản thuần tuyến tính	Có, bảo toàn cấu trúc phân cấp gốc
Ngữ cảnh vị trí	Dựa vào vùng chồng lấp	Breadcrumb path nhúng vào mỗi chunk
Ưu điểm	Đơn giản, phân bổ tải đều, phù hợp cho trích xuất hàng loạt	Bảo toàn ngữ nghĩa, truy xuất chính xác, hỗ trợ đa phương thức
Hạn chế	Có thể cắt ngang ý tưởng	Phức tạp hơn, phụ thuộc vào chất lượng phân tích cấu trúc

Sự phối hợp giữa hai chiến lược phân đoạn tạo nên một kiến trúc lưu trữ kép (dual-indexing architecture) phục vụ cho hai luồng truy xuất khác nhau trong hệ thống. Chiến lược phân đoạn kích thước cố định cung cấp nguyên liệu đầu vào cho quy trình xây dựng đồ thị tri thức, nơi các thực thể và quan hệ được trích xuất và lưu trữ trong cơ sở dữ liệu đồ thị, phục vụ cho các chế độ truy vấn cục bộ (local) và toàn cục (global) của LightRAG. Trong khi đó, chiến lược phân đoạn phân cấp đa phương thức cung cấp các phân đoạn có cấu trúc ngữ nghĩa chặt chẽ, được lưu trữ trong cơ sở dữ liệu vector kèm theo siêu dữ liệu phong phú (metadata), phục vụ cho việc truy xuất ngữ cảnh chính xác khi hệ thống cần bổ sung thông tin chi tiết từ tài liệu gốc. Kiến trúc kép này đảm bảo rằng hệ thống vừa có khả năng lập luận đa bước trên đồ thị tri thức, vừa có khả năng truy xuất ngữ cảnh tài liệu gốc một cách trung thực và đầy đủ.

Phương pháp thứ nhất - phân đoạn theo kích thước cố định (fixed-size chunking) - chia văn bản thành các đoạn có độ dài xác định trước (thường

từ 512 đến 1200 token), kèm theo một vùng chồng lấp (overlap window) chiếm khoảng 10–20% kích thước đoạn giữa hai đoạn liên tiếp. Cơ chế chồng lấp này đóng vai trò như một “cầu nối ngữ cảnh”: khi mô hình xử lý đoạn thứ i , nó đã có sẵn phần đuôi của đoạn thứ $i-1$ làm nền tảng, từ đó tránh được hiện tượng cắt đứt giữa chừng một câu hoặc một luận điểm quan trọng.

Phương pháp thứ hai - phân đoạn theo cấu trúc tự nhiên (structure-aware chunking) - khai thác các dấu hiệu cấu trúc sẵn có trong tài liệu như tiêu đề, tiêu đề phụ, danh sách có đánh số, hay các đoạn văn được phân tách bởi ký tự xuống dòng kép. Hệ thống sử dụng các biểu thức chính quy (regex) kết hợp với phân tích cú pháp nhẹ (lightweight parsing) để nhận diện ranh giới tự nhiên này. Ưu điểm của phương pháp này là mỗi đoạn văn giữ nguyên tính nguyên vẹn ngữ nghĩa - ví dụ, toàn bộ một mục trong tài liệu kỹ thuật sẽ không bị tách ra giữa chừng.

3.1.3.1 Lựa chọn mô hình nhúng (Embedding Model)

Để tối ưu hóa phân hệ truy xuất ngữ cảnh (Retriever) trong kiến trúc RAG và cải thiện trực tiếp chỉ số độ gọi bối cảnh (Context Recall), hệ thống đề xuất sử dụng mô hình nhúng mã nguồn mở `intfloat/multilingual-e5-large-instruct` được phát triển bởi Microsoft Research [33]. Mô hình này sở hữu kiến trúc biểu diễn hai chiều Encoder sâu sắc, được huấn luyện dựa trên tập ngữ liệu khổng lồ đa ngôn ngữ và đã chứng minh năng lực vượt trội khi đạt điểm số cao nhất trong bảng xếp hạng năng lực xử lý tiếng Việt tiêu chuẩn VN-MTEB (Vietnamese Massive Text Embedding Benchmark) công bố tại hội nghị khoa học EACL 2026 [27]. Qua nghiên cứu thực nghiệm, sự lựa chọn này đem lại các giá trị tối ưu vượt bậc cùng chiến lược xử lý các hạn chế vật lý bao gồm các khía cạnh cụ thể.

Về cơ chế học có chỉ dẫn theo tác vụ (Task-aware Instruction Tuning), điểm cải tiến cốt lõi của phiên bản Instruct so với các kiến trúc nhúng phẳng thông thường là khả năng hiểu sâu sắc mục đích nghiệp vụ thông qua câu lệnh chỉ dẫn hệ thống [14]. Trước khi thực hiện vector hóa, hệ thống sẽ tự động cấu hình chèn tiền tố ngữ cảnh định hướng: chèn `"query:"` vào trước câu hỏi thảo luận học vụ của sinh viên và chèn `"passage:"` vào trước các

phân đoạn tài liệu bối cảnh thô. Cơ chế này giúp mô hình chủ động căn chỉnh ma trận biểu diễn không gian, kéo các vector câu hỏi và vector bối cảnh chứa câu trả lời học vụ lại gần nhau hơn, giải quyết bài toán truy xuất bối cảnh chính xác [33].

Về khả năng căn chỉnh đa ngữ (Cross-lingual Alignment), do đặc thù của kho tri thức học tập ngành Công nghệ thông tin đan xen dày đặc thuật ngữ tiếng Anh và tiếng Việt, mô hình E5 có khả năng ánh xạ các thực thể ngữ nghĩa đồng dạng của hai ngôn ngữ về cùng các vùng lân cận trong không gian vector 1024 chiều [33]. Điều này giúp hệ thống phản hồi chính xác về bối cảnh ngay cả khi sinh viên đặt câu hỏi bằng tiếng Việt nhưng tài liệu đề cương được lưu trữ lại bằng tiếng Anh [33].

Về hạn chế kỹ thuật của giới hạn vật lý và giải pháp tối ưu, hạn chế lớn nhất của mô hình nhúng này là cửa sổ ngữ cảnh đầu vào (Context Window) bị giới hạn nghiêm ngặt ở mức 512tokens [33]. Nếu phân đoạn văn bản vượt quá giới hạn này, mô hình sẽ kích hoạt cơ chế tự động cắt tỉa phần đuôi, gây thất thoát nghiêm trọng thông tin cấu trúc môn học. Để xử lý triệt để hạn chế này, hệ thống LightRAG đã thu hẹp cấu hình phân đoạn tịnh tiến (Chunking), hạ kích thước mỗi mảnh (Chunk Size) từ 1200 tokens mặc định xuống mức 400–500 tokens, phối hợp với cửa sổ dịch chuyển chồng lấp 50 tokens. Chiến lược này giúp bối cảnh văn bản lọt gọn vào giới hạn vật lý của mô hình biểu diễn mà vẫn bảo toàn ngữ nghĩa.

Về mặt kiến trúc vận hành thực tế hệ thống, mô hình nhúng mở multilingual-e5-large-instruct sở hữu dung lượng tham số ở mức trung bình (560 triệu tham số) [33], mở ra hai kịch bản triển khai linh hoạt đáp ứng hoàn hảo từng môi trường hạ tầng:

3.1.4 Cơ chế trích xuất thực thể và xây dựng quan hệ

Sau khi văn bản được chia thành các phân đoạn (chunk), LightRAG thực hiện quy trình trích xuất tri thức để chuyển đổi dữ liệu phi cấu trúc thành các thành phần đồ thị tri thức tường minh. Khác với các hệ thống RAG truyền thống thường bỏ qua mối liên kết giữa các đoạn văn, cơ chế này tập trung vào việc nhận diện các thực thể (Entities) và thiết lập các quan hệ (Relationships) xuyên suốt tài liệu.

3.1.4.1 Chiến lược trích xuất đa nhiệm trong một lần gọi (One-step Extraction)

Để tối ưu hóa chi phí tính toán và duy trì tính nhất quán ngữ nghĩa, LightRAG sử dụng một lời nhắc có cấu trúc (structured prompt) yêu cầu mô hình ngôn ngữ lớn (LLM) thực hiện đồng thời hai nhiệm vụ trích xuất trong một lần gọi API duy nhất.

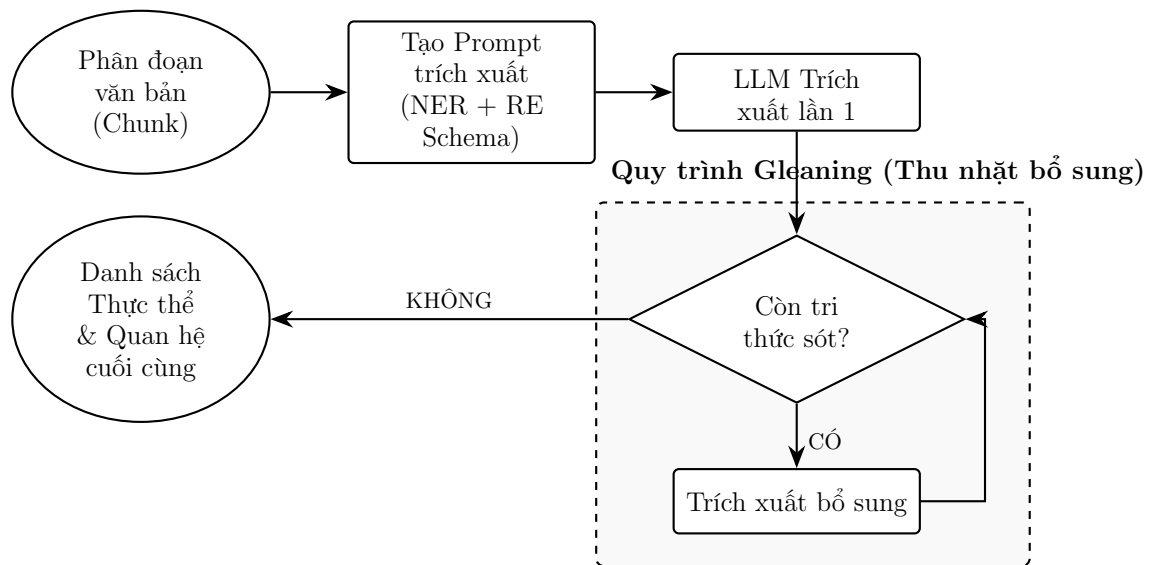
Nhiệm vụ 1: Nhận diện thực thể (Entity Extraction). Mô hình được yêu cầu xác định tất cả các thực thể quan trọng xuất hiện trong chunk. Đối với mỗi thực thể, LLM phải trích xuất các thuộc tính cốt lõi bao gồm tên định danh (Entity Name), loại thực thể (Entity Type) như PERSON, ORGANIZATION, CONCEPT, EVENT,... và một đoạn văn bản ngắn mô tả (Entity Description) giải thích vai trò, đặc điểm và ngữ cảnh của thực thể đó. Khác với NER truyền thống [21], phương pháp này khai thác tối đa khả năng hiểu ngữ nghĩa sâu của LLM để tạo ra các mô tả tri thức phong phú.

Nhiệm vụ 2: Trích xuất quan hệ (Relation Extraction). Khác với các hệ thống đồ thị tri thức truyền thống chỉ sử dụng cấu trúc bộ ba (triple), hệ thống mở rộng biểu diễn thành cấu trúc bộ tứ (quadruple) với các thuộc tính. Sau khi xác định các thực thể, mô hình thực hiện quét các cặp thực thể để tìm kiếm các mối liên kết có ý nghĩa. Một quadruple trích xuất bao gồm hai thực thể tham gia (Source & Target), các từ khóa định nghĩa bản chất liên kết (Relationship Keywords) và mô tả chi tiết (Relationship Description) lý do tại sao hai thực thể này có liên quan đến nhau dựa trên bằng chứng trong văn bản.

3.1.4.2 Cơ chế Gleaning: Tự chỉnh sửa và Thu nhặt bổ sung

Một trong những thách thức lớn nhất của việc trích xuất tri thức bằng LLM là hiện tượng bỏ sót thông tin do giới hạn về khả năng chú ý (attention) khi xử lý các đoạn văn phức tạp. LightRAG giải quyết vấn đề này thông qua kỹ thuật *Gleaning* (thu nhặt bổ sung).

Sau vòng trích xuất đầu tiên, hệ thống không kết thúc ngay mà thực hiện gửi lại danh sách tri thức vừa tìm được cho LLM kèm theo một yêu cầu phản biện: “Dựa trên đoạn văn này và danh sách thực thể/quan hệ đã có, liệu còn bất kỳ chi tiết nào bị bỏ sót không?”. Nếu mô hình phát hiện thêm tri thức mới, mô hình sẽ trả về kết quả bổ sung để hệ thống hợp nhất vào kết quả ban đầu. Quy trình này lặp lại cho đến khi mô hình xác nhận không còn thông tin sót hoặc đạt giới hạn số vòng lặp cấu hình.



Hình 3.5: Luồng logic quy trình trích xuất tri thức và cơ chế Gleaning

Kết quả cuối cùng là một tập hợp các thực thể và quan hệ có cấu trúc, sẵn sàng cho bước hợp nhất đồ thị (Graph Merging). Cơ chế này đảm bảo rằng mỗi đoạn văn bản đều được khai thác triệt để tri thức, tạo tiền đề cho việc xây dựng một đồ thị tri thức dày đặc và chính xác.

3.1.5 Luồng xử lý Prompt linh hoạt cho đa dạng nguồn dữ liệu

Một thách thức lớn trong việc xây dựng đồ thị tri thức cho môi trường học thuật là sự khác biệt sâu sắc về cấu trúc và ngôn từ giữa các nguồn

tài liệu. Đề cương chi tiết môn học thường mang tính quy phạm, ngắn gọn và sử dụng dày đặc các mã ký hiệu viết tắt như G1, G2 (mục tiêu môn học), CLO (chuẩn đầu ra), hay các thuật ngữ quản lý đào tạo như tích lũy, tiên quyết. Trong khi đó, dữ liệu từ hệ thống website của khoa lại mang tính chất thông tin đại chúng, chứa nhiều tin tức, sự kiện và mô tả tổng quát. Một lời nhắc trích xuất tổng quát duy nhất khó có thể bao quát tốt cả hai dạng tài liệu này: lời nhắc tối ưu cho đề cương sẽ bỏ sót ngữ cảnh tin tức, còn lời nhắc tối ưu cho website lại không nhận diện được các mã ký hiệu chuyên ngành. Để giải quyết vấn đề này, hệ thống triển khai mô-đun luồng xử lý lời nhắc linh hoạt (Dynamic Prompt Flow) đặt tại `lightrag/prompt_flow`, với cơ chế tự động điều phối lời nhắc theo miền tri thức (domain) của từng phân đoạn.

Về mặt thiết kế, mô-đun áp dụng đồng thời ba mẫu thiết kế phần mềm: *Registry*, *Strategy* và *Protocol*. Mỗi miền tri thức được hiện thực thành một mô-đun chiến lược (strategy) tuân thủ giao diện `EntityExtractionModule`, bắt buộc cung cấp đúng bốn hằng số lời nhắc: lời nhắc hệ thống (`ENTITY_EXTRACTION_SYSTEM_PROMPT`), tập ví dụ mẫu (`ENTITY_EXTRACTION_EXAMPLES`), lời nhắc người dùng (`ENTITY_EXTRACTION_USER_PROMPT`) và lời nhắc thu nhật bổ sung (`ENTITY_CONTINUE_EXTRACTION_USER_PROMPT`). Bộ điều phối (`PromptDispatcher`) duy trì một sổ đăng ký (`Registry`) ánh xạ tên miền sang mô-đun tương ứng; khi gặp một nhãn miền chưa được đăng ký, hệ thống tự động lùi về miền mặc định `curriculum`. Thiết kế này cho phép bổ sung một miền tri thức mới mà không phải sửa đổi mã nguồn của tầng trích xuất, đảm bảo khả năng mở rộng.

Hệ thống hiện hỗ trợ bốn miền tri thức đặc thù cho dữ liệu đào tạo bậc đại học. Miền `curriculum` dành cho chương trình đào tạo, tập trung vào ngành, khoa, khối kiến thức, môn học, mục tiêu chương trình (PO), chuẩn đầu ra (ELO), số tín chỉ và điều kiện tiên quyết. Miền `course` dành cho đề cương chi tiết học phần, ưu tiên các chuẩn đầu ra môn học (CLO), nội dung chương, hình thức kiểm tra và tài liệu tham khảo. Miền `teacher` dành cho danh sách nhân sự giảng dạy, phân biệt giảng viên cơ hữu, thỉnh giảng và cán bộ hỗ trợ cùng học hàm, học vị. Cuối cùng, miền `resolution` dành cho các văn bản pháp lý, hành chính thuần túy như nghị quyết hội

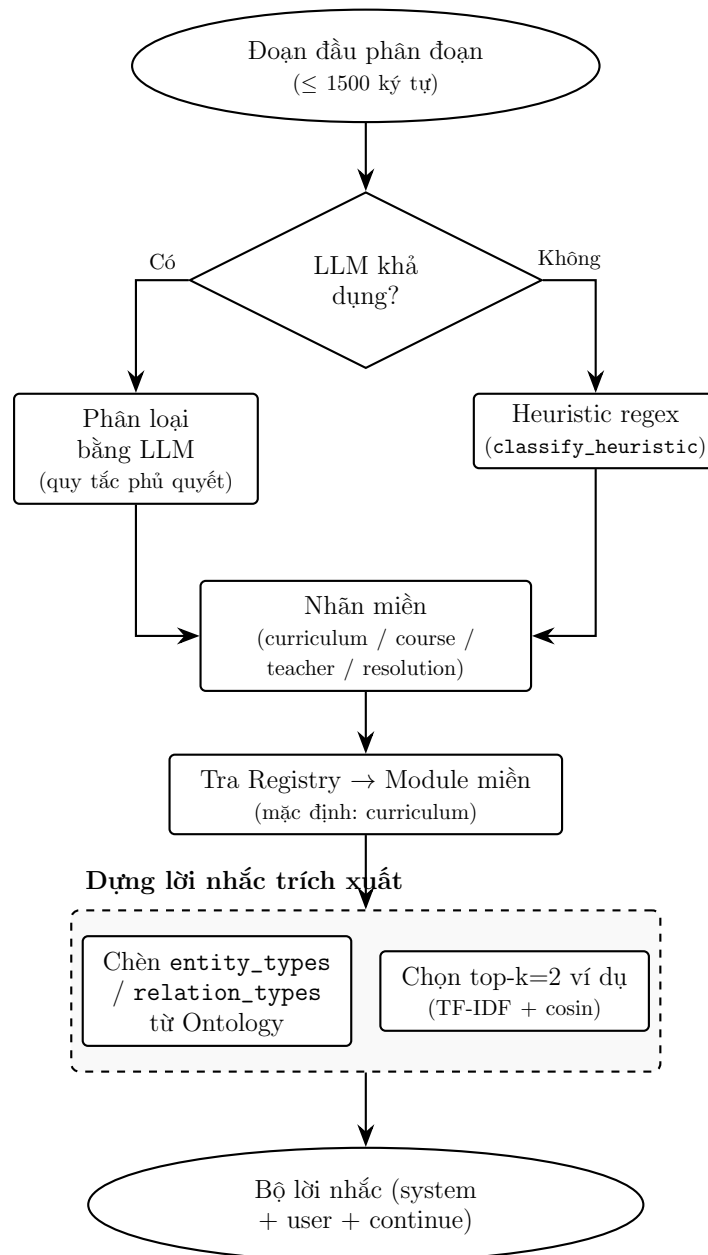
đồng trường hay quy chế tổ chức.

Việc xác định miền của một phân đoạn được thực hiện theo cơ chế hai tầng nhằm cân bằng giữa độ chính xác và khả năng chịu lỗi. Tầng thứ nhất sử dụng một mô hình ngôn ngữ lớn đóng vai trò bộ phân loại nhẹ (lightweight classifier): hệ thống chỉ gửi khoảng 1500 ký tự đầu của phân đoạn kèm theo một lời nhắc phân loại ngắn gọn, yêu cầu mô hình trả về đúng một nhãn miền. Do các văn bản giáo dục Việt Nam thường mở đầu bằng phần thủ tục hành chính (“QUYẾT ĐỊNH V/v Ban hành...”, “Căn cứ...”), lời nhắc phân loại được bổ sung một quy tắc phủ quyết (critical override): nếu phần mở đầu hành chính này thực chất nhằm ban hành chương trình đào tạo hoặc đề cương môn học, văn bản phải được phân vào **curriculum** hoặc **course** thay vì bị nhầm thành **resolution**. Tầng thứ hai là một bộ phân loại dựa trên biểu thức chính quy và tính điểm theo từ khóa đặc trưng (**classify_heuristic**), đóng vai trò phương án dự phòng khi mô hình ngôn ngữ không khả dụng hoặc trả về kết quả không hợp lệ.

Sau khi xác định được miền, bộ điều phối nạp các kiểu thực thể và kiểu quan hệ đặc thù từ lớp bản thể học (ontology) của miền đó vào các vị trí giữ chỗ **{entity_types}** và **{relation_types}** trong lời nhắc. Ví dụ, miền **curriculum** định nghĩa các kiểu thực thể như **Major**, **KnowledgeBlock**, **Course**, **LearningOutcome** (ELO), **ProgramObjective** (PO) cùng các kiểu quan hệ như **PREREQUISITE_OF** (học trước), **CONTAINS** (khối kiến thức chứa môn học) hay **MAPS_TO** (ELO liên kết với PO); khi miền không khai báo bản thể học, hệ thống dùng quan hệ tổng quát **RELATES_TO** làm mặc định. Việc áp đặt sẵn bộ kiểu chuyên biệt giúp định hướng mô hình ngôn ngữ tập trung vào tri thức có giá trị nhất của miền và duy trì tính nhất quán về định danh (Entity Canonicalization) ngay từ bước sơ khởi, từ đó giảm thiểu số lượng thực thể phân mảnh và tạo điều kiện cho việc hợp nhất các nút đồng nghĩa ở các bước sau. Cần lưu ý rằng ở giai đoạn hiện tại, việc đối chiếu kết quả trích xuất với bản thể học chỉ dừng ở mức ghi cảnh báo (warn-only), còn cơ chế loại bỏ tự động dữ liệu vi phạm được để ngỏ như một điểm mở rộng trong tương lai.

Để kiểm soát chi phí token, mô-đun không chèn toàn bộ tập ví dụ mẫu vào mỗi lần gọi mà áp dụng cơ chế chọn ví dụ động (dynamic few-shot selection). Một bộ chỉ mục TF-IDF kết hợp độ tương đồng cosin được xây

dựng thuần Python (không phụ thuộc thư viện ngoài) để xếp hạng các ví dụ theo mức độ liên quan với nội dung phân đoạn hiện tại, sau đó chỉ giữ lại hai ví dụ ($\text{top-k} = 2$) phù hợp nhất. Cách làm này giảm đáng kể lượng token tiêu thụ so với việc nhúng toàn bộ ví dụ, đồng thời vẫn bảo toàn chất lượng trích xuất nhờ luôn cung cấp đúng ngữ cảnh mẫu sát với phân đoạn. Trong trường hợp việc xếp hạng gặp lỗi, hệ thống tự động lùi về phương án trả về các ví dụ đầu danh sách, bảo đảm quy trình không bị gián đoạn. Hình 3.6 mô tả tổng quan luồng điều phối lời nhắc từ phân đoạn đầu vào đến bộ lời nhắc trích xuất hoàn chỉnh.



Hình 3.6: Luồng điều phối lời nhắc linh hoạt theo miền tri thức (Dynamic Prompt Flow)

3.1.6 Kiểm chứng tri thức bằng chuẩn SHACL và đối chiếu bằng chứng xác định

Để đảm bảo tính toàn vẹn và chất lượng của dữ liệu tri thức trước khi đưa vào đồ thị chính thức, hệ thống tích hợp một bộ kiểm chứng lấy cảm hứng từ chuẩn SHACL (Shapes Constraint Language) [32]. SHACL cung cấp một ngôn ngữ để mô tả các ràng buộc về cấu trúc và ngữ nghĩa của dữ liệu đồ thị, cho phép tự động hóa việc phát hiện các lỗi phổ biến trong kết quả trích xuất bằng mô hình ngôn ngữ lớn [18]. Toàn bộ logic kiểm chứng được hiện thực trong mô-đun `lightrag/shacl`, tổ chức thành một quy trình đánh giá nghiêm ngặt và đối chiếu bằng chứng (deterministic text-grounding) thực thi trên kết quả trích xuất của từng phân đoạn.

Lớp bảo vệ thứ nhất (Guard 1) – Khử trùng lặp. Hệ thống đối chiếu mã định danh phân đoạn (`chunk_id`) với kho lưu trữ hiện có. Nếu phân đoạn đã tồn tại, hệ thống chuyển sang trạng thái nạp lại (`RE_INGEST`) và điều chỉnh cách tính bậc kết nối của thực thể để tránh cộng dồn trùng lặp; ngược lại, phân đoạn mới được cho phép tiếp tục (`PROCEED`). Lớp này ngăn việc xử lý lại dữ liệu dư thừa và bảo vệ tính toàn vẹn của đồ thị.

Lớp bảo vệ thứ hai (Guard 2) – Kiểm tra cấu trúc và đối chiếu bằng chứng. Đây là trọng tâm của quy trình. Mỗi thực thể và quan hệ được kiểm tra qua một tập các định nghĩa hình dạng (Shapes). Đối với thực thể, các quy tắc bao gồm: bắt buộc có tên hợp lệ (`missing_name`, mức lỗi), bắt buộc có kiểu xác định khác “UNKNOWN” (`missing_type`, mức lỗi), bắt buộc có siêu dữ liệu nguồn gốc `source_id` (`no_provenance`, mức lỗi), và cảnh báo thực thể bị cô lập khi số kết nối nhỏ hơn ngưỡng tối thiểu (`isolated_entity`). Đáng chú ý, số kết nối của một thực thể được tính bằng cách kết hợp bậc cục bộ (trong phạm vi phân đoạn) với bậc toàn cục truy vấn đồng thời từ cơ sở dữ liệu đồ thị, nhằm tránh báo cô lập sai đối với những thực thể đã có liên kết ở các phân đoạn trước. Đối với quan hệ, hệ thống kiểm tra sự tồn tại của thực thể nguồn và đích, phát hiện quan hệ tự trở (`self_loop`), độ dài tối thiểu của mô tả (mặc định 15 ký tự) và dải giá trị hợp lệ của trọng số trong khoảng $[0, 1]$. Ngoài ra, một bộ chặn ngôn ngữ (language guard) sẽ từ chối mọi giá trị chứa ký tự nằm ngoài bảng chữ Latinh và tiếng Việt (`contains_foreign_script`), loại bỏ các

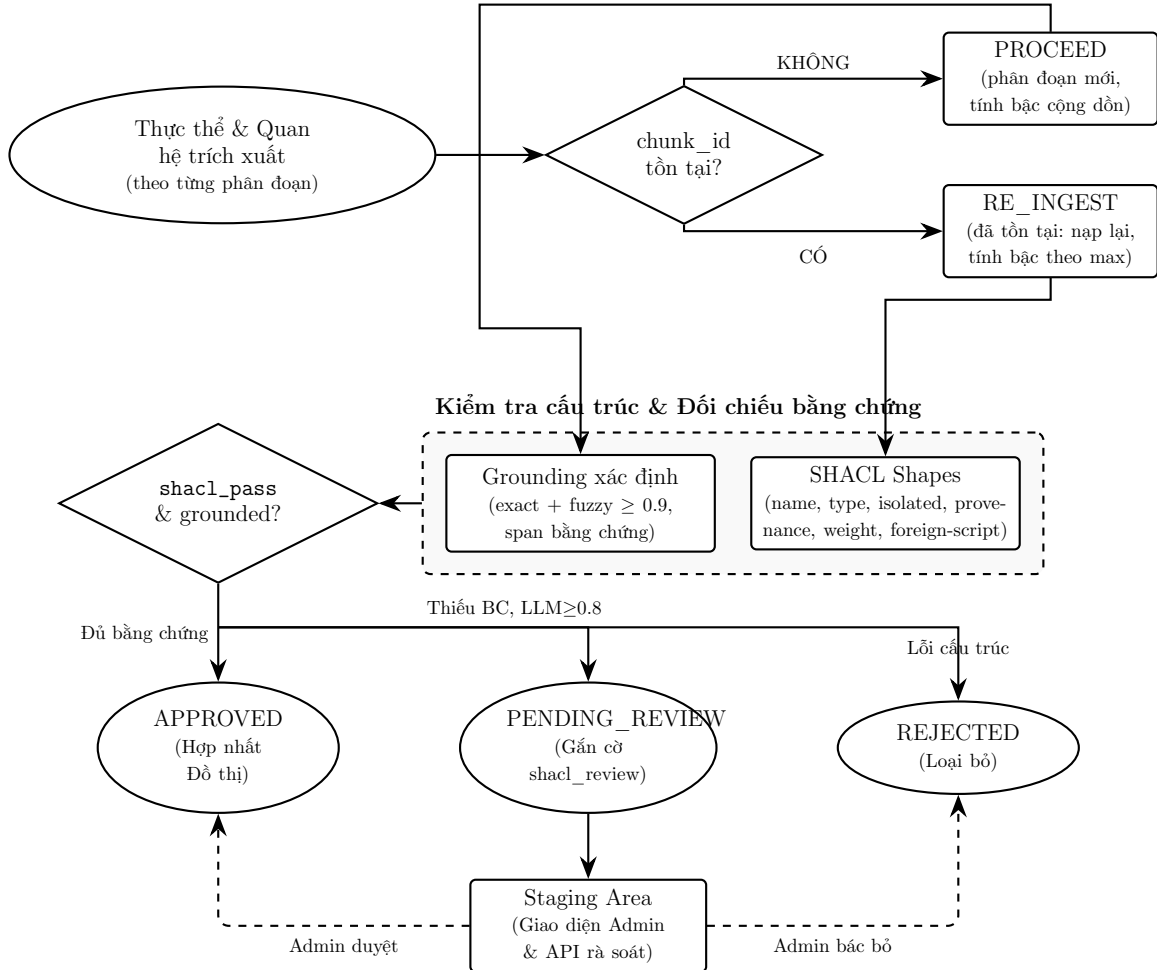
trường hợp mô hình ngôn ngữ vô tình sinh ra ký tự ngoại lai. Mỗi vi phạm được gán một trong ba mức nghiêm trọng: lỗi (error), cảnh báo (warning) hoặc thông tin (info).

Điểm khác biệt cốt lõi của bộ kiểm chứng là cơ chế đối chiếu bằng chứng văn bản một cách xác định (deterministic text-grounding). Thay vì hỏi mô hình ngôn ngữ “thực thể này có trong văn bản không?” rồi tin vào câu trả lời, hệ thống tự tính toán bằng chứng: trước tiên tìm khớp chính xác chuỗi con (điểm số 1.0), nếu không có thì dùng so khớp mờ (fuzzy matching qua `rapidfuzz` hoặc `difflib`) với ngưỡng chấp nhận 0.9. Kết quả trả về không phải một ý kiến mà là chính đoạn văn bản khớp được kèm theo vị trí ký tự trong phân đoạn – đóng vai trò bằng chứng có thể kiểm tra lại. Đối với quan hệ, hệ thống yêu cầu cả hai đầu mút đều có bằng chứng và đồng xuất hiện trong cùng một cửa sổ 200 ký tự, qua đó củng cố độ tin cậy rằng quan hệ là có thật chứ không phải hai lần nhắc rời rạc.

Quyết định cuối cùng cho mỗi thực thể và quan hệ được tổng hợp tại tầng ra quyết định thuần túy theo nguyên tắc phân cấp niềm tin: các kiểm tra xác định (cấu trúc và bằng chứng văn bản) giữ quyền quyết định tuyệt đối, thay vì hoàn toàn phụ thuộc vào điểm số tự tin của LLM phản biện. Cụ thể, nếu vi phạm cấu trúc ở mức lỗi, dữ liệu bị từ chối (**REJECTED**); nếu hợp lệ về cấu trúc và có bằng chứng văn bản xác định, dữ liệu được phê duyệt (**APPROVED**); nếu hợp lệ về cấu trúc nhưng thiếu bằng chứng (ví dụ suy luận ngầm ẩn), dữ liệu chỉ được đưa vào trạng thái chờ xem xét (**PENDING_REVIEW**) khi độ tin cậy của LLM phản biện đạt ngưỡng 0.8. Nghĩa là, LLM chỉ được phép đẩy các trường hợp ranh giới vào diện xem xét, chứ không bao giờ được quyền tự ý chấp nhận một tri thức vô căn cứ.

Kết quả kiểm chứng điều hướng trực tiếp quá trình hợp nhất dữ liệu vào đồ thị. Các mục bị từ chối (**REJECTED**) sẽ bị loại bỏ ngay lập tức, ngăn chặn việc làm ô nhiễm đồ thị. Các mục hợp lệ (**APPROVED**) được hợp nhất bình thường. Đối với các mục chờ xem xét (**PENDING_REVIEW**), quy trình quản lý dữ liệu chờ duyệt này được đưa vào một vùng đệm lưu trữ gọi là Staging Area (Cơ sở dữ liệu Staging). Tại đây, người quản trị có thể can thiệp thủ công thông qua giao diện quản trị hoặc các API để rà soát chi tiết từng thực thể hoặc mối quan hệ (Staging Review): nếu được phê duyệt (Admin Approved), dữ liệu sẽ được chuyển trạng thái thành **APPROVED** và

hợp nhất vào đồ thị tri thức chính thức; ngược lại nếu bị bác bỏ (Admin Rejected), dữ liệu sẽ chuyển thành **REJECTED** và bị xóa hoàn toàn khỏi vùng đệm. Việc lưu vết minh bạch từng quyết định (kèm theo đoạn văn bản làm bằng chứng) đảm bảo rằng đồ thị luôn có cơ sở vững chắc và đáng tin cậy. Hình 3.7 minh họa quy trình kiểm chứng dựa trên SHACL kết hợp rà soát Staging Review.

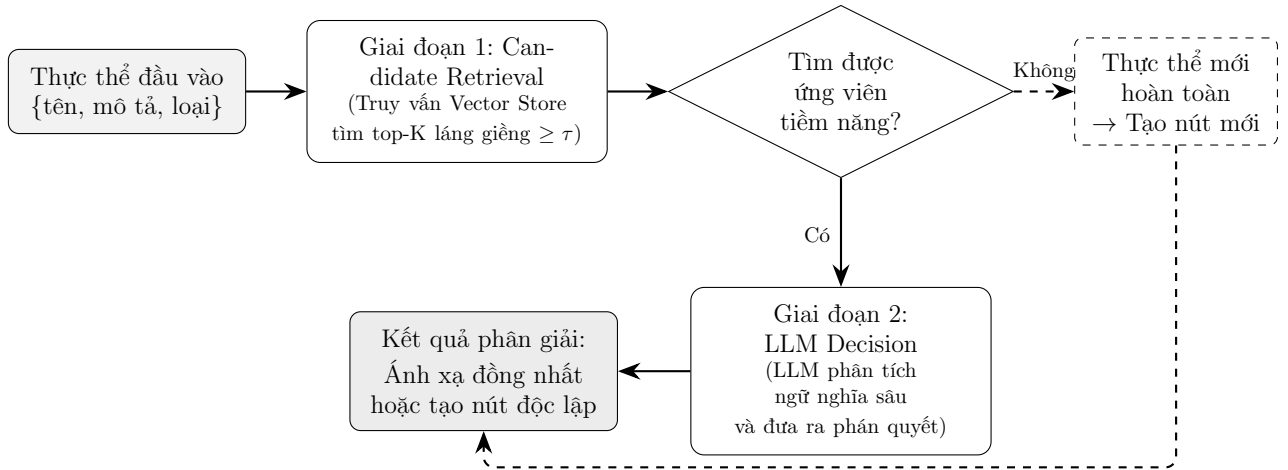


Hình 3.7: Quy trình kiểm chứng tri thức SHACL và đối chiếu bằng chứng xác định

3.1.7 Xử lý thực thể trùng lặp trong văn bản học thuật

Để xử lý vấn đề thực thể trùng lặp đảm bảo truy xuất đầy đủ thông tin, đề tài đề xuất một module phân giải thực thể (*entity resolution*) được tích hợp vào hai giai đoạn, hoạt động trước khi bất kỳ nút nào được ghi vào đồ thị. Module này được thiết kế theo kiến trúc hai giai đoạn xử lý nối tiếp: Candidate Retrieval (Truy hồi ứng viên) và LLM Decision (LLM đưa ra quyết định).

Cách tiếp cận này được xây dựng trên nền tảng của hai công trình nghiên cứu trong lĩnh vực căn chỉnh thực thể (*entity alignment*) giữa các knowledge graph: HELEA [15] chứng minh hiệu quả của pipeline hai giai đoạn: đầu tiên truy hồi ứng viên bằng embedding similarity, sau đó dùng mô hình ngôn ngữ lớn để xếp hạng lại (*reranking*). Cơ chế này vừa đảm bảo không bỏ sót ứng viên tiềm năng, vừa tận dụng khả năng suy luận ngữ nghĩa sâu của LLM. Kế tiếp là EasyEA [4] chứng minh rằng mô hình ngôn ngữ lớn, khi được cung cấp đủ ngữ cảnh, có khả năng đưa ra quyết định căn chỉnh thực thể chính xác theo phương thức *zero-shot* không cần dữ liệu huấn luyện nhân, phù hợp với bối cảnh corpus tiếng Việt học thuật hiện tại. Trong Hình 3.8 Trình bày kiến trúc tổng thể của nhất đồ thị.



Hình 3.8: Kiến trúc hai giai đoạn của module phân giải thực thể. Giai đoạn Candidate Retrieval sử dụng embedding similarity để truy hồi các thực thể ngữ nghĩa tương đồng từ vector store. Giai đoạn LLM Decision chỉ được kích hoạt khi có ứng viên vượt ngưỡng τ , giúp tiết kiệm chi phí gọi LLM cho những thực thể hoàn toàn mới.

Giai đoạn 1: Candidate Retrieval. Mỗi thực thể cần phân giải được biểu diễn dưới dạng một vector embedding bằng cách kết hợp cả tên lẫn mô tả của thực thể đó thành một chuỗi đầu vào thống nhất. Cách biểu diễn này cho phép vector nắm bắt đồng thời hai chiều thông tin: chiều định danh (tên gọi) và chiều ngữ nghĩa (nội dung, thuộc tính). Sau đó, hệ thống thực hiện tìm kiếm láng giềng gần nhất trong vector store để truy hồi tối đa K thực thể đã tồn tại trong knowledge graph có vector gần nhất với thực thể đang xét. Ngưỡng tương đồng cosine τ đóng vai trò bộ lọc: chỉ những ứng viên có $\cos(\mathbf{v}_1, \mathbf{v}_2) \geq \tau$ mới được coi là *ứng viên tiềm năng* và đưa vào

giai đoạn tiếp theo. Cơ chế này tương đồng với giai đoạn truy hồi ứng viên trong kiến trúc HELEA [15], đảm bảo không bỏ sót các cặp thực thể có ngữ nghĩa gần nhau.

Giai đoạn 2: LLM Decision. Khi giai đoạn Candidate Retrieval xác định được ít nhất một ứng viên tiềm năng, mô hình ngôn ngữ lớn được kích hoạt để đưa ra phán quyết cuối cùng. Không giống với các phương pháp truyền thống chỉ dựa vào điểm số cosine similarity, LLM có khả năng suy luận ở mức ngữ nghĩa sâu hơn, cũng như phân biệt các thực thể bề ngoài tương tự nhưng thực chất khác biệt. Chẳng hạn, mô hình xác định “*Nhập môn Lập trình*” và “*Lập trình Hướng đối tượng*” là hai học phần khác nhau mặc dù đều thuộc nhóm lập trình, trong khi “*HCMUS*” và “*Trường Đại học Khoa học Tự nhiên*” là cùng một tổ chức.

3.1.7.1 Thiết kế quan hệ SAME_AS: Liên kết thay vì xóa bỏ

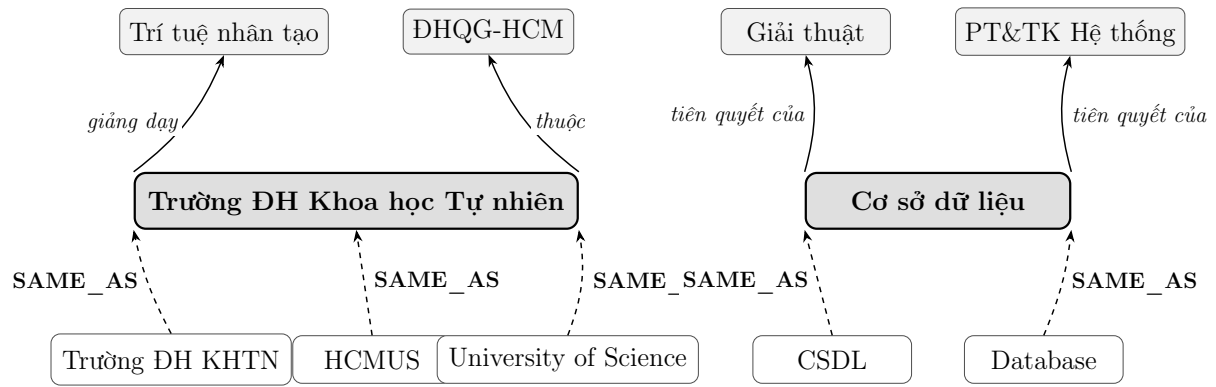
Khi mô hình ngôn ngữ lớn xác nhận hai thực thể là đồng nhất, hệ thống có 2 lựa chọn: *hợp nhất vật lý* hai nút thành một (xóa một nút, gộp toàn bộ dữ liệu vào nút còn lại), hay *tạo quan hệ liên kết* giữa hai thực thể này và giữ nguyên cả hai? Đề tài lựa chọn giải pháp thứ hai thông qua quan hệ SAME_AS.

Quyết định này không phải là sự thỏa hiệp mà xuất phát từ ba lý do cơ bản. *Thứ nhất*, cả HELEA lẫn EasyEA đều tiếp cận entity alignment theo hướng tạo ra *tập hợp cặp đồng nhất* giữa hai knowledge graph độc lập không ai trong số họ đề xuất xóa bỏ hoặc hợp nhất vật lý các nút. Quan hệ SAME_AS là biểu diễn ngữ nghĩa chuẩn của kết quả alignment, tương đương với thuộc tính owl:sameAs trong các chuẩn ngữ nghĩa web (*Semantic Web*) [23]. *Thứ hai*, hợp nhất vật lý là thao tác không thể hoàn tác: nếu mô hình ngôn ngữ lớn đưa ra quyết định nhầm, dữ liệu của nút bị xóa không thể phục hồi. Ngược lại, một quan hệ SAME_AS có thể được xóa bỏ đơn giản nếu phát hiện sai sót, trong khi toàn bộ thông tin gốc vẫn được bảo toàn. *Thứ ba*, thiết kế SAME_AS bảo toàn *nguồn gốc ngữ nghĩa (provenance)*: mỗi nút vẫn duy trì liên kết đến đúng các tài liệu đã sử dụng hình thức tên đó, phục vụ cho việc truy vết và kiểm chứng thông tin sau này.

Về mặt hành vi trong knowledge graph, thiết kế này cho phép hệ thống truy vấn tổng hợp thông tin từ cả hai nút thông qua phép duyệt cạnh SAME_AS. Ví dụ, khi người dùng hỏi về “HCMUS”, hệ thống không chỉ truy xuất thông tin gắn với nút “HCMUS” mà còn có thể traverse sang nút “Trường Đại học Khoa học Tự nhiên” để bổ sung thêm tri thức liên quan, cho ra câu trả lời toàn diện hơn.

3.1.7.2 Kết quả trong knowledge graph

Minh họa cấu trúc của knowledge graph sau khi áp dụng module phân giải thực thể với dữ liệu từ corpus học thuật.



Hình 3.9: Ví dụ cấu trúc knowledge graph sau khi áp dụng phân giải thực thể trên corpus học thuật HCMUS. Các nút viền đậm, màu xám đậm là thực thể dạng chuẩn (*canonical*); các nút màu trắng là hình thức tên thay thế (*alias*) được liên kết về nút chuẩn qua quan hệ SAME_AS (đường nét đứt). Quan hệ ngữ nghĩa thông thường (mũi tên nét liền) chỉ xuất phát từ nút chuẩn, đảm bảo tri thức không bị phân tán.

3.1.7.3 Đồng bộ hóa với vector store

Song song với việc xây dựng đồ thị quan hệ, hệ thống duy trì một cơ sở dữ liệu vector (*vector store*) được đồng bộ liên tục. Sau mỗi thao tác hợp nhất, toàn bộ thực thể và quan hệ - với mô tả đã được tổng hợp - đều được cập nhật vào vector store. Kiến trúc lưu trữ kép này (*dual storage*) là nền tảng cho hai chế độ truy vấn của LightRAG: đồ thị tri thức phục vụ duyệt cấu trúc quan hệ tường minh, trong khi vector store cho phép tìm kiếm ngữ nghĩa linh hoạt khi câu hỏi của người dùng không khớp chính xác với bất kỳ tên thực thể nào đã được lập chỉ mục.

3.1.7.4 So sánh với công trình liên quan

Bảng 3.3 tổng hợp mức độ tương đồng về mặt thiết kế giữa module phân giải thực thể của đề tài và hai công trình nghiên cứu tham chiếu.

Bảng 3.3: So sánh các thành phần thiết kế với công trình liên quan

Thành phần	HELEA	EasyEA	Phương án đề tài
Embedding kết hợp tên & mô tả	✓	✓	Áp dụng. Biểu diễn thực thể qua vector kết hợp hai chiều ngữ nghĩa.
Truy hồi Top- K qua vector similarity	✓	✓	Áp dụng với lọc ngưỡng cosine τ .
LLM ra quyết định cuối cùng	✓	✓	Áp dụng theo zero-shot.
Quan hệ SAME_AS (thay vì merge vật lý)	✓	✓	Áp dụng. Bảo toàn cả hai nút và provenance.
Ngữ cảnh triple láng giềng trong prompt	—	✓	Chưa tích hợp. Xác định là hướng phát triển tiếp theo.
Fine-tuning encoder với hard negatives	✓	—	Không áp dụng (chọn hướng zero-shot).

Hai điểm khác biệt chính so với các công trình tham chiếu cần được làm rõ. Đối với việc không fine-tuning encoder, đây là quyết định có chủ ý: corpus tiếng Việt trong lĩnh vực học thuật gần như không có tập dữ liệu căn chỉnh thực thể được gán nhãn, khiến việc huấn luyện không khả thi. Phương án zero-shot mà EasyEA kiểm chứng giải quyết đúng hạn chế này. Đối với ngữ cảnh triple láng giềng, EasyEA chỉ ra rằng việc bổ sung các bộ ba quan hệ liên quan của ứng viên vào prompt giúp LLM phân biệt tốt hơn các trường hợp thực thể mơ hồ - ví dụ, một thực thể là *học phần* và một thực thể là *phần mềm* có thể cùng mang tên “Cơ sở dữ liệu” nhưng có bộ quan hệ hoàn toàn khác nhau trong knowledge graph. Việc tích hợp thành phần này được xác định là hướng phát triển ưu tiên trong giai đoạn tiếp theo.

3.1.8 Quản lý và duy trì bền vững hệ thống

LightRAG thiết kế hệ thống quản trị với triết lý nhất quán tăng dần (incremental consistency): thay vì xây dựng lại toàn bộ đồ thị mỗi khi có tài liệu mới, hệ thống chỉ xử lý các đoạn văn mới hoặc đã thay đổi, sau đó hợp nhất kết quả vào đồ thị hiện có. Cơ chế làm sạch tự động (auto-cleaning) thường xuyên kiểm tra tính nhất quán giữa đồ thị tri thức và nội dung tài liệu gốc: nếu một tài liệu bị xóa khỏi kho dữ liệu, tất cả các nút và cạnh có nguồn gốc duy nhất từ tài liệu đó sẽ tự động bị loại bỏ, trong khi các nút được hỗ trợ bởi nhiều nguồn sẽ chỉ được cập nhật lại mô tả.

Người quản trị có thể can thiệp thủ công thông qua các API quản trị: hiệu chỉnh nhãn thực thể (entity label editing), cập nhật mô tả hoặc hợp nhất thủ công hai thực thể bị nhận diện nhầm là khác nhau. Sau bất kỳ thay đổi nào, hệ thống kích hoạt quy trình đồng bộ hóa tự động: véc-tơ nhúng của các thực thể bị ảnh hưởng được tính toán lại và cập nhật trong cơ sở dữ liệu vector, đảm bảo sự nhất quán tuyệt đối giữa hai lớp lưu trữ.

Cuối cùng, LightRAG hỗ trợ tùy chỉnh lời nhắc trích xuất theo lĩnh vực (domain-specific prompt tuning): người dùng có thể cung cấp danh sách các kiểu thực thể ưu tiên và các mẫu quan hệ đặc thù, từ đó định hướng mô hình tập trung vào tri thức có giá trị nhất trong từng miền chuyên môn cụ thể.

3.2 Mô hình hóa trạng thái hệ thống (Agent State)

Phân hệ trợ lý ảo được xây dựng trên nền tảng LangGraph và thiết kế dưới dạng một đồ thị trạng thái có hướng (state graph), trong đó mỗi nút đại diện cho một tác vụ tính toán và các cạnh đại diện cho luồng điều khiển. Trạng thái dùng chung của hệ thống được mô hình hóa bằng kiểu **AgentState** và có thể biểu diễn bởi tập hợp $S = \{Q, U, H, A, D, K, M, G, F\}$. Trong đó, Q (Query) là câu hỏi đầu vào, U (User ID) là mã định danh người dùng phục vụ cá nhân hóa, và H (Chat History) là lịch sử hội thoại ngắn hạn đã được tóm tắt. Thành phần A (Analysis) lưu kết quả phân tích ý định, D (Decision) là quyết định điều phối với ba giá trị `direct_answer`, `retrieve` hoặc `reject`, còn K (Key-

words) là tập từ khóa truy xuất được tách thành hai mức cao và thấp. Cuối cùng, M (Long-term Memory) là hồ sơ người dùng dài hạn, G (Generation Request) là gói ngữ cảnh đầy đủ để sinh câu trả lời, và F (Final Response) là câu trả lời cuối cùng trả về cho người dùng. So với các kiến trúc có vòng lặp tự đánh giá, hệ thống đề xuất chủ trương giữ luồng điều phối tuyến tính và tắt định để tối ưu độ trễ phản hồi, dồn chất lượng vào khâu phân tích đầu vào và truy xuất tri thức thay vì hậu kiểm nhiều vòng.

3.2.1 Kiến trúc các tác vụ (Agents)

Hệ thống bao gồm năm thành phần xử lý chuyên biệt, được hiện thực dưới dạng các hàm trạng thái (stateless functions) và phối hợp theo một đồ thị có hướng do LangGraph điều phối, thay vì các tác nhân tự hành lập kế hoạch nhiều lượt.

Thứ nhất, Analyze Agent (`analyze_agent`) là thành phần trung tâm của khâu định tuyến. Bằng một lần gọi mô hình ngôn ngữ lớn với định dạng đầu ra JSON, agent này phân loại ý định người dùng, đưa ra quyết định điều phối (`direct_answer`, `retrieve` hoặc `reject`), chọn chế độ truy vấn LightRAG, viết lại câu hỏi thành dạng độc lập (standalone query), và trích xuất hai tập từ khóa: từ khóa mức thấp (cụ thể, như mã môn học, tên giảng viên) và mức cao (khái quát, như chính sách, so sánh). Agent cũng đảm nhiệm vai trò kiểm duyệt: khi phát hiện nội dung không phù hợp, quyết định bị ép sang `reject`.

Thứ hai, Generator Agent (`generator`) chịu trách nhiệm dựng ngữ cảnh sinh chứ không trực tiếp gọi mô hình. Với nhánh truy xuất, persona của trợ lý được đặt vào trường `user_prompt` để LightRAG vẫn giữ được khuôn mẫu nội bộ và chèn ngữ cảnh tri thức `context_data`; với nhánh trả lời trực tiếp, agent dựng một `system_prompt` đầy đủ; còn với nhánh từ chối, agent trả về một thông điệp từ chối lịch sự đã soạn sẵn.

Thứ ba, Summarizer Agent (`summarizer`) nén lịch sử hội thoại khi vượt quá bốn lượt thành một đến hai câu súc tích, nhằm giảm số token mà Analyze Agent phải xử lý và duy trì ngữ cảnh cốt lõi qua các phiên dài.

Thứ tư, Long-term Memory Agent (`longterm`) quản lý hồ sơ người dùng bền vững trên MongoDB. Thay vì tóm tắt lại toàn bộ hồ sơ, agent

chỉ trích xuất các sự kiện ổn định (chuyên ngành, sở thích trả lời, định hướng học tập dài hạn) dưới dạng JSON rồi hợp nhất các thông tin này vào hồ sơ hiện có bằng mã Python có khử trùng lặp, tránh hiện tượng trôi dạt nội dung tóm tắt.

Cuối cùng, LLM Factory (11m) là lớp trừu tượng hóa nhà cung cấp mô hình, cho phép chuyển đổi linh hoạt giữa OpenAI và Ollama (mô hình cục bộ) qua biến môi trường và tự chuẩn hóa các tham số khác biệt giữa hai nền tảng.

3.2.2 Quy trình điều phối và Luồng dữ liệu

Đồ thị điều phối được biên dịch cùng một bộ lưu điểm kiểm tra (checkpoint) trên MongoDB và gồm bốn nút thực thi tuần tự: **analyzer** → **prepare_context** → **generate** → (có điều kiện) **update_long_term** → **END**. Quy trình thực thi được mô tả qua các bước logic sau:

Bước 1: Phân tích và Định tuyến. Khi nhận câu hỏi Q , nút **analyzer** gọi *Analyze Agent* (sau khi đã tóm tắt lịch sử qua *Summarizer*) để xác định ý định, quyết định điều phối, chế độ truy vấn và các từ khóa. Kết quả phân tích được chuẩn hóa: nếu nội dung không phù hợp thì quyết định bị ép sang **reject**; nếu giá trị không hợp lệ thì lùi an toàn về **retrieve**.

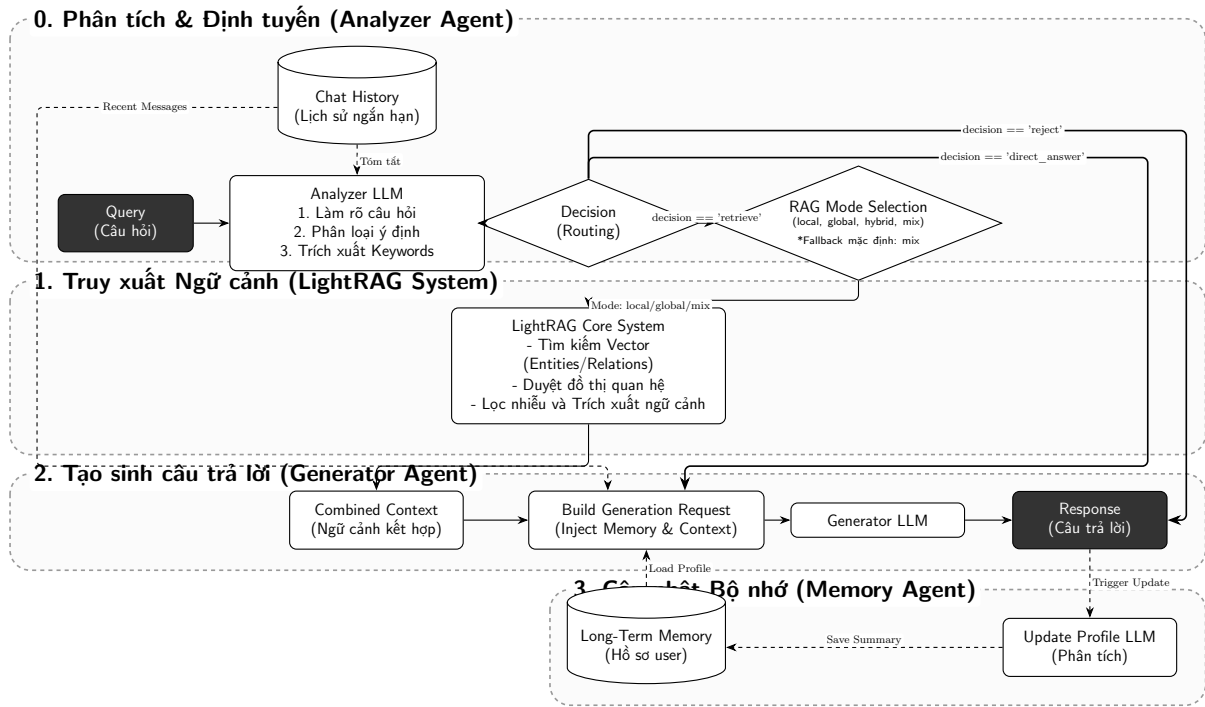
Bước 2: Chuẩn bị ngữ cảnh. Nút **prepare_context** không gọi mô hình ngôn ngữ. Nút này chỉ nạp bộ nhớ dài hạn M từ MongoDB khi khâu phân tích xác định là cần thiết, rồi dựng gói yêu cầu sinh G (**generation_request**) chứa câu hỏi, chế độ truy vấn, persona và các từ khóa. Việc tách riêng bước này giúp luồng phát trực tuyến (streaming) có thể dừng tại đây trước khi sinh.

Bước 3: Sinh câu trả lời. Nút **generate** phân nhánh theo quyết định điều phối: với **retrieve**, hệ thống gọi LightRAG kèm các từ khóa đã trích sẵn để truy xuất tri thức từ đồ thị; với **direct_answer**, hệ thống gọi trực tiếp mô hình ngôn ngữ mà không truy xuất (phù hợp cho chào hỏi, trò chuyện hoặc lời khuyên học tập chung); với **reject**, hệ thống trả về thông điệp từ chối đã soạn sẵn.

Bước 4: Cập nhật bộ nhớ dài hạn (có điều kiện). Khác với cập nhật theo chu kỳ cố định, hệ thống chỉ kích hoạt nút **update_long_term** khi khâu

phân tích đánh dấu cờ `should_update_long_term` - tức khi người dùng cung cấp một sự kiện ổn định đáng ghi nhớ. Khi đó, *Long-term Memory Agent* trích xuất và hợp nhất thông tin mới vào hồ sơ; ngược lại, đồ thị kết thúc ngay sau bước sinh để giảm độ trễ.

Quy trình điều phối tổng thể và luồng dữ liệu giữa các phân hệ được mô tả trực quan trong Hình 3.10.



Hình 3.10: Sơ đồ kiến trúc điều phối Agent tổng thể của trợ lý ảo

3.2.3 Cơ chế truy xuất tri thức (LightRAG Integration)

Khác với RAG truyền thống dựa thuần túy trên độ tương đồng vector, hệ thống sử dụng LightRAG để truy xuất trên cấu trúc đồ thị tri thức (Knowledge Graph). Điểm tối ưu quan trọng là các từ khóa đã được Analyze Agent trích xuất sẵn ở mức cao và mức thấp được truyền thẳng vào LightRAG:

$$Q_{RAG} = Q_{standalone} + K_{high} + K_{low}$$

Nhờ vậy, LightRAG bỏ qua được bước tự trích xuất từ khóa nội bộ (vốn cần thêm một lần gọi mô hình), giúp giảm độ trễ mà vẫn định hướng truy xuất chính xác. Tùy theo độ khái quát và mục đích tra cứu của câu hỏi, hệ thống hỗ trợ bốn chế độ truy vấn chính bao gồm:

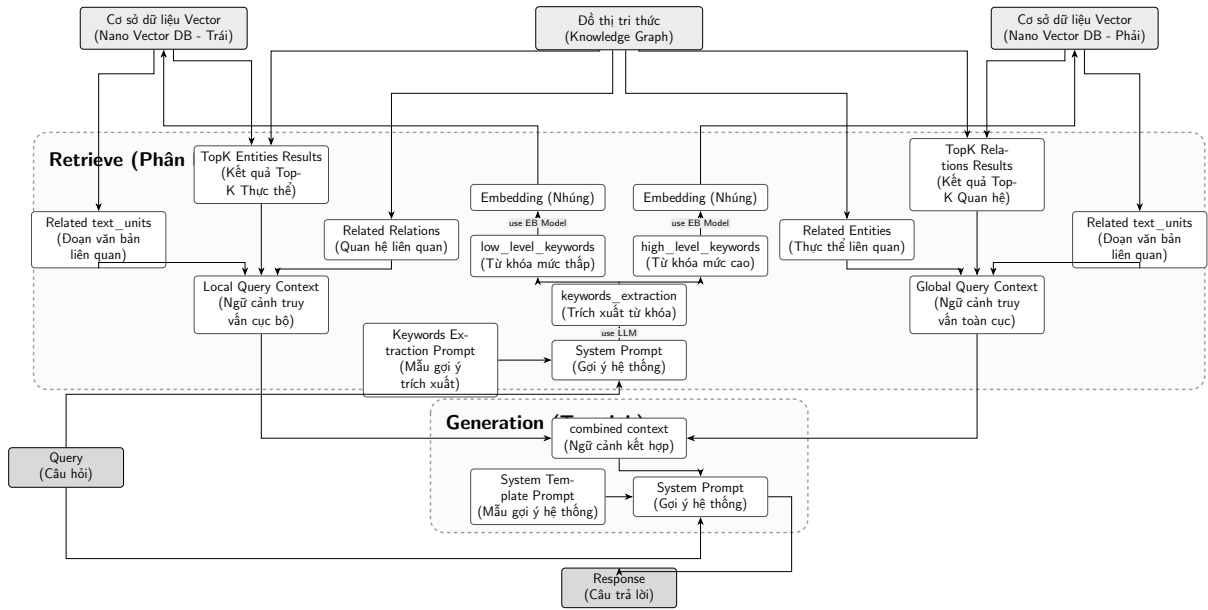
Chế độ truy vấn cơ bản (Naive Query): Đây là phương pháp tìm kiếm truyền thống dựa trên độ tương đồng vector. Hệ thống truy xuất các đoạn văn bản (text chunks) có bản nhúng vector gần nhất với câu hỏi mà không tận dụng cấu trúc đồ thị. Chế độ này phù hợp cho các câu hỏi đơn giản, yêu cầu trích xuất thông tin trực tiếp từ một đoạn văn cụ thể.

Chế độ truy vấn cục bộ (Local Query): Chế độ này được tối ưu hóa cho các câu hỏi chi tiết về một khái niệm cụ thể (Low-level Query). Hệ thống sử dụng cơ chế tìm kiếm vector để xác định các thực thể liên quan nhất, sau đó mở rộng truy xuất các nút lân cận và các mối quan hệ trực tiếp trong đồ thị tri thức. Quy trình này cung cấp một ngữ cảnh tri thức tập trung, giúp mô hình ngôn ngữ trả lời chính xác các thuộc tính và mối quan hệ vi mô của thực thể.

Chế độ truy vấn toàn cục (Global Query): Đối với các câu hỏi mang tính tổng quát cao (High-level Query), đòi hỏi sự hiểu biết về toàn bộ kho dữ liệu, chế độ toàn cục phát huy vai trò chủ đạo. Thay vì tìm kiếm đoạn văn rời rạc, hệ thống tận dụng cấu trúc đồ thị ở mức độ vĩ mô bằng cách tổng hợp các mô tả tóm tắt của thực thể và quan hệ quan trọng. Cách tiếp cận này giúp mô hình ngôn ngữ bao quát được các chủ đề lớn và đưa ra những nhận định mang tính hệ thống vượt qua giới hạn của sở ngữ cảnh thông thường.

Chế độ truy vấn hỗn hợp (Mix Query): Đây là sự kết hợp giữa tìm kiếm cục bộ và toàn cục. Chế độ Mix đồng thời truy xuất các chi tiết cụ thể từ đồ thị lân cận và các tóm tắt tri thức vĩ mô từ cấp độ toàn cục. Quy trình xử lý này đảm bảo câu trả lời vừa có độ chính xác của dữ liệu chi tiết, vừa có chiều sâu của lập luận hệ thống, đáp ứng tốt nhất cho các nhiệm vụ phân tích tri thức đa tầng.

Cơ chế truy xuất hai luồng cục bộ và toàn cục cùng quy trình lọc ngữ cảnh của LightRAG được thể hiện chi tiết ở Hình 3.11.



Hình 3.11: Cơ chế truy xuất tri thức và lọc ngữ cảnh hai tầng trong Ligh-tRAG

Việc viết lại câu hỏi thành dạng độc lập kết hợp với từ khóa giúp khắc phục hiện tượng mất ngữ cảnh khi câu hỏi của sinh viên quá ngắn hoặc mơ hồ. Toàn bộ trạng thái hội thoại được lưu bền vững qua cơ chế checkpoint trên MongoDB, cho phép khôi phục đúng bước đang xử lý khi có gián đoạn. Để gia tăng độ linh hoạt trong việc đáp ứng các nhu cầu tra cứu đa dạng, hệ thống cung cấp hai cấu hình truy vấn chuyên biệt bao gồm cấu hình truy vấn "flash" và cấu hình truy vấn "pro". Cấu hình "flash" ưu tiên tối ưu hóa thời gian phản hồi bằng cách giới hạn số lượng kết quả trả về từ cơ sở dữ liệu, trong khi cấu hình "pro" tập trung khai thác triệt để các mối quan hệ phức tạp nhằm mang lại ngữ cảnh toàn diện nhất. Song song với đó, hệ thống tích hợp thêm một luồng tra cứu đồ thị trực tiếp bằng ngôn ngữ truy vấn cấu trúc. Luồng xử lý này cho phép hệ thống đếm số lượng thực thể hoặc kiểm chứng các sự kiện cụ thể một cách độc lập mà không cần phải ánh xạ qua không gian vector nhúng, qua đó giảm thiểu đáng kể chi phí tính toán. Đối với các truy vấn đòi hỏi thông tin ngữ nghĩa sâu hơn thông qua các chế độ tìm kiếm hỗn hợp, sự chính xác của ngữ cảnh được đảm bảo bởi một cơ chế lọc kết quả nghiêm ngặt ngay sau khi quá trình truy xuất từ cơ sở dữ liệu hoàn tất. Dựa trên các thông số đặc tả như ngành học hay học kỳ đã được trích xuất từ bước phân tích ý định, hệ thống tự động đối chiếu và loại bỏ các mảnh thông tin nhiễu hoặc sai lệch

bối cảnh. Quá trình này đảm bảo kho liệu đầu vào cho bước sinh văn bản hoàn toàn tinh sạch. Việc chuẩn bị ngữ cảnh còn được cá nhân hóa sâu sắc thông qua việc lồng ghép trực tiếp hồ sơ bộ nhớ dài hạn của người dùng vào trong hướng dẫn tạo sinh, giúp câu trả lời gắn liền với định hướng của từng sinh viên. Nhằm đảm bảo tính chuẩn xác tuyệt đối trước khi phản hồi người dùng, kiến trúc điều phối tích hợp một tác tử kiểm chứng học thuật hoạt động như một màng lọc cuối cùng. Tác tử này được thiết kế chuyên biệt để thẩm định tính xác thực đối với các nhóm câu hỏi hệ trọng như lộ trình học tập, môn tiên quyết hay quy định tín chỉ. Quá trình kiểm chứng liên tục đối chiếu chéo câu trả lời dự kiến với các quy chuẩn học thuật cốt lõi nhằm ngăn chặn tình trạng ảo giác thông tin của mô hình tạo sinh. Cuối cùng, hệ thống áp dụng cơ chế xử lý bất đồng bộ toàn diện, cho phép truyền phát dữ liệu theo dạng chuỗi luồng liên tục ngay khi có kết quả, đồng thời thực thi các tác vụ cập nhật bộ nhớ dài hạn dưới nền sau khi kết thúc phản hồi, qua đó mang lại trải nghiệm tương tác liền mạch và tối ưu nhất.

3.2.4 Thuật toán điều phối (Pseudocode)

Algorithm 1 Quy trình điều phối Agent

Require: *query*, *user_id* ▷ Đầu vào từ sinh viên
Ensure: *answer* ▷ Câu trả lời cuối cùng

- 1: *routing* $\leftarrow$ AnalyzeAgent(*query*, Summarize(*history*))
- 2: *decision* $\leftarrow$ *routing.decision* ▷ direct_answer | retrieve | reject
- 3: **if** *decision* = “reject” **then**
- 4: **return** REJECT_MESSAGE
- 5: *memory* $\leftarrow$ LoadFromMongo(*user_id*) ▷ chỉ khi *routing.needs_lt*
- 6: *genReq* $\leftarrow$ BuildGenerationRequest(*routing*, *memory*)
- 7: **if** *decision* = “retrieve” **then**
- 8: *answer* $\leftarrow$ LightRAG(*genReq.query*, *genReq.mode*, K_{high} , K_{low})
- 9: **else**
- 10: *answer* $\leftarrow$ LLM(*genReq.system_prompt*, *query*)
- 11: **if** *routing.should_update_lt* **then**
- 12: UpdateMemory(*user_id*, *query*, *answer*) ▷ trích xuất + hợp nhất hồ sơ
- 13: **return** *answer*

3.3 Thiết lập thực nghiệm

3.3.1 Thiết lập thực nghiệm phân hệ trích xuất (MinerU)

3.3.1.1 Môi trường thực nghiệm

Hệ thống được triển khai thực nghiệm trên hạ tầng máy chủ đám mây nhằm đảm bảo khả năng tính toán mạnh mẽ cho các tác vụ xử lý ảnh và văn bản. Quá trình xử lý tài liệu đầu vào sử dụng công cụ MinerU (từ phiên bản 3.2.3) tích hợp các mô hình Vision-Language Model (VLM), OCR và Mathematical Formula Recognition (MFR).

3.3.1.2 Mô tả dữ liệu và quá trình thu thập

Để thực nghiệm và đánh giá chất lượng của bộ giải mã MinerU, nghiên cứu đã tiến hành thu thập tập dữ liệu Chương trình đào tạo chuẩn của sáu ngành và khối ngành thuộc Khoa, bao gồm: Cử nhân tài năng, Công nghệ thông tin, Hệ thống thông tin, Khoa học máy tính, Kỹ thuật phần mềm, và Trí tuệ nhân tạo. Nhằm đảm bảo tính đại diện, tài liệu được phân rã theo từng khóa tuyển sinh từ năm 2022 đến năm 2025.

3.3.1.3 Độ đo để đánh giá (Performance Metrics)

Phương pháp đánh giá được thực hiện thông qua việc đối sánh kết quả trích xuất tự động với nhãn chuẩn (Ground Truth). Để đánh giá mức độ chính xác cơ bản, nghiên cứu sử dụng CER (Character Error Rate) và WER (Word Error Rate) nhằm phản ánh tỷ lệ sai sót ở cấp độ ký tự và từ. Tương tự, Edit Distance Sim (Độ tương đồng khoảng cách sửa đổi) được áp dụng để đo lường sự sai khác chuỗi tổng thể. Nhằm định lượng mức độ trùng khớp văn bản sâu hơn, BLEU và METEOR được sử dụng để đánh giá độ bảo toàn n-gram. Đặc biệt, đối với cấu trúc dữ liệu phức tạp, TEDS-Struct và TEDS-Content (Tree-Edit-Distance-based Similarity) đóng vai trò đo lường chuyên biệt cho mức độ bảo toàn cấu trúc bảng biểu.

Biện luận về độ quan trọng của các chỉ số: Đối với kiến trúc truy xuất tăng cường đồ thị, các chỉ số BLEU, TEDS-Struct, TEDS-Content và Edit Distance Sim đóng vai trò then chốt nhất. Nếu mô hình bóc tách thất bại

trong việc tái tạo cấu trúc bảng (TEDS-Struct thấp), lượng tri thức sẽ bị suy biến. Chỉ số BLEU cao minh chứng văn bản bảo toàn chính xác các cụm từ gốc, giúp LLM không sai lệch ngữ cảnh. Edit Distance Sim đảm bảo tính nguyên vẹn của mã học phần, tạo tiền đề để thuật toán đối soát và hợp nhất thực thể hoạt động ổn định.

3.3.2 Thiết lập thực nghiệm Hệ thống Đồ thị tri thức và Đa tác nhân

3.3.2.1 Môi trường thực nghiệm

Đối với quá trình đánh giá chất lượng đồ thị tri thức bằng phương pháp LLM-as-a-Judge, nhóm nghiên cứu lựa chọn mô hình Qwen2.5-72B-Instruct [29] làm trọng tài đánh giá (Judge). Việc lựa chọn mô hình này dựa trên ba lý do chính. Trước tiên, Qwen2.5-72B-Instruct là dòng mô hình mã nguồn mở hàng đầu với 72 tỷ tham số, sở hữu năng lực tư duy logic và lập luận phức tạp vượt trội so với các mô hình nhỏ hơn, phù hợp với yêu cầu đánh giá ngữ nghĩa sâu của các quan hệ trong đồ thị. Tiếp theo, mô hình này có khả năng hiểu và phân tích tiếng Việt chuyên ngành xuất sắc - điều này đã được xác thực khoa học qua bộ đánh giá VMLU Benchmarks (2025), trong đó Qwen2.5-72B-Instruct đạt hiệu năng hàng đầu trong hầu hết các tác vụ ngôn ngữ tiếng Việt [3], giúp phán xét chính xác các khái niệm đặc thù của giáo dục đại học Việt Nam.

3.3.2.2 Mô tả dữ liệu và quá trình thu thập

Tiếp nối quá trình tiền xử lý văn bản bằng công cụ MinerU, tập dữ liệu đầu vào của hệ thống được xây dựng dựa trên các tài liệu thực tế về chương trình đào tạo, đề cương môn học và các quy chế học vụ của Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM. Tập dữ liệu bao gồm nhiều tài liệu chính, được chia thành ba nhóm chính. Nhóm thứ nhất là các mô tả chương trình đào tạo chuẩn của sáu ngành và khối ngành thuộc Khoa, gồm: Cử nhân tài năng, Công nghệ thông tin, Hệ thống thông tin, Khoa học máy tính, Kỹ thuật phần mềm và Trí tuệ nhân tạo, được phân rã theo từng khóa tuyển sinh từ năm 2022 đến năm

2025. Nhóm thứ hai tập trung vào đề cương chi tiết của hàng loạt học phần từ đại cương đến chuyên ngành, chứa đựng các thông tin cốt lõi như chuẩn đầu ra (ELO) và điều kiện tiên quyết. Nhóm thứ ba bao gồm các văn bản quy định và thông báo học vụ, điển hình là *Quy định chuẩn đầu ra Tiếng Anh từ Khóa 2022* và *Thông báo đăng ký học phần Học kỳ 2 Năm học 2025-2026*. Việc lựa chọn tập dữ liệu nội bộ và có tính đặc thù cao nhằm mục đích kiểm chứng khả năng của hệ thống LightRAG trong việc thấu hiểu và bóc tách các tri thức chuyên ngành phức tạp, vốn đòi hỏi sự liên kết chặt chẽ về mặt logic.

3.3.2.3 Đánh giá chất lượng nội tại của đồ thị tri thức (Graph Quality Metrics)

Nghiên cứu này chú trọng vào việc đo lường chất lượng nội tại của đồ thị (intrinsic graph quality) dựa trên mức độ trung thực so với văn bản gốc. Để thực hiện điều này, nghiên cứu áp dụng tổ hợp ba phương pháp đánh giá tiên tiến nhằm giải quyết ba vấn đề cốt lõi thường gặp khi trích xuất đồ thị tự động: tình trạng phân mảnh, hiện tượng ảo giác, và lỗi thiết kế lược đồ.

Đầu tiên là, nhằm giải quyết tình trạng phân mảnh thông tin - khi đồ thị tạo ra các thực thể rời rạc thay vì một mạng lưới liên kết, nghiên cứu áp dụng phương pháp đánh giá tính toàn vẹn cấu trúc (Graph Structure Health) theo đề xuất của Seo và cộng sự [30]. Cụ thể, phương pháp này đo lường số lượng các nút mồ côi (dangling nodes) và thống kê tỷ lệ bao phủ văn bản (chunk coverage). Nhờ đó, hệ thống có thể đảm bảo rằng hầu hết các thực thể trích xuất ra đều được kết nối chặt chẽ và không bỏ sót thông tin quan trọng từ tài liệu gốc. Kế tiếp để giảm thiểu hiện tượng ảo giác (hallucination) - tức tình trạng mô hình ngôn ngữ tự bịa đặt ra các mối quan hệ sai lệch (ví dụ: tự sáng tác ra môn học tiên quyết không tồn tại) - nghiên cứu sử dụng phương pháp đánh giá LLM-as-a-Judge do Edge và cộng sự (nhóm tác giả GraphRAG) [7] khởi xướng. Áp dụng phương pháp này, nghiên cứu lấy mẫu ngẫu nhiên các quan hệ sinh ra và yêu cầu mô hình Qwen2.5-72B đóng vai trò giám khảo độc lập. Mô hình sẽ đối chiếu trực tiếp từng mối quan hệ với đoạn văn bản gốc tương ứng để chấm điểm

tính có căn cứ (groundedness). Phương pháp này giúp hệ thống cam kết độ tin cậy tuyệt đối của tri thức được trích xuất. Sau cùng là ngăn chặn tình trạng trôi dạt lược đồ (schema drift) và đảm bảo cấu trúc đồ thị thực sự phục vụ được mục tiêu truy vấn, nghiên cứu áp dụng chuẩn kiểm định tính hợp lệ lược đồ (Ontology Validation). Theo hướng tiếp cận của hệ thống OOPS! do Poveda-Villalón và cộng sự [28] phát triển, kết hợp với các tiêu chuẩn thiết kế siêu dữ liệu của LightRAG [10], phương pháp này tự động rà soát các lỗi logic trong thiết kế đồ thị và kiểm tra khả năng trả lời các câu hỏi năng lực (Competency Questions). Việc này giúp phát hiện sớm các cấu trúc đồ thị lỗi từ trong trứng nước trước khi đưa vào truy vấn thực tế. Về mặt phương pháp luận, quy trình kiểm định tính hợp lệ dựa trên danh mục bẫy lỗi (pitfalls) của OOPS! vẫn là một chuẩn mực cốt lõi được áp dụng xuyên suốt trong các nghiên cứu hiện đại. Minh chứng là trong một nghiên cứu y tế công bố năm 2026 [9], phương pháp validation này đã được sử dụng thành công để kiểm thử tự động một hệ thống phân cấp tri thức y học phức tạp (MeSO). Bằng cách quét và phát hiện các lỗi cấu trúc như thiếu ràng buộc thuộc tính nghịch đảo hoặc sai lệch phân cấp, thực nghiệm năm 2026 đã chứng minh tính hiệu quả vượt thời gian của phương pháp này trong việc duy trì sự nhất quán logic và bảo vệ tính toàn vẹn của các mô hình đồ thị tri thức trước khi đưa vào vận hành.

3.3.2.4 Đánh giá chất lượng sinh văn bản và hiệu năng điều phối (RAG & Agentic Metrics)

Bên cạnh chất lượng đồ thị, chất lượng truy xuất và sinh văn bản cũng được đánh giá nghiêm ngặt thông qua khung đo lường RAGAS (Retrieval-Augmented Generation Assessment) [8]. Nghiên cứu áp dụng ba chỉ số cốt lõi của phương pháp này. Đầu tiên, độ liên quan của câu trả lời (Answer Relevance) được tính toán nhằm đảm bảo phản hồi đi thẳng vào trọng tâm truy vấn. Thứ hai, độ trung thực (Faithfulness) đo lường khả năng sinh văn bản bám sát ngữ cảnh từ đồ thị, đóng vai trò là lớp phòng thủ thứ hai giúp hạn chế tối đa hiện tượng ảo giác. Cuối cùng, độ gọi bối cảnh (Context Recall) đánh giá năng lực của hệ thống trong việc thu thập đủ các khái niệm phân tán trên đồ thị để tạo nên một câu trả lời hoàn chỉnh.

Về mặt hiệu năng kiến trúc, nghiên cứu tiến hành đo lường năng lực điều phối đa tác nhân (Multi-Agent). Cụ thể, tỷ lệ điều phối đúng (Routing Accuracy) được sử dụng để đánh giá độ chính xác của Tác tử phân tích (Orchestrator) khi quyết định phân luồng câu hỏi vào các chế độ xử lý phù hợp (Local, Global, Hybrid hay Cypher Lookup). Đồng thời, tốc độ phản hồi trung bình (Average Latency) được ghi nhận nhằm so sánh độ trễ giữa luồng xử lý đồ thị trực tiếp so với luồng RAG truyền thống, từ đó chứng minh hiệu quả tối ưu hóa thời gian thực của kiến trúc phân tuyến mới.

Chương 4

Cài đặt thử nghiệm và kết quả

4.1 Đánh giá chất lượng MinerU trên tập tài liệu đầu vào

Dựa trên các thiết lập thực nghiệm, nghiên cứu đã tiến hành trích xuất và đo lường trên tập dữ liệu. Khoa học máy tính khóa 2022 là một minh chứng xuất sắc khi đạt CER chỉ 0.0019. Với chỉ số Edit Distance Sim, hầu hết tài liệu thuộc Hệ thống thông tin đều đạt trên 0.99, cho thấy độ chính xác gần như tuyệt đối. CNTT khóa 2022 đạt BLEU lên tới 0.9928. Đối với cấu trúc dữ liệu phức tạp, kết quả thực nghiệm chỉ ra phần lớn các khóa đều đạt TEDS-Struct 1.0000 dù KTPM khóa 2024 có sự sụt giảm nhẹ.

Bảng 4.1: Kết quả chi tiết đánh giá MinerU trên các tập tài liệu Chương trình đào tạo (2022-2025)

Tập tài liệu	CER ↓	WER ↓	Edit Dist Sim ↑	BLEU ↑	METEOR ↑	TEDS-Struct ↑	TEDS-Content ↑
Công nghệ thông tin (CNTT)							
Khóa 2022	0.0008	0.0045	0.9992	0.9928	0.9963	1.0000	0.9999
Khóa 2023	0.1255	0.1571	0.8745	0.7743	0.8149	0.6055	0.5552
Khóa 2024	0.2166	0.1808	0.8151	0.8008	0.8475	0.7416	0.7051
Khóa 2025	0.0006	0.0035	0.9994	0.9926	0.9974	1.0000	0.9999
Cử nhân tài năng (CNTN)							
Khóa 2022	0.0023	0.0107	0.9977	0.9804	0.9912	0.9925	0.9924
Khóa 2023	0.1462	0.1247	0.8720	0.8714	0.8508	0.7298	0.6946
Khóa 2024	0.1643	0.1249	0.8539	0.8538	0.8696	0.9994	0.9632
Khóa 2025	0.0020	0.0100	0.9980	0.9805	0.9914	1.0000	0.9998
Hệ thống thông tin (HTTT)							
Khóa 2022	0.0009	0.0051	0.9991	0.9929	0.9958	1.0000	0.9999
Khóa 2023	0.0011	0.0056	0.9989	0.9892	0.9954	1.0000	0.9997
Khóa 2024	0.0070	0.0284	0.9930	0.9469	0.9736	1.0000	0.9995
Khóa 2025	0.0038	0.0202	0.9962	0.9617	0.9807	1.0000	0.9999
Khoa học máy tính (KHMT)							
Khóa 2022	0.0019	0.0102	0.9981	0.9845	0.9914	1.0000	0.9999
Khóa 2023	0.0018	0.0096	0.9982	0.9832	0.9920	1.0000	0.9997
Khóa 2024	0.0061	0.0188	0.9939	0.9676	0.9853	1.0000	0.9994
Khóa 2025	0.0014	0.0075	0.9986	0.9843	0.9935	0.9976	0.9936
Kỹ thuật phần mềm (KTPM)							
Khóa 2022	0.0012	0.0069	0.9988	0.9891	0.9943	1.0000	1.0000
Khóa 2023	0.0016	0.0084	0.9984	0.9843	0.9930	1.0000	0.9998
Khóa 2024	0.2633	0.2137	0.7760	0.8018	0.8069	0.3342	0.2291
Khóa 2025	0.0031	0.0167	0.9969	0.9678	0.9862	1.0000	0.9997
Trí tuệ nhân tạo (TNTT)							
Khóa 2022	0.0013	0.0074	0.9987	0.9876	0.9939	1.0000	1.0000
Khóa 2023	0.0014	0.0074	0.9986	0.9854	0.9939	1.0000	0.9998
Khóa 2024	0.2266	0.1660	0.8075	0.8117	0.8432	0.4277	0.3081
Khóa 2025	0.0109	0.0201	0.9891	0.9644	0.9849	0.4247	0.3373

4.2 Đánh giá chất lượng nội tại của Đồ thị tri thức (Knowledge Graph)

Dựa trên các tiêu chí và phương pháp đánh giá đã được trình bày ở Chương 3, hệ thống tiến hành đo lường chất lượng đồ thị tri thức trên mô hình Qwen2.5-72B-Instruct. Đặc biệt, đối với phương pháp đánh giá LLM-as-a-Judge, việc lấy mẫu đánh giá đồ thị được thực hiện ở hai quy mô $N = 50$ và $N = 400$. Cấp độ $N = 50$ đóng vai trò tập mẫu kiểm tra nhanh (sanity check), giúp hệ thống phản hồi nhanh chóng trong quá trình tinh chỉnh prompt và gỡ lỗi sơ bộ mà không tốn quá nhiều chi phí API. Cấp độ $N = 400$, trong khi đó, là tập mẫu chính thức được xác định dựa trên lý thuyết hội tụ tri thức và công thức tính cỡ mẫu thống kê, nhằm

đảm bảo bao phủ các mối quan hệ hiểm gặp trong đồ thị và đạt độ tin cậy thống kê 95% với biên độ sai số xấp xỉ 5%.

Bảng kết quả dưới đây tóm tắt các chỉ số hiệu năng thực nghiệm trên ba khía cạnh cốt lõi: tính toàn vẹn cấu trúc (Structure Health), độ chính xác trích xuất (Extraction Correctness) và tính hợp lệ lược đồ (Ontology Validation).

Bảng 4.2: Kết quả đánh giá nội tại của Đồ thị tri thức

Tiêu chí đánh giá	Tập mẫu nhỏ (N=50)	Tập mẫu chuẩn (N=400)
Khía cạnh 1: Tính toàn vẹn cấu trúc (Đánh giá trên toàn bộ đồ thị)		
Tổng số nút (Nodes) / Tổng số cạnh (Edges)	3.808 / 3.775	
Tỷ lệ nút mồ côi (Dangling Nodes)	0.8% (31 nút)	
Độ phủ văn bản (Chunk Coverage)	96.1% (570/593 chunks)	
Khía cạnh 2: Độ chính xác trích xuất (LLM-as-a-Judge)		
Độ có cơ sở (Groundedness Precision)	98.00%	94.08%
Tỷ lệ ảo giác (Hallucination - NO verdict)	2.0%	5.0%
Tỷ lệ chưa rõ ràng (UNKNOWN verdict)	0.0%	5.0%
Khía cạnh 3: Tính hợp lệ lược đồ (Ontology Validation)		
Tỷ lệ vượt qua câu hỏi năng lực (CQ Pass Rate)	71.4%	
Lỗi thiết kế nghiêm trọng (Critical/Major Pitfalls)	0	
Lỗi thiết kế nhỏ (Minor Pitfalls - Cảnh báo vòng lặp)	1	

4.2.1 Phân tích và Nhận xét kết quả

Kết quả thực nghiệm từ Bảng 4.2 cho thấy hệ thống đồ thị tri thức đạt được chất lượng rất khả quan trong bối cảnh dữ liệu học thuật.

Xét về mặt cấu trúc (Structure Health), đồ thị thể hiện tính liên kết chặt chẽ với tỷ lệ nút mồ côi (Dangling Nodes) cực thấp, chỉ 0.8% tương đương 31 nút, chứng minh rằng hầu hết các thực thể trích xuất ra đều hòa nhập vào ngữ cảnh chung, triệt tiêu hiệu quả các “ốc đảo tri thức” cô lập. Hơn nữa, độ phủ văn bản đạt 96.1%, khẳng định gần như toàn bộ thông tin trọng yếu từ các tài liệu chương trình đào tạo đã được hấp thụ thành công vào mạng lưới đồ thị mà không bị bỏ sót.

Xét về độ chính xác trích xuất (Extraction Correctness), quá trình đối sánh giữa $N = 50$ và $N = 400$ cho thấy sự hội tụ của dữ liệu. Ở tập mẫu nhỏ, độ chính xác (Groundedness) đạt 98.00%, nhưng khi mở rộng lên 400 mẫu, chỉ số này giảm nhẹ về 94.08%, kèm theo 5.0% tỷ lệ ảo giác và 5.0% không xác định. Dù vậy, theo tiêu chuẩn đánh giá LLM-as-a-Judge của Edge và cộng sự [7], mức Groundedness vượt ngưỡng 0.85 đã được coi là rất tốt, và tỷ lệ 94.08% trên tập mẫu lớn đủ đảm bảo độ tin cậy thống

kê cao, chứng minh mô hình giám khảo có khả năng nhận diện và trích xuất đúng các ràng buộc chuyên môn phức tạp như học phần tiên quyết và chuẩn đầu ra.

Cuối cùng, về tính hợp lệ lược đồ (Ontology Validation), lược đồ đạt tỷ lệ phản hồi 71.4% đối với các câu hỏi năng lực cơ bản. Công cụ rà soát tự động không phát hiện bất kỳ lỗi thiết kế nghiêm trọng nào, chỉ tồn tại duy nhất một cảnh báo nhỏ (Minor P24) liên quan đến khả năng xảy ra vòng lặp, vốn có thể kiểm soát dễ dàng ở tầng ứng dụng.

Tóm lại, việc ứng dụng các tiêu chuẩn Ontology và phương pháp đánh giá LLM-as-a-Judge bằng mô hình tham số lớn đã cung cấp một góc nhìn định lượng sâu sắc về chất lượng nội tại của đồ thị. Kết quả thực nghiệm khẳng định mạng lưới đồ thị xây dựng hoàn toàn đáp ứng được các yêu cầu khắt khe của hệ thống GraphRAG trong môi trường học thuật.

4.3 Đánh giá chất lượng truy xuất và sinh văn bản (RAG Metrics)

Để đánh giá chất lượng của phân hệ sinh câu trả lời và hiệu quả của tầng truy xuất bối cảnh, nhóm thực hiện đã thực thi tập kịch bản tự động trên bộ dữ liệu kiểm thử bao gồm 40 câu hỏi chuyên sâu xoay quanh quy chế học vụ và thông tin các môn học thuộc chương trình đào tạo. Quy trình đánh giá áp dụng phương pháp LLM-as-a-Judge với mô hình giám khảo Qwen2.5-14B hoạt động trên môi trường điện toán đám mây GPU của Vast.ai. Hệ thống được cấu hình với các kỹ thuật tối ưu hóa cốt lõi bao gồm mở rộng câu hỏi (Query Expansion) và truy xuất kết hợp (Hybrid Retrieval).

Khung đánh giá RAGAS (Retrieval-Augmented Generation Assessment) [8] được áp dụng để tính toán các chỉ số dựa trên mối quan hệ phân tích giữa bốn thành phần: câu hỏi đầu vào, ngữ cảnh truy xuất từ đồ thị tri thức, câu trả lời do hệ thống sinh ra và đáp án mẫu chuẩn (Ground Truth).

Về độ liên quan của câu trả lời (Answer Relevance), hệ thống đạt 79.12%. Chỉ số này đo lường mức độ đáp ứng đúng trọng tâm và bám

sát ý định ban đầu của người dùng [20], được tính bằng cách mô hình giám khảo sinh ngược lại các câu hỏi giả định từ câu trả lời thực tế rồi đo độ tương đồng cosine với câu hỏi gốc. Về độ chính xác của câu trả lời (Answer Correctness), hệ thống đạt 67.15%, phản ánh mức độ trùng khớp về dữ kiện và ngữ nghĩa giữa câu trả lời của hệ thống so với đáp án chuẩn thông qua sự kết hợp giữa so khớp thực tế (Factual Overlap) và đo lường tương đồng cấu trúc văn bản. Về độ trung thực (Faithfulness), hệ thống đạt 67.25%, đo lường tỷ lệ nội dung được chứng minh có căn cứ từ ngữ cảnh đồ thị tri thức nhằm hạn chế hiện tượng ảo giác [16]. Cuối cùng, về độ gọi ngữ cảnh (Context Recall), hệ thống đạt 74.15%, xác định tỷ lệ các sự thật cốt lõi trong đáp án chuẩn được bao phủ đầy đủ bởi ngữ cảnh trích xuất từ đồ thị [7], nhờ cơ chế duyệt tìm cấu trúc thực thể liên kết giúp định vị hiệu quả các khái niệm phân tán.

Các thông số định lượng đo lường từ quá trình thực nghiệm trên bộ mẫu 40 câu hỏi được tổng hợp cụ thể trong Bảng 4.3.

Bảng 4.3: Kết quả thực nghiệm chất lượng truy xuất và sinh văn bản theo khung RAGAS

Chỉ số đánh giá	Kết quả	Ý nghĩa chỉ số trong hệ thống
Độ liên quan (Answer Relevance)	79.12%	Mức độ đáp ứng và bám sát ý định của người dùng
Độ chính xác (Answer Correctness)	67.15%	Độ trùng khớp thông tin so với đáp án chuẩn
Độ trung thực (Faithfulness)	67.25%	Tỷ lệ nội dung được chứng minh dựa trên tài liệu
Độ gọi ngữ cảnh (Context Recall)	74.15%	Tỷ lệ trích xuất thành công bối cảnh từ đồ thị

4.4 Đánh giá hiệu năng điều phối đa tác nhân (Agentic Metrics)

Việc phân rã hệ thống thành cấu trúc đa tác nhân điều phối thông qua LangGraph giúp tăng tính linh hoạt xử lý, song cũng làm thay đổi các đặc

tính vận hành về mặt thời gian và tài nguyên tính toán. Về tỷ lệ điều phối đúng công cụ (Routing Accuracy), Tác nhân Orchestrator đạt độ chính xác cao trong việc phân loại ý định người dùng và điều hướng đến đúng chế độ truy xuất phù hợp (Local, Global hoặc Hybrid). Về tốc độ phản hồi (Average Latency), thời gian xử lý trung bình từ khi hệ thống tiếp nhận truy vấn cho đến khi hoàn thành việc xuất câu trả lời là khoảng 7.32 giây trên một câu hỏi. Độ trễ này phản ánh tổng chi phí thời gian của chuỗi tác vụ phức hợp bao gồm bước chuẩn hóa câu hỏi qua Query Expansion, quá trình duyệt tìm dữ liệu trên đồ thị LightRAG và năng lực lập luận sinh văn bản từ mô hình ngôn ngữ lớn.

4.5 Đánh giá tài nguyên triển khai và ước lượng khả năng chịu tải

4.5.1 Môi trường triển khai và hiện trạng sử dụng tài nguyên

Hệ thống được triển khai thực tế trên một máy chủ ảo (VPS) có cấu hình 2 vCPU và 3 GB RAM (dung lượng khả dụng thực tế khoảng 2.9 GB). Trên máy chủ này, hai tiến trình dịch vụ được vận hành song song trong cùng một môi trường Python: (1) máy chủ LightRAG đảm nhiệm việc lưu trữ Đồ thị tri thức, kho vector và thực thi truy vấn RAG; (2) dịch vụ Chat API đảm nhiệm luồng điều phối đa tác nhân LangGraph, xác thực người dùng và chuyển tiếp (proxy) truy vấn đến máy chủ LightRAG. Một điểm quan trọng trong kiến trúc triển khai là cả mô hình ngôn ngữ lớn (Qwen3-14B) lẫn mô hình nhúng (multilingual-e5-large) đều được gọi thông qua API bên ngoài, do đó máy chủ VPS không phải gánh chi phí suy luận mô hình mà chủ yếu đảm nhiệm việc tìm kiếm vector, duyệt đồ thị và điều phối các luồng xử lý bất đồng bộ.

Kết quả đo đạc trên môi trường vận hành cho thấy ở trạng thái chờ (đã nạp toàn bộ dữ liệu nhưng chưa có truy vấn), hệ thống chiếm khoảng 1.75 GB RAM, tức phần dư địa còn lại cho việc phục vụ truy vấn chỉ vào khoảng 1.1 GB. Phần lớn bộ nhớ này dùng để lưu trữ thường trực dữ liệu tri thức: Đồ thị tri thức gồm 3,822 thực thể và 10,493 quan hệ, cùng khoảng 15,200 vector nhúng 1,024 chiều. Qua thực nghiệm nạp lại toàn bộ

tập tin lưu trữ, riêng phần dữ liệu này chiếm xấp xỉ 450 MB bộ nhớ, phần còn lại thuộc về thư viện Python và bộ nhớ đệm của hai tiến trình. Bảng 4.4 tổng hợp hiện trạng sử dụng tài nguyên của hệ thống.

Bảng 4.4: Hiện trạng sử dụng tài nguyên của hệ thống trên máy chủ VPS

Hạng mục	Giá trị	Ghi chú
CPU	2 vCPU	Chủ yếu xử lý truy xuất và điều phối I/O
RAM khả dụng	2.9 GB	Tổng dung lượng của VPS
RAM chiếm dụng khi chờ	1.75 GB	Hai tiến trình dịch vụ và hệ điều hành
Dữ liệu tri thức nạp RAM	≈ 450 MB	Đồ thị, kho vector và các kho khóa-giá trị
Dư địa phục vụ truy vấn	≈ 1.1 GB	Dùng cho ngữ cảnh và bộ đệm mỗi truy vấn
Song song hóa lời gọi mô hình	4 luồng	<code>MAX_ASYNC</code> cho cả LLM và mô hình nhúng
Độ trễ trung bình mỗi truy vấn	7.32 giây	Đo tại Mục 4.4

4.5.2 Phương pháp và kết quả ước lượng khả năng chịu tải

Để ước lượng số lượng người dùng đồng thời mà hệ thống có thể phục vụ, nhóm áp dụng định luật Little [22] trong lý thuyết hàng đợi, với công thức $N = \lambda \times R$, trong đó N là số truy vấn đang được xử lý đồng thời, λ là tốc độ truy vấn đến hệ thống và R là thời gian phản hồi trung bình. Mô hình hành vi người dùng được thiết lập từ quan sát thực tế trong quá trình vận hành thử nghiệm: một sinh viên đang hoạt động (active user) gửi trung bình một câu hỏi mỗi 60–90 giây (bao gồm thời gian gõ câu hỏi và đọc câu trả lời), và mỗi truy vấn chiếm $R = 7.32$ giây xử lý theo số liệu đo tại Mục 4.4.

Việc ước lượng được xem xét trên ba giới hạn tài nguyên. Thứ nhất, về bộ nhớ, mỗi truy vấn đang xử lý cần khoảng 15–30 MB cho việc dựng ngữ

cảnh truy xuất (tối đa 6,000 token), bộ đệm luồng phản hồi và phiên kết nối; với dư địa 1.1 GB và biên an toàn 50%, hệ thống chịu được khoảng 20–30 truy vấn đồng thời trước khi có rủi ro tràn bộ nhớ. Thứ hai, về CPU, chi phí tính toán trên máy chủ cho mỗi truy vấn tương đối thấp (tìm kiếm vector trên 15,200 vector chỉ mất vài mili giây, tổng chi phí CPU khoảng 0.3–0.5 giây/truy vấn) nên với 2 vCPU, đây không phải là nút thắt cổ chai ở quy mô hiện tại. Thứ ba, về mức độ song song hóa các lời gọi mô hình, môi trường vận hành được cấu hình đồng bộ ở mức 4 cho cả lời gọi mô hình ngôn ngữ lớn (MAX_ASYNC=4) lẫn lời gọi mô hình nhúng: tại một thời điểm, máy chủ LightRAG xử lý song song tối đa 4 lượt sinh câu trả lời, các truy vấn đến thêm phải xếp hàng chờ tại khâu này. Do khâu sinh câu trả lời chiếm phần lớn thời gian (khoảng 5 giây trong tổng độ trễ 7.32 giây của mỗi truy vấn), đây chính là giới hạn quyết định thông lượng của toàn hệ thống.

Với 4 luồng sinh song song và mỗi lượt sinh kéo dài khoảng 5 giây, thông lượng tối đa theo tính toán của hệ thống đạt khoảng 40–50 truy vấn/phút. Tuy nhiên, trong điều kiện vận hành thực tế, năng lực này bị suy giảm bởi nhiều yếu tố không hiện diện trong mô hình lý tưởng: độ trễ mạng biến động khi gọi API mô hình từ xa, độ dài câu hỏi và câu trả lời không đồng đều giữa các người dùng, các truy vấn đến dồn theo cụm (burst) thay vì rải đều, cùng chi phí duy trì kết nối streaming. Do đó, nhóm áp dụng hệ số suy hao 30–40% lên con số tính toán để phản ánh năng lực phục vụ thực tế, thu được thông lượng bền vững khoảng 25–35 truy vấn/phút. Áp dụng định luật Little với chu kỳ gửi câu hỏi 60–90 giây của mỗi người dùng, hệ thống phục vụ ổn định khoảng **25–40 người dùng hoạt động đồng thời** với độ trễ giữ ở mức 7–12 giây. Khi số người dùng đồng thời vượt ngưỡng khoảng 50–70 người, các truy vấn bắt đầu xếp hàng tại khâu sinh câu trả lời và độ trễ tăng gấp 2–3 lần; hiệu ứng xếp hàng này chính là nguyên nhân của hiện tượng phản hồi chậm mà một bộ phận người dùng ghi nhận trong quá trình vận hành ở những thời điểm nhiều người cùng truy cập. Nếu quy đổi theo tỷ lệ truy cập đồng thời thông thường của ứng dụng web (5–10% tổng số người dùng hoạt động trong ngày), cấu hình hiện tại phục vụ được nhóm khoảng 250–400 sinh viên sử dụng mỗi ngày, phù hợp với quy mô triển khai hiện tại ở phạm vi một khoa. Đáng chú ý, do chi phí suy luận

mô hình đã được đưa ra API bên ngoài, hệ thống vẫn còn dư địa mở rộng trên chính phần cứng hiện tại: nếu nâng `MAX_ASYNC` lên mức 8–12, năng lực phục vụ (sau khi trừ suy hao) tăng lên khoảng 50–80 người dùng đồng thời, khi đó giới hạn bộ nhớ (20–30 truy vấn đồng thời trong 1.1 GB dư địa) cùng với hạn mức tần suất của nhà cung cấp API mới trở thành trần chịu tải thực tế của máy chủ. Bảng 4.5 tổng hợp năng lực phục vụ theo từng kịch bản.

Bảng 4.5: Năng lực phục vụ thực tế của hệ thống theo từng kịch bản cấu hình (đã áp dụng hệ số suy hao 30–40%)

Kịch bản	Người dùng đồng thời	Thông lượng	Độ trễ
Vận hành hiện tại (<code>MAX_ASYNC=4</code>)	25–40	25–35 truy vấn/phút	7–12 giây
Hiện tại, quá tải	50–70	25–35 truy vấn/phút	15–25 giây
Tinh chỉnh <code>MAX_ASYNC=8–12</code>	50–80	40–60 truy vấn/phút	8–15 giây
Giới hạn phần cứng (RAM)	$\approx$ 80–100	$\approx$ 60–70 truy vấn/phút	Tăng nhanh khi vượt ngưỡng

Các con số trên được xây dựng từ số liệu đo đạc trực tiếp trên môi trường vận hành đang phục vụ người dùng thực (độ trễ trung bình 7.32 giây, mức chiếm dụng RAM 1.75 GB, cấu hình song song mức 4) và đã trừ hệ số suy hao 30–40% cho các điều kiện vận hành thực tế, do đó phản ánh sát năng lực phục vụ của hệ thống trong giai đoạn triển khai hiện tại. Để bảo vệ năng lực này trước các hành vi sử dụng bất thường, hệ thống được trang bị cơ chế giới hạn tần suất truy cập (rate limiting) ở mức 30 yêu cầu/phút cho người dùng đã đăng nhập và 10 yêu cầu/phút cho khách, ngăn một người dùng đơn lẻ chiếm dụng toàn bộ năng lực xử lý.

4.5.3 Nhận xét và đề xuất tối ưu hóa

Từ kết quả ước lượng, nhóm rút ra một số nhận xét về khả năng mở rộng của hệ thống. Do toàn bộ suy luận mô hình đã được đưa ra API bên ngoài, nút thắt thông lượng hiện tại nằm ở mức song song hóa lời gọi mô hình (đang đặt đồng bộ ở mức 4 cho cả LLM và mô hình nhúng) chứ chưa chạm đến giới hạn phần cứng; vì vậy, việc nâng dần `MAX_ASYNC` kết hợp theo dõi mức tiêu thụ bộ nhớ là biện pháp cải thiện có chi phí thấp nhất và nên được thực hiện trước tiên khi lượng người dùng tăng. Về bộ nhớ, cần bổ sung phân vùng swap để phòng ngừa tình trạng tiến trình bị hệ điều hành chấm dứt đột ngột (OOM) khi tải tăng đột biến, đồng thời lưu ý rằng bộ nhớ đệm phản hồi LLM và lịch sử hội thoại sẽ tăng dần theo thời gian vận hành. Về dài hạn, việc thay thế các phương án lưu trữ dạng tệp JSON trong bộ nhớ (NanoVectorDB, NetworkX) bằng các hệ cơ sở dữ liệu chuyên dụng như PostgreSQL, Qdrant hoặc Redis sẽ vừa giảm áp lực RAM, vừa cho phép nhân bản dịch vụ theo chiều ngang (horizontal scaling) khi triển khai ở quy mô toàn trường. Cuối cùng, quá trình nạp và lập chỉ mục tài liệu mới – vốn tiêu tốn nhiều bộ nhớ và lời gọi LLM – nên được lên lịch vào các khung giờ thấp điểm để tránh cạnh tranh tài nguyên với luồng phục vụ người dùng.

4.6 Đánh giá hệ thống từ phía người dùng cuối

Quá trình thử nghiệm thực tế (beta testing) được tiến hành với sự tham gia của các sinh viên Khoa Công nghệ Thông tin. Hệ thống đã giải quyết trơn tru các câu hỏi chuẩn, cung cấp lộ trình học tập logic, mở ra cơ hội ứng dụng mạnh mẽ các mô hình ngôn ngữ lớn vào giáo dục [17].

4.6.1 Đánh giá trải nghiệm của người dùng

Để đánh giá tính thực tiễn, mức độ hiệu quả và chất lượng trải nghiệm người dùng đối với hệ thống trợ lý ảo hỗ trợ học vụ LearningPath, một khảo sát thực nghiệm đã được tiến hành. Khảo sát thu thập ý kiến phản hồi từ các sinh viên đang theo học tại Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM (HCMUS) sau khi trải nghiệm trực tiếp hệ thống trên cả hai

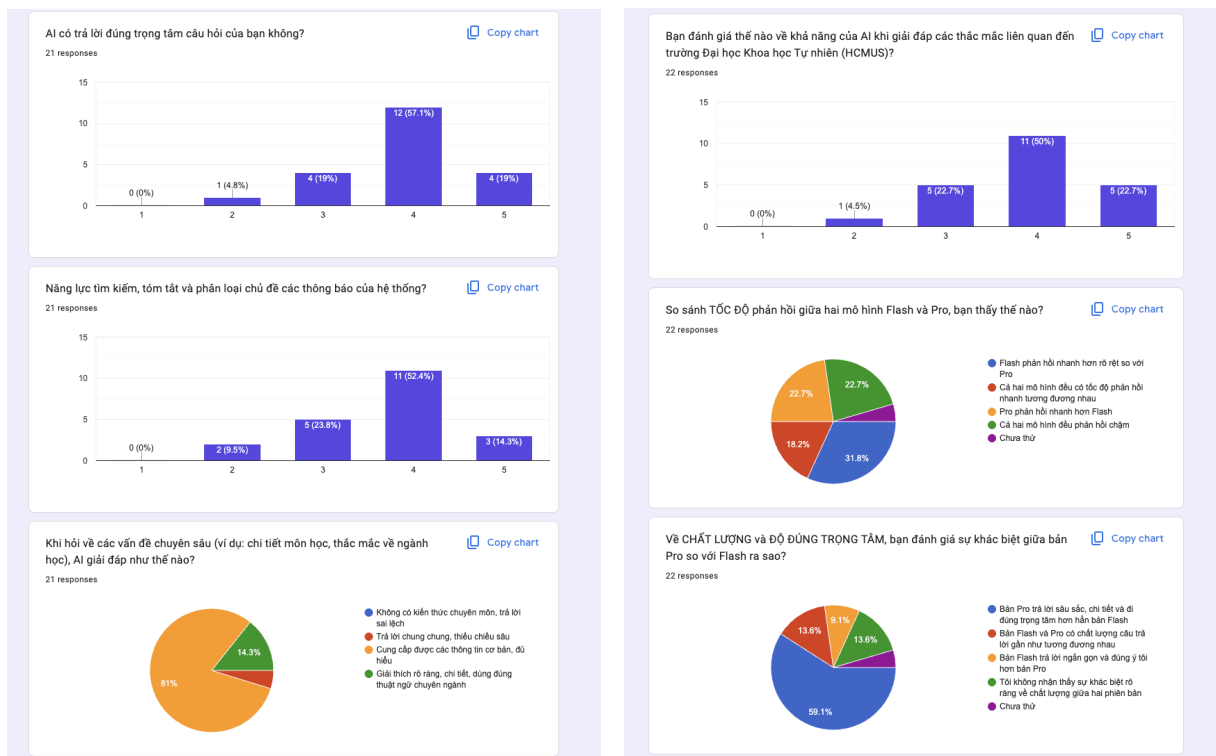
nền tảng Website và Mobile. Mẫu khảo sát ghi nhận ý kiến đánh giá từ các khía cạnh: chất lượng nội dung phản hồi, hiệu năng giữa hai chế độ Flash/Pro, trải nghiệm cá nhân hóa sau khi đăng nhập, thiết kế giao diện UI/UX và mức độ hài lòng tổng thể.

4.6.1.1 Thiết lập khảo sát và đối tượng đánh giá

Khảo sát được triển khai dưới hình thức biểu mẫu trực tuyến (Google Forms) kết hợp với phỏng vấn nhanh một số sinh viên tham gia thử nghiệm. Tổng số lượng mẫu phản hồi ghi nhận hợp lệ là khoảng 32 phản hồi. Đối tượng tham gia khảo sát là sinh viên thuộc các chuyên ngành khác nhau của Khoa Công nghệ thông tin (Hệ thống thông tin, Kỹ thuật phần mềm, Khoa học máy tính, Công nghệ thông tin, v.v.), là nhóm người dùng mục tiêu có nhu cầu tra cứu và định hướng học tập cao.

4.6.1.2 Đánh giá chất lượng phản hồi của trợ lý ảo AI

Khía cạnh chất lượng câu trả lời được đánh giá qua khả năng trả lời đúng trọng tâm, năng lực tìm kiếm và tổng hợp thông tin học vụ, và mức độ chính xác đối với các câu hỏi chuyên sâu. Kết quả khảo sát được thể hiện cụ thể tại Hình 4.1.



Hình 4.1: Kết quả khảo sát về chất lượng câu trả lời và so sánh hai chế độ phục vụ

Đối với câu hỏi "*AI có trả lời đúng trọng tâm câu hỏi của bạn không?*" (Hình 4.1a), kết quả ghi nhận sự đánh giá rất tích cực từ người dùng: có đến 57,1% người dùng đánh giá ở mức điểm 4 (tốt) và 19% ở mức điểm 5 (xuất sắc); chỉ có 4,8% đánh giá ở mức điểm 2. Điều này chứng minh rằng việc kết hợp phân tích từ khóa đa tầng từ Analyze Agent và cơ chế truy xuất LightRAG đã giúp mô hình ngôn ngữ lớn xác định đúng ngữ cảnh cần trả lời, hạn chế tối đa việc đi chệch hướng hoặc đưa ra các thông tin không liên quan.

Về "*Năng lực tìm kiếm, tóm tắt và phân loại chủ đề các thông báo của hệ thống*" (Hình 4.1a), có 52,4% người dùng cho điểm 4 và 14,3% cho điểm 5. Khả năng này hỗ trợ sinh viên nhanh chóng nắm bắt các thông tin quan trọng từ các thông báo dài của nhà trường mà không mất nhiều thời gian đọc toàn văn.

Đặc biệt, khi đối mặt với các câu hỏi chuyên sâu (như chi tiết học phần, thắc mắc lộ trình môn học của các chuyên ngành), trợ lý ảo chứng tỏ sự hữu ích vượt trội khi có đến 81% ý kiến đánh giá rằng hệ thống "*Cung cấp*

được các thông tin cơ bản, đủ hiểu" và 14,3% đánh giá ở mức cao nhất *"Giải thích rõ ràng, chi tiết, dùng đúng thuật ngữ chuyên ngành"*. Đáng chú ý là không ghi nhận bất kỳ phản hồi nào đánh giá hệ thống *"Không có kiến thức chuyên môn, trả lời sai lệch"*. Kết quả này củng cố tính thực tiễn của Đồ thị tri thức (Knowledge Graph) được xây dựng từ quy chế và chương trình đào tạo chính thống, đóng vai trò như một kho tri thức đáng tin cậy ngăn ngừa hiện tượng ảo giác (hallucination) thường gặp ở các LLM thông thường.

Bên cạnh đó, khả năng giải đáp các thắc mắc liên quan đặc thù đến trường HCMUS (Hình 4.1b) cũng đạt mức độ hài lòng cao với 50% điểm 4 và 22,7% điểm 5.

4.6.1.3 So sánh hiệu năng giữa chế độ Flash và Pro

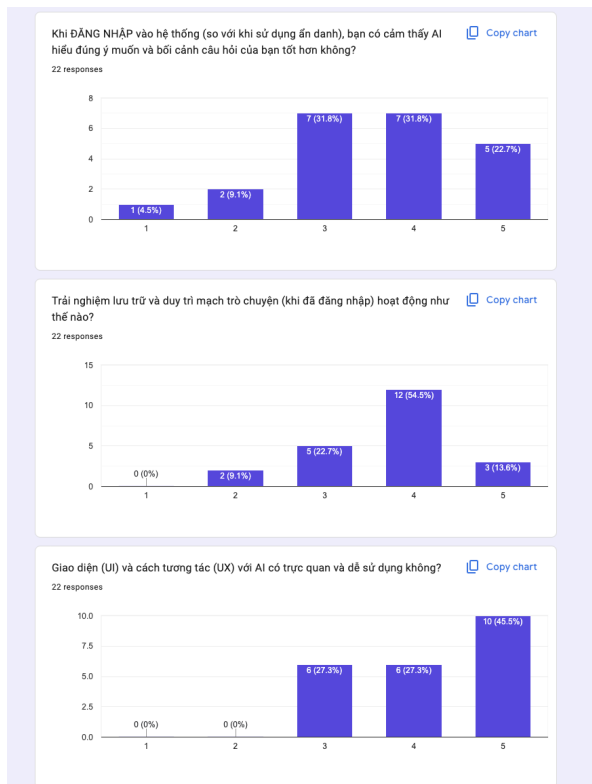
Hệ thống thiết lập hai chế độ hoạt động: Flash (sử dụng mô hình nhỏ gọn để phản hồi nhanh) và Pro (sử dụng mô hình lớn, suy luận sâu). Khảo sát thu thập ý kiến người dùng để làm rõ sự khác biệt thực tế giữa hai chế độ này.

Về tốc độ phản hồi, có 31,8% người dùng nhận thấy chế độ Flash phản hồi nhanh hơn rõ rệt so với Pro. Tuy nhiên, cũng có một bộ phận người dùng nhận định cả hai chế độ đều phản hồi chậm (22,7%), điều này phản ánh hạn chế về mặt phần cứng và độ trễ mạng khi gọi các API của mô hình ngôn ngữ lớn từ xa.

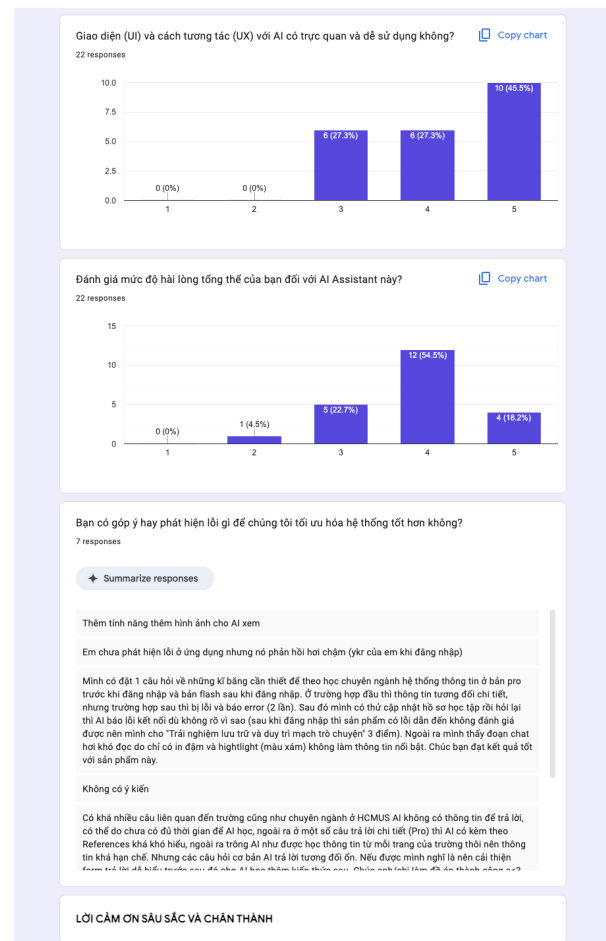
Về chất lượng và độ đúng trọng tâm, có đến 59,1% ý kiến đồng ý rằng bản Pro trả lời sâu sắc, chi tiết và đi đúng trọng tâm hơn hẳn bản Flash (Hình 4.1b). Trong khi đó, chỉ có 9,1% cho rằng bản Flash trả lời ngắn gọn và đúng ý hơn. Điều này chỉ ra rằng đối với các bài toán học vụ mang tính phức tạp cao (như tư vấn lộ trình học hay giải quyết các ràng buộc điều kiện tiên quyết), sinh viên sẵn sàng chấp nhận một khoảng thời gian chờ đợi lâu hơn để đổi lấy câu trả lời có chất lượng vượt trội từ chế độ Pro.

4.6.1.4 Đánh giá trải nghiệm cá nhân hóa và quản lý hội thoại

Trải nghiệm người dùng khi thực hiện đăng nhập và sử dụng tính năng lưu trữ lịch sử trò chuyện được minh họa tại Hình 4.2a.



a) Đánh giá trải nghiệm cá nhân hóa và quản lý lịch sử



b) Đánh giá giao diện UI/UX và độ hài lòng tổng thể

Hình 4.2: Kết quả khảo sát về trải nghiệm hệ thống và mức độ hài lòng tổng thể

Đối với câu hỏi so sánh chất lượng tư vấn trước và sau khi đăng nhập (tức là khi hệ thống bắt đầu kích hoạt Long-term Memory dựa trên hồ sơ học tập của sinh viên), phần lớn người dùng nhận thấy sự cải thiện rõ rệt: 31,8% đánh giá ở mức điểm 3, 31,8% ở mức điểm 4 và 22,7% ở mức điểm 5 về khả năng hiểu đúng ý muốn và bối cảnh cá nhân của hệ thống. Tính năng duy trì mạch trò chuyện và lưu trữ lịch sử cũng nhận được phản hồi tốt với 54,5% điểm 4 và 13,6% điểm 5.

4.6.1.5 Đánh giá giao diện và mức độ hài lòng chung

Về mặt thiết kế mỹ thuật và trải nghiệm tương tác (UI/UX) (Hình 4.2b), hệ thống được đánh giá cao với 45,5% điểm 5 (xuất sắc) và 27,3% điểm 4 (tốt). Giao diện tối giản, phân bố cục rõ ràng trên cả

hai nền tảng giúp sinh viên dễ dàng làm quen và thao tác ngay lần đầu sử dụng.

Tổng kết lại, mức độ hài lòng chung của người dùng đối với trợ lý ảo hỗ trợ học vụ đạt kết quả khả quan với 54,5% đánh giá mức 4 và 18,2% đánh giá mức 5; chỉ có 4,5% đánh giá ở mức thấp (điểm 2).

4.6.1.6 Tổng hợp phản hồi đóng góp và hướng cải thiện

Mặc dù hệ thống đáp ứng tốt các nhu cầu cơ bản, các phản hồi mở tự do từ người dùng (ghi nhận ở Hình 4.2b và Hình 4.3) đã chỉ ra một số điểm hạn chế cần khắc phục trong các giai đoạn phát triển tiếp theo:

Hình 4.3: Các phản hồi đóng góp ý kiến tự do từ người dùng khảo sát

Trước hết là việc cải thiện tốc độ phản hồi và độ ổn định của hệ thống. Một số người dùng phản hồi rằng hệ thống đôi lúc gặp hiện tượng phản hồi chậm hoặc báo lỗi kết nối (đầu ra hiển thị "[error]") sau một vài lượt trò chuyện liên tiếp. Ngoài ra, tính năng dừng tạo câu trả lời ("stop response") đôi khi gây treo hoặc crash ứng dụng.

Tiếp theo là yêu cầu mở rộng và làm phong phú kho tri thức học vụ. Người dùng nhận thấy một số câu hỏi quá chi tiết hoặc thuộc về các cập nhật quy chế rất mới chưa được AI trả lời đầy đủ, do dữ liệu nạp vào đồ thị tri thức tại thời điểm khảo sát vẫn còn giới hạn. Cần thiết lập cơ chế

cập nhật tự động bằng cách cào dữ liệu định kỳ để làm mới kho tri thức liên tục.

Hơn nữa, việc tối ưu hóa hình thức trình bày câu trả lời của trợ lý ảo cũng cần được lưu ý. Một số ý kiến đóng góp rằng định dạng tin nhắn của AI (chỉ sử dụng in đậm và khối highlight màu xám) đôi khi làm văn bản trở nên đơn điệu và khó theo dõi thông tin nổi bật. Vì vậy, hệ thống nên bổ sung thêm định dạng bảng biểu, liệt kê rõ ràng hơn và hỗ trợ hiển thị các sơ đồ trực quan.

Cuối cùng, một số lỗi kỹ thuật liên quan đến đăng nhập cần được khắc phục triệt để. Một số người dùng gặp khó khăn khi đăng nhập bằng tài khoản email hoặc tính năng đăng nhập nhanh qua Google. Đây là các lỗi ở tầng phân quyền cần được cấu hình lại và kiểm thử kỹ lưỡng hơn trong tương lai.

4.7 Thiết kế và xây dựng ứng dụng

4.7.1 Ý tưởng ứng dụng

Sinh viên Khoa Công nghệ Thông tin tại Trường Đại học Khoa học Tự nhiên (HCMUS) đang phải đối mặt với một khối lượng thông tin học vụ lớn và phân tán: từ chương trình đào tạo, đề cương chi tiết từng môn học, quy chế học vụ đến mạng lưới môn tiên quyết và song hành. Để tìm được câu trả lời cho một câu hỏi đơn giản như “Mình cần học gì trước khi học Trí tuệ Nhân tạo?”, sinh viên thường phải lần lượt tra cứu qua nhiều tệp PDF rời rạc hoặc chờ đợi giờ tư vấn của giảng viên.

Ý tưởng trung tâm của hệ thống là xây dựng một trợ lý học tập thông minh có khả năng hiểu sâu về toàn bộ kho tri thức học vụ của Khoa. Hệ thống không chỉ trả lời các câu hỏi đơn lẻ mà còn hiểu được mối liên hệ logic giữa các môn học, chuyên ngành và yêu cầu tốt nghiệp, từ đó đưa ra lời tư vấn cá nhân hóa và có căn cứ rõ ràng.

4.7.1.1 Các tình huống sử dụng tiêu biểu

Tình huống 1: Tra cứu lộ trình môn học. Sinh viên năm hai đang lập kế hoạch học tập đặt câu hỏi: “Mình muốn học môn Trí tuệ Nhân tạo thì

cần học những môn gì trước?”. Hệ thống AI Agent phân tích câu hỏi, truy xuất đồ thị tri thức để xác định chuỗi môn tiên quyết, sau đó trả lời chi tiết kèm mã môn học và lộ trình học kỳ gợi ý.

Tình huống 2: Kiểm tra điều kiện tốt nghiệp. Sinh viên năm bốn hỏi: “Mình còn thiếu những gì để đủ điều kiện xét tốt nghiệp?”. Hệ thống đối chiếu hồ sơ học tập cá nhân được lưu trong bộ nhớ dài hạn với quy định chuẩn đầu ra của Khoa, liệt kê các môn còn thiếu và đưa ra gợi ý đăng ký cho học kỳ tiếp theo.

Tình huống 3: Hỏi đáp về quy chế học vụ. Sinh viên thắc mắc: “Nếu mình trượt môn tiên quyết thì kỳ sau có được đăng ký môn tiếp theo không?”. Hệ thống tra cứu quy chế đào tạo tín chỉ của trường, trả lời chính xác với đoạn trích dẫn từ văn bản quy chế tương ứng.

4.7.1.2 Đối tượng người dùng mục tiêu

Hệ thống hướng tới hai nhóm người dùng chính: Sinh viên Khoa Công nghệ Thông tin tại Trường Đại Học Khoa Học Tự Nhiên, những người cần tra cứu thông tin học vụ và lập kế hoạch học tập.

4.7.2 Phân tích và đặc tả yêu cầu

4.7.2.1 Yêu cầu chức năng

Hệ thống được xác định cần đáp ứng các nhóm chức năng chính được liệt kê trong Bảng 4.6.

Bảng 4.6: Danh sách yêu cầu chức năng

STT	Nhóm chức năng	Mô tả
1	Xác thực người dùng	Đăng ký tài khoản, đăng nhập bằng email/mật khẩu, khôi phục mật khẩu qua email.
2	Chat với AI Agents	Đặt câu hỏi học vụ bằng ngôn ngữ tự nhiên, nhận câu trả lời có nguồn gốc từ đồ thị tri thức, xem lại lịch sử hội thoại theo phiên.

4.7.2.2 Yêu cầu phi chức năng

Bảng 4.7: Yêu cầu phi chức năng

STT	Yêu cầu	Ý nghĩa / Mô tả
1	Hiệu suất	Giao diện AI Chat phải đảm bảo tải trong vòng ≤ 2 giây để đảm bảo trải nghiệm người dùng.
2	Khả năng mở rộng	Hệ thống phải mở rộng được để xử lý lượng lớn người dùng và phiên hội thoại đồng thời.
3	Khả dụng	Hệ thống phải hoạt động liên tục với up-time $\geq 99.9\%$ /năm.
4	Đáp ứng	Các thao tác cơ bản như đăng nhập, AI Chat phản hồi nhanh (dưới 1 giây).
5	Khả năng sử dụng	UI/UX đơn giản, dễ hiểu, dễ đặt câu hỏi AI.

4.7.2.3 Danh sách Actor

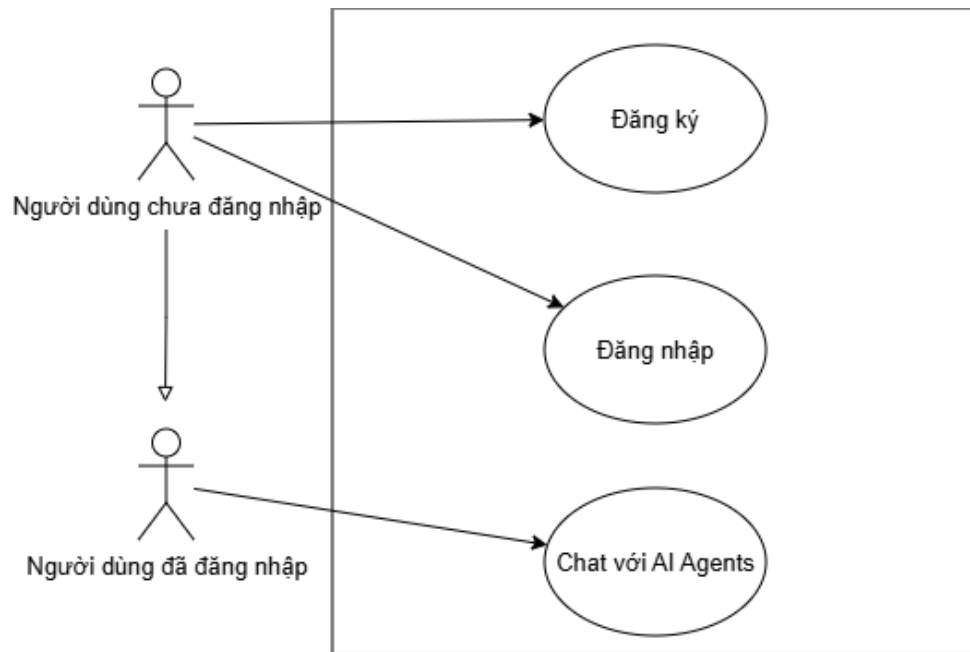
Hệ thống xác định một Actor chính tương tác trực tiếp với toàn bộ các chức năng:

Bảng 4.8: Danh sách Actor

STT	Actor	Ý nghĩa / Ghi chú
1	Người dùng	Sinh viên trực tiếp sử dụng hệ thống, thao tác các chức năng và gửi phản hồi.

4.7.3 Mô hình Use Case

Hệ thống có một phân hệ chính là Sinh viên, tương tác với hệ thống qua ba chức năng: đăng ký tài khoản, đăng nhập và chat với AI Agents. Sơ đồ Use Case tổng quan được thể hiện tại Hình 4.4.



Hình 4.4: Sơ đồ Use Case hệ thống

4.7.3.1 Danh sách Use Case

Bảng 4.9: Danh sách Use Case

STT	Tên Use Case	Mô tả	Actor
1	Đăng ký tài khoản	Sinh viên tạo tài khoản mới trên hệ thống.	Sinh viên
2	Đăng nhập	Sinh viên đăng nhập bằng email và mật khẩu.	
3	Chat với AI Agents	Sinh viên đặt câu hỏi học vụ và nhận tư vấn thông minh từ AI.	

4.7.3.2 Đặc tả Use Case

Use Case: Đăng ký tài khoản

Tên use-case	Đăng ký tài khoản
Tóm tắt	Cho phép sinh viên tạo tài khoản mới để sử dụng hệ thống.

Tác nhân	Sinh viên
Use-case liên quan	Đăng nhập
Điều kiện tiên quyết	Email chưa được đăng ký trong hệ thống.
Dòng sự kiện chính	- Sinh viên truy cập trang đăng ký. - Sinh viên nhập họ tên, email và mật khẩu. - Sinh viên xác nhận mật khẩu và đồng ý điều khoản. - Sinh viên nhấn nút “Đăng ký”. - Hệ thống kiểm tra hợp lệ và tạo tài khoản. - Hệ thống chuyển hướng sang trang đăng nhập.
Dòng sự kiện phụ	A1. Rẽ nhánh tại bước 2, email đã tồn tại: Hệ thống hiển thị thông báo lỗi và yêu cầu sinh viên sử dụng email khác.
Hậu điều kiện	Tài khoản được tạo thành công; sinh viên có thể đăng nhập vào hệ thống.

Use Case: Đăng nhập

Tên use-case	Đăng nhập hệ thống
Tóm tắt	Cho phép sinh viên xác thực danh tính và truy cập vào hệ thống.
Tác nhân	Sinh viên
Use-case liên quan	Đăng ký tài khoản, Chat với AI Agents
Điều kiện tiên quyết	Sinh viên đã có tài khoản được đăng ký trong hệ thống.

Dòng sự kiện chính	- Sinh viên truy cập trang đăng nhập. - Sinh viên nhập email và mật khẩu. - Sinh viên nhấn nút “Đăng nhập”. - Hệ thống xác thực thông tin đăng nhập. - Hệ thống cấp phát JWT token và chuyển hướng đến trang chính.
Dòng sự kiện phụ	A1. Rẽ nhánh tại bước 4, thông tin không hợp lệ: Hệ thống hiển thị thông báo lỗi và yêu cầu sinh viên nhập lại. A2. Quên mật khẩu: Sinh viên chọn “Quên mật khẩu”, hệ thống gửi email chứa liên kết khôi phục.
Hậu điều kiện	Sinh viên đăng nhập thành công và có thể sử dụng đầy đủ chức năng hệ thống.

Use Case: Chat với AI Agents

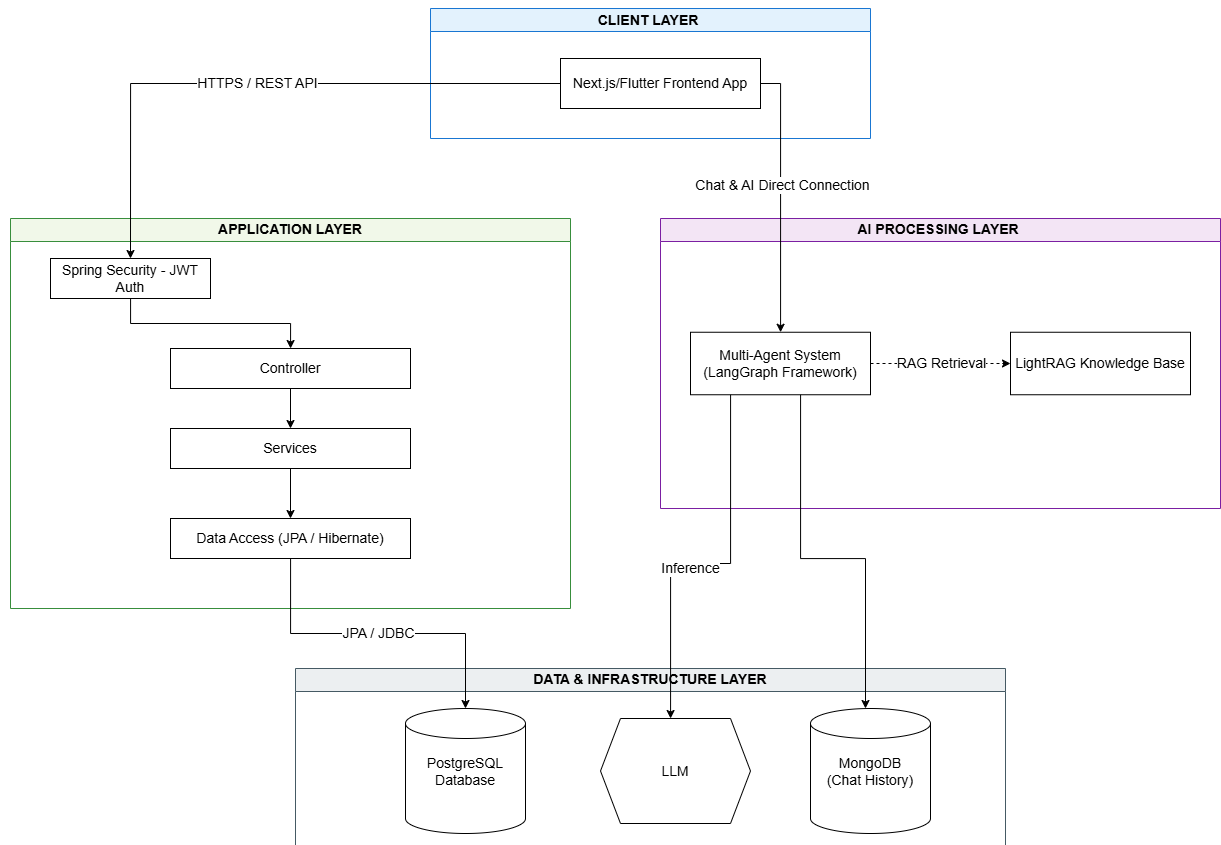
Tên use-case	Chat với AI Agents
Tóm tắt	Sinh viên đặt câu hỏi về học vụ và nhận câu trả lời thông minh từ AI Agent.
Tác nhân	Sinh viên
Use-case liên quan	Đăng nhập
Điều kiện tiên quyết	Sinh viên đã đăng nhập, dịch vụ AI Agent đang hoạt động.

Dòng sự kiện chính	- Sinh viên điều hướng đến phần AI Chat. - Sinh viên nhập câu hỏi liên quan đến học vụ. - Hệ thống chuyển tiếp câu hỏi đến hệ thống AI Agents. - AI Agent phân tích, truy xuất đồ thị tri thức và tạo phản hồi. - Hệ thống hiển thị câu trả lời trong giao diện chat. - Sinh viên có thể đặt câu hỏi tiếp theo, lặp lại bước 2-5.
Dòng sự kiện phụ	A1. Rẽ nhánh tại bước 5, dịch vụ AI không khả dụng: Hệ thống hiển thị thông báo lỗi và gợi ý sinh viên thử lại sau. A2. Câu hỏi ngoài phạm vi: AI phản hồi rằng câu hỏi nằm ngoài phạm vi hỗ trợ của hệ thống.
Hậu điều kiện	Sinh viên nhận được câu trả lời, cuộc hội thoại được lưu vào lịch sử phiên.

4.7.4 Kiến trúc hệ thống

Mô hình kiến trúc tổng thể của hệ thống được đề xuất dựa trên kiến trúc đa tầng (Multi-tier Architecture), nhằm đảm bảo tính tách biệt giữa giao diện người dùng, logic nghiệp vụ và các phân hệ xử lý trí tuệ nhân tạo. Sơ đồ chi tiết được thể hiện tại Hình 4.5.

Sơ lược về kiến trúc được nhóm sử dụng như sau: hệ thống được tổ chức theo mô hình đa tầng gồm bốn tầng chính, trong đó Tầng Giao diện tiếp nhận yêu cầu từ sinh viên và phân phối sang hai luồng xử lý độc lập, Tầng Nghiệp vụ xử lý logic xác thực và nghiệp vụ, Tầng Điều phối AI đảm nhận toàn bộ xử lý trí tuệ nhân tạo, và Tầng Dữ liệu cung cấp hạ tầng lưu trữ chuyên biệt cho từng loại dữ liệu.



Hình 4.5: Mô hình kiến trúc hệ thống tổng thể

4.7.4.1 Tầng Giao diện (Client Layer)

Tầng Giao diện được thiết kế tối ưu hóa trải nghiệm người dùng đa nền tảng, bao gồm hai thành phần Client hoạt động đồng bộ. Thành phần thứ nhất là Web Client dựa trên nền tảng Next.js, cung cấp giao diện tương tác hiện đại trên máy tính để bàn, tập trung vào việc quản lý lộ trình học tập trực quan và hiển thị dữ liệu phân tích học vụ quy mô rộng. Thành phần thứ hai là Mobile Client được xây dựng trên nền tảng Flutter và ngôn ngữ Dart, được phát triển độc lập cho thiết bị di động. Mã nguồn của thành phần này được tổ chức theo mô hình quản lý trạng thái tập trung thông qua bộ thư viện Provider (gồm MultiProvider và ChangeNotifierProvider), giúp tách biệt hoàn toàn tầng giao diện UI và tầng xử lý logic nghiệp vụ, từ đó duy trì luồng dữ liệu đồng nhất trên toàn bộ ứng dụng.

Khi sinh viên sử dụng ứng dụng, tầng này vận hành hai luồng giao tiếp song song tùy theo tính năng được yêu cầu.

Đối với các tác vụ liên quan đến tài khoản và thông tin cá nhân, ứng dụng giao tiếp với Tầng Nghiệp vụ thông qua giao thức HTTPS và REST

API. Đây là luồng truyền thống, đảm bảo tính nhất quán và bảo mật cho các thao tác đọc ghi dữ liệu có cấu trúc.

Đối với tính năng Chat tư vấn học vụ, ứng dụng thiết lập kết nối trực tiếp đến Tầng Điều phối AI (Chat & AI Direct Connection) mà không đi qua Tầng Nghiệp vụ trung gian. Thiết kế giúp giảm thiểu độ trễ và cho phép hiển thị câu trả lời theo từng phần, giúp mang lại trải nghiệm hội thoại tự nhiên và mượt mà hơn cho người dùng.

4.7.4.2 Tầng Nghiệp vụ (Application Layer)

Tầng Nghiệp vụ được xây dựng trên Spring Boot, đảm nhận nhiệm vụ xử lý toàn bộ logic nghiệp vụ và kiểm soát luồng dữ liệu giữa các tầng trong hệ thống.

Mọi yêu cầu từ phía Client trước tiên đều phải đi qua lớp bảo mật Spring Security kết hợp với cơ chế xác thực JWT (JSON Web Token). Giúp kiểm tra tính hợp lệ của token và phân quyền truy cập trước khi cho phép yêu cầu đi tiếp vào bên trong hệ thống.

Sau khi xác thực thành công, yêu cầu được định tuyến đến Controller rồi chuyển xuống lớp Services để thực thi logic nghiệp vụ. Lớp Data Access sử dụng Spring Data JPA và Hibernate để ánh xạ đối tượng và giao tiếp với cơ sở dữ liệu PostgreSQL thông qua giao thức JPA/JDBC.

4.7.4.3 Tầng Điều phối AI (AI Processing Layer)

Tầng Điều phối AI là trung tâm xử lý trí tuệ nhân tạo của hệ thống, tiếp nhận câu hỏi từ sinh viên qua luồng trực tiếp từ Client thông qua Chat & AI Direct Connection.

Khi nhận được câu hỏi, Multi-Agent System được xây dựng trên Lang-Graph Framework sẽ điều phối các tác nhân AI chuyên biệt để phân tích ý định câu hỏi, lập kế hoạch trả lời và tổng hợp kết quả. Trước khi sinh ra câu trả lời, hệ thống kích hoạt cơ chế RAG Retrieval để truy xuất các đoạn thông tin liên quan từ LightRAG Knowledge Base, kho tri thức học vụ đã được xây dựng và lập chỉ mục từ tài liệu của Khoa. Việc bổ sung ngữ cảnh từ tri thức thực tế giúp hạn chế tối đa hiện tượng “ảo giác” (hallucination) của mô hình ngôn ngữ, đảm bảo câu trả lời có căn cứ rõ ràng.

Sau khi có đầy đủ ngữ cảnh, hệ thống dùng LLM để tạo ra câu trả lời cuối cùng và trả về cho sinh viên.

4.7.4.4 Tầng Dữ liệu và Hạ tầng (Data & Infrastructure Layer)

Tầng này gồm ba thành phần chính, mỗi thành phần phục vụ một mục đích riêng biệt.

PostgreSQL được dùng để lưu thông tin tài khoản của sinh viên, bao gồm thông tin đăng nhập, phân quyền và hồ sơ cá nhân. Nhóm chọn PostgreSQL vì hỗ trợ tốt các ràng buộc toàn vẹn dữ liệu, phù hợp với yêu cầu bảo mật cao cho dữ liệu người dùng. Tầng Nghiệp vụ kết nối với PostgreSQL thông qua giao thức JPA/JDBC.

MongoDB lưu lịch sử hội thoại giữa sinh viên và AI Agent theo từng phiên. Do dữ liệu hội thoại không có cấu trúc cố định và cần truy xuất nhanh, MongoDB với cấu trúc tài liệu linh hoạt là lựa chọn phù hợp hơn so với cơ sở dữ liệu quan hệ.

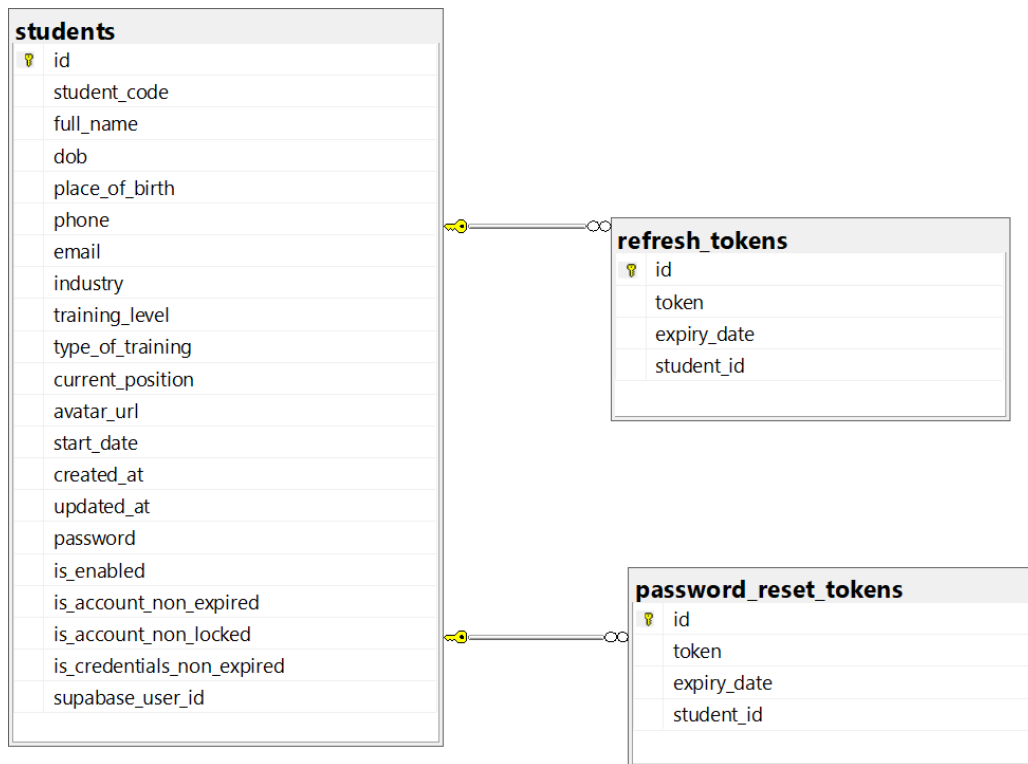
LLM (Large Language Model) là thành phần thực hiện suy luận và sinh ra câu trả lời cuối cùng. Khi Tầng Điều phối AI đã thu thập đủ ngữ cảnh từ Knowledge Graph, yêu cầu Inference được gửi đến LLM để tổng hợp và trả lời cho sinh viên.

4.7.5 Thiết kế chi tiết các thành phần

Hệ thống sử dụng ba loại cơ sở dữ liệu với vai trò khác nhau: PostgreSQL lưu thông tin tài khoản người dùng, MongoDB lưu lịch sử hội thoại với AI Agent, và Knowledge Graph (LightRAG) lưu toàn bộ tri thức học vụ phục vụ cho tính năng tư vấn.

4.7.5.1 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu Quan hệ (PostgreSQL) được dùng để lưu thông tin tài khoản và hồ sơ người dùng trong hệ thống. Cấu trúc quan hệ giúp đảm bảo tính toàn vẹn dữ liệu và hỗ trợ phân quyền chặt chẽ. Sơ đồ tổng quan của cơ sở dữ liệu được thể hiện tại Hình 4.6.



Hình 4.6: Mô hình sơ đồ cơ sở dữ liệu quan hệ người dùng

Bảng 4.13: Danh sách các bảng trong cơ sở dữ liệu quan hệ

STT	Tên bảng	Ý nghĩa
1	students	Lưu toàn bộ thông tin hồ sơ người dùng: mã sinh viên, họ tên, ngày sinh, email, mật khẩu, ngành học, trình độ đào tạo và các cờ trạng thái tài khoản (is_enabled, is_account_non_expired, is_account_non_locked,...).
2	refresh_tokens	Lưu token phiên làm việc (token, expiry_date) để duy trì trạng thái đăng nhập mà không cần xác thực lại.
3	password_reset_tokens	Lưu token bảo mật (token, expiry_date) dùng trong quy trình đặt lại mật khẩu.

Ràng buộc Khóa ngoại

Bảng 4.14: Mô tả khóa ngoại của các bảng trong CSDL quan hệ

STT	Tên Bảng	Tên cột	Bảng tham chiếu	Cột tham chiếu
1	refresh_tokens	student_id	students	id
2	password_reset_tokens	student_id	students	id

Cơ sở dữ liệu Phi cấu trúc (MongoDB) để phục vụ cho tác nhân duy trì bộ nhớ dài hạn (Long-term Memory Agent) và quản lý trạng thái luồng suy luận trong kiến trúc LangGraph, hệ thống sử dụng MongoDB. Dữ liệu được tổ chức linh hoạt để lưu trữ lịch sử hội thoại và các điểm kiểm soát (checkpoints) của AI Agent.

Bảng 4.15: Danh sách các Collection trong cơ sở dữ liệu hcmus_langgraph

STT	Tên Collection	Ý nghĩa dữ liệu thực tế
1	session_info	Lưu trữ thông tin định danh của phiên chat bao gồm: session_id, user_id, tiêu đề cuộc hội thoại và thời gian cập nhật gần nhất.
2	long_term_memory	Lưu trữ các tri thức hoặc thông tin quan trọng được đúc kết từ các cuộc hội thoại cũ để AI Agent sử dụng cho các lần tương tác sau.
3	checkpoints	Lưu trữ trạng thái hiện tại (State) của luồng LangGraph, cho phép hệ thống phục hồi lại đúng bước đang xử lý khi có lỗi hoặc ngắt quãng.

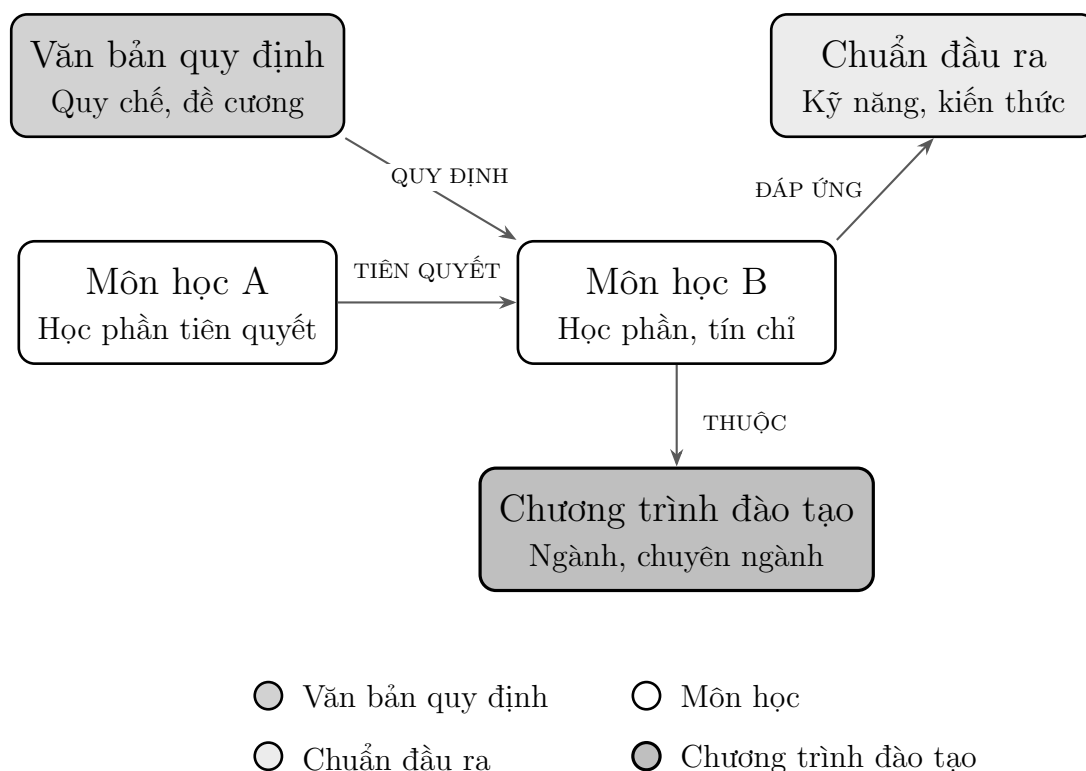
STT	Tên Collection	Ý nghĩa dữ liệu thực tế
4	checkpoint_writes	Ghi lại nhật ký các thay đổi của trạng thái Agent qua từng bước trong luồng điều phối.

4.7.5.2 Cơ sở dữ liệu Đồ thị (Knowledge Graph)

Hệ thống sử dụng LightRAG để xây dựng đồ thị tri thức học vụ, phục vụ lưu trữ thông tin phức tạp với nhiều mối quan hệ đan xen. Thay vì cấu trúc bảng (Table), dữ liệu được biểu diễn dưới dạng Thực thể (Nodes) và Mối quan hệ (Edges/Relationships), lưu trữ trong Neo4j.

Cấu trúc đồ thị tri thức được phân tích theo hai lớp: lớp khái niệm (logical view) thể hiện ngữ nghĩa các loại thực thể và quan hệ, và lớp lưu trữ vật lý (physical view) phản ánh cách triển khai thực tế trong Neo4j.

4.7.5.2.1 Lớp khái niệm (logical view) Hình 4.7 thể hiện mô hình khái niệm các loại thực thể và quan hệ chính trong đồ thị tri thức học vụ.



Hình 4.7: Mô hình khái niệm các loại thực thể và quan hệ chính trong đồ thị tri thức học vụ

Về mặt khái niệm, đồ thị gồm bốn nhóm thực thể chính. Môn học là thực thể trung tâm, lưu các học phần cùng thông tin tín chỉ, các môn có thể liên kết tiên quyết với nhau qua quan hệ TIÊN QUYẾT. Chương trình đào tạo nhóm các môn học theo ngành và chuyên ngành qua quan hệ THUỘC. Chuẩn đầu ra mô tả các kỹ năng, kiến thức cần đạt qua quan hệ ĐÁP ỨNG. Văn bản quy định gồm quy chế và đề cương là nguồn gốc tri thức của hệ thống, liên kết qua quan hệ QUY ĐỊNH.

4.7.5.2.2 Lớp lưu trữ vật lý (physical view) Về mặt triển khai trong Neo4j, toàn bộ các quan hệ đều được lưu dưới một relationship type duy nhất là DIRECTED, ngữ nghĩa cụ thể được mã hóa trong thuộc tính keywords do mô hình ngôn ngữ lớn sinh ra trong quá trình trích xuất.

Mỗi thực thể được lưu dưới dạng một node với hai nhãn: nhãn `workspace` (phân vùng dữ liệu, ví dụ base) và nhãn `entity_type` (phân loại ngữ nghĩa, ví dụ Course, Concept). Bảng 4.16 liệt kê các thuộc tính của node.

Bảng 4.16: Thuộc tính của node thực thể trong Knowledge Graph (Neo4j)

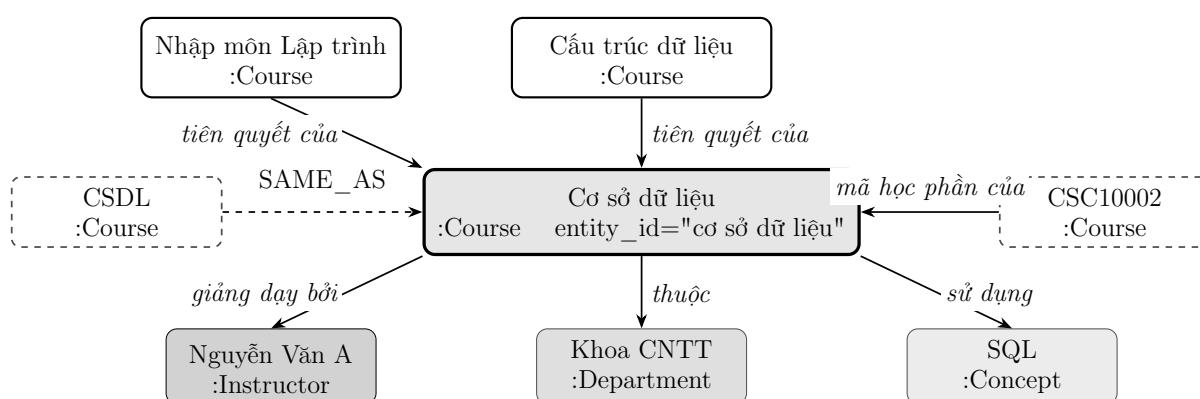
Thuộc tính	Kiểu	Mô tả & Ví dụ
entity_id	string	Tên thực thể (chuẩn hóa chữ thường). VD: "cơ sở dữ liệu"
entity_type	string	Loại thực thể do LLM phân loại. VD: "Course"
description	string	Mô tả tổng hợp từ nhiều tài liệu, các đoạn nối bằng <SEP>. VD: "Môn học về quản lý dữ liệu<SEP>Nghiên cứu SQL"
source_id	string	Danh sách ID chunk nguồn, nối bằng <SEP>. VD: "chunk-abc123<SEP>chunk-def456"
file_path	string	Tài liệu gốc chứa thực thể, nhiều giá trị nối bằng <SEP>. VD: "chuong_trinh.pdf<SEP>decuong.pdf"
created_at	integer	Unix timestamp thời điểm tạo. VD: 1719571200
truncate	string	Trạng thái giới hạn source_id: "" hoặc "KEEP Old"

Trường hợp đặc biệt là quan hệ đồng nhất thực thể, được biểu diễn bằng cùng type DIRECTED nhưng với keywords = "SAME_AS". Bảng 4.17 mô tả các thuộc tính của relationship.

Bảng 4.17: Thuộc tính của relationship trong Knowledge Graph (Neo4j)

Thuộc tính	Kiểu	Mô tả & Ví dụ
keywords	string	Từ khóa mô tả bản chất quan hệ, do LLM sinh ra. VD: "tiên quyết, học trước" hoặc "SAME_AS"
description	string	Diễn giải quan hệ theo ngữ cảnh tài liệu, nhiều mô tả nối bằng <SEP>. VD: "Nhập môn Lập trình là môn tiên quyết của Cơ sở dữ liệu"
weight	float	Trọng số quan hệ, tỉ lệ với số lần quan hệ xuất hiện. VD: 3.0
source_id	string	ID các chunk nguồn hỗ trợ quan hệ này. VD: "chunk-abc123"
file_path	string	Tài liệu gốc của quan hệ. VD: "chuong_trinh.pdf"
created_at	integer	Unix timestamp thời điểm tạo. VD: 1719571200

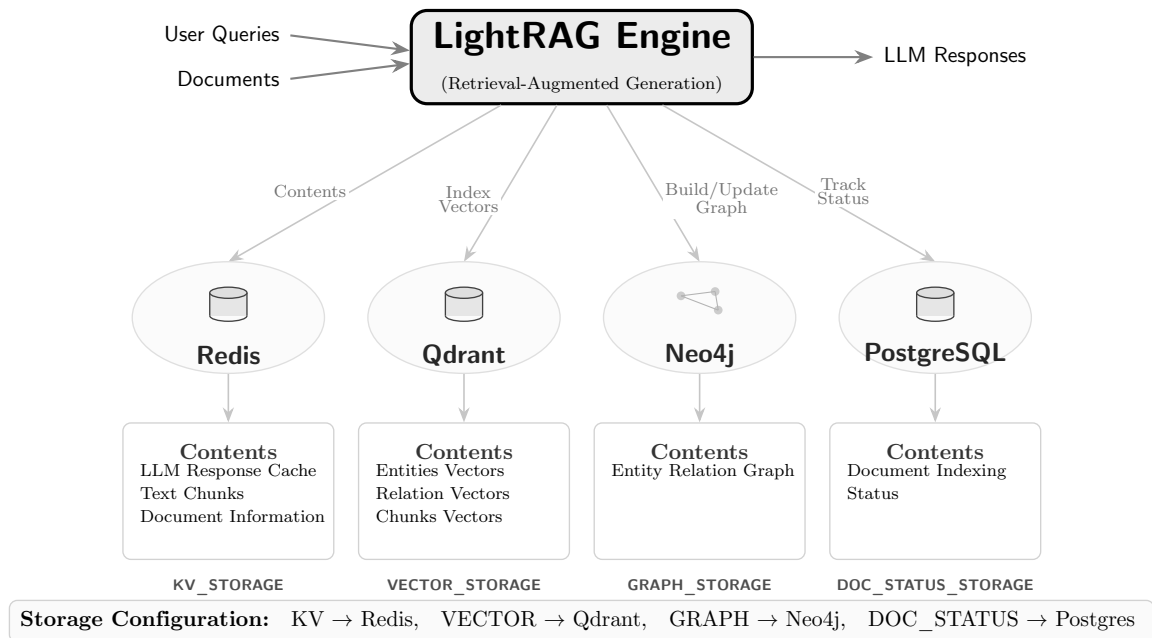
Hình 4.8 minh họa một đoạn đồ thị tri thức được trích xuất từ đề cương môn học, trong đó Cơ sở dữ liệu là thực thể canonical, CSDL và CSC10002 là alias liên kết qua quan hệ SAME_AS, các mũi tên thể hiện quan hệ ngữ nghĩa với nhãn keywords tương ứng.



Hình 4.8: Ví dụ subgraph trích xuất từ đề cương môn học trong Knowledge Graph

4.7.5.3 Kiến trúc triển khai lưu trữ cho LightRAG

Để vận hành hiệu quả các quy trình lập chỉ mục và truy xuất tri thức, hệ thống LightRAG được triển khai với kiến trúc lưu trữ hỗn hợp gồm bốn thành phần chuyên biệt. Sơ đồ tổng quan được thể hiện tại Hình 4.9.



Hình 4.9: Tổng quan kiến trúc lưu trữ và triển khai LightRAG

Kiến trúc lưu trữ của LightRAG được thiết kế theo hướng module hóa (modular design), bao gồm 4 thành phần chuyên biệt nhằm tối ưu hóa việc quản lý và truy xuất dữ liệu. Dựa trên yêu cầu về hiệu năng trong môi trường triển khai thực tế, nghiên cứu này lựa chọn tổ hợp công nghệ lưu trữ như sau (chi tiết tại Bảng 4.18 và Hình 4.9):

KV_STORAGE (Lưu trữ Key-Value): Sử dụng Redis để quản lý bộ nhớ đệm phản hồi của LLM (LLM response cache), các đoạn văn bản (text chunks) và siêu dữ liệu tài liệu. Redis được chọn nhờ tốc độ đọc/ghi in-memory siêu tốc, đặc biệt phù hợp cho truy xuất tần suất cao. Cấu hình tự động sao lưu dữ liệu (persistence rules) cũng được thiết lập để tránh mất mát khi dịch vụ khởi động lại.

VECTOR_STORAGE (Lưu trữ Vector): Sử dụng Qdrant để lưu trữ vector nhúng của thực thể, quan hệ và các đoạn văn bản. Qdrant là cơ sở dữ liệu vector chuyên dụng, cung cấp khả năng tìm kiếm ngữ nghĩa cực nhanh và độ chính xác cao đối với dữ liệu quy mô lớn.

GRAPH_STORAGE (Lưu trữ Đồ thị): Sử dụng Neo4j để lưu trữ toàn bộ cấu trúc đồ thị tri thức (Entity Relation Graph). Theo khuyến nghị từ tài liệu chính thức của LightRAG, Neo4j mang lại hiệu năng truy vấn đồ thị vượt trội nhất trong môi trường sản phẩm (production).

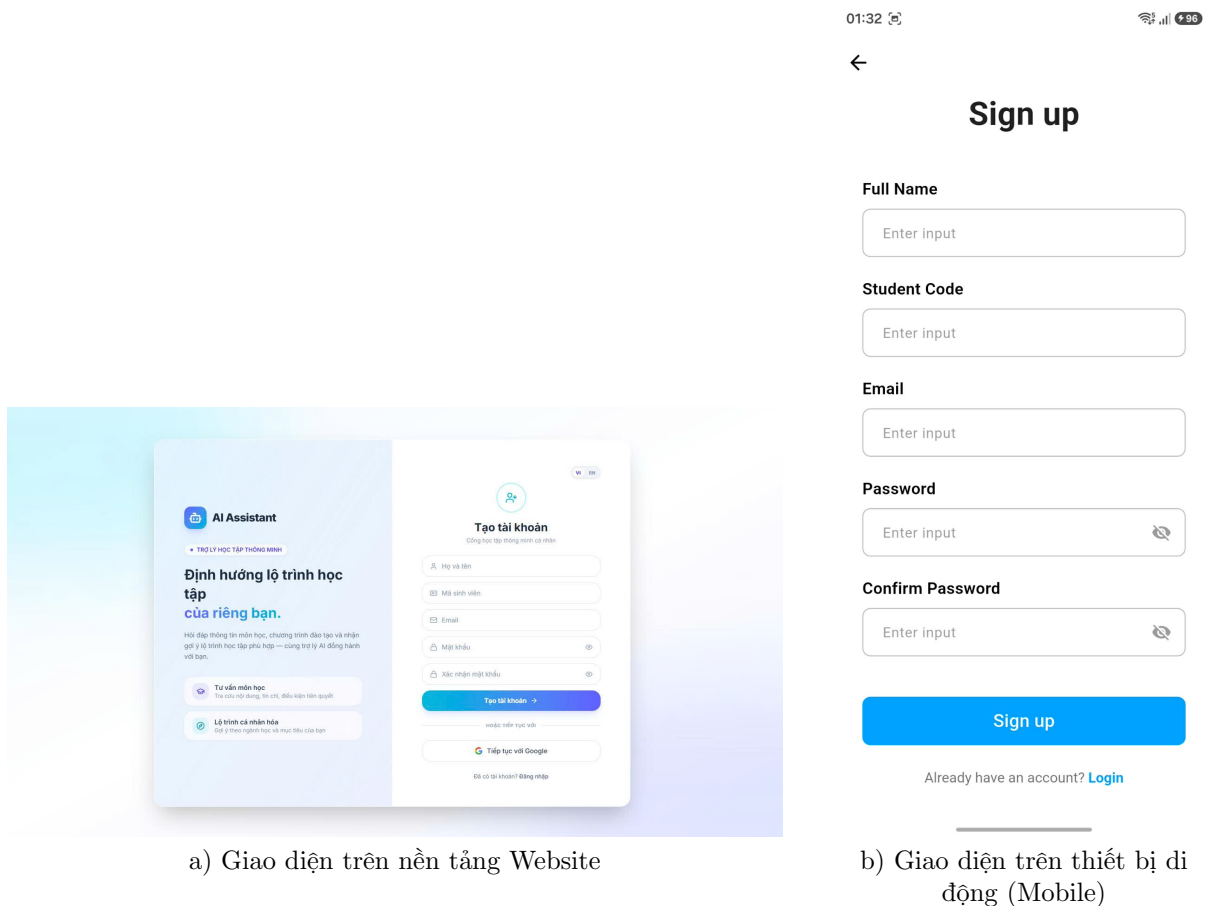
DOC_STATUS_STORAGE (Trạng thái tài liệu): Sử dụng PostgreSQL để theo dõi trạng thái lập chỉ mục của từng tài liệu, giúp ngăn chặn xử lý trùng lặp. PostgreSQL được chọn để đảm bảo tính nhất quán (ACID) tuyệt đối và an toàn cho các giao dịch trạng thái.

Bảng 4.18: Các thành phần lưu trữ trong kiến trúc triển khai LightRAG

Thành phần	Công nghệ	Lớp triển khai	Dữ liệu lưu trữ
KV_STORAGE	Redis	RedisKVStorage	LLM Response Cache, Text Chunks, Document Information
VECTOR_STORAGE	Qdrant	QdrantVectorDBStorage	Entities Vectors, Relation Vectors, Chunks Vectors
GRAPH_STORAGE	Neo4j	Neo4JStorage	Entity Relation Graph
DOC_STATUS_STORAGE	PostgreSQL	PGDocStatusStorage	Document Indexing Status

4.7.6 Thiết kế giao diện

4.7.6.1 Màn hình Đăng ký tài khoản



Hình 4.10: Giao diện Màn hình Đăng ký tài khoản trên hai nền tảng

Mục đích: Cung cấp biểu mẫu cho phép sinh viên khởi tạo tài khoản mới để đăng nhập và sử dụng hệ thống LearningPath trên cả thiết bị máy tính (Website) và thiết bị di động (Mobile).

Thành phần giao diện:

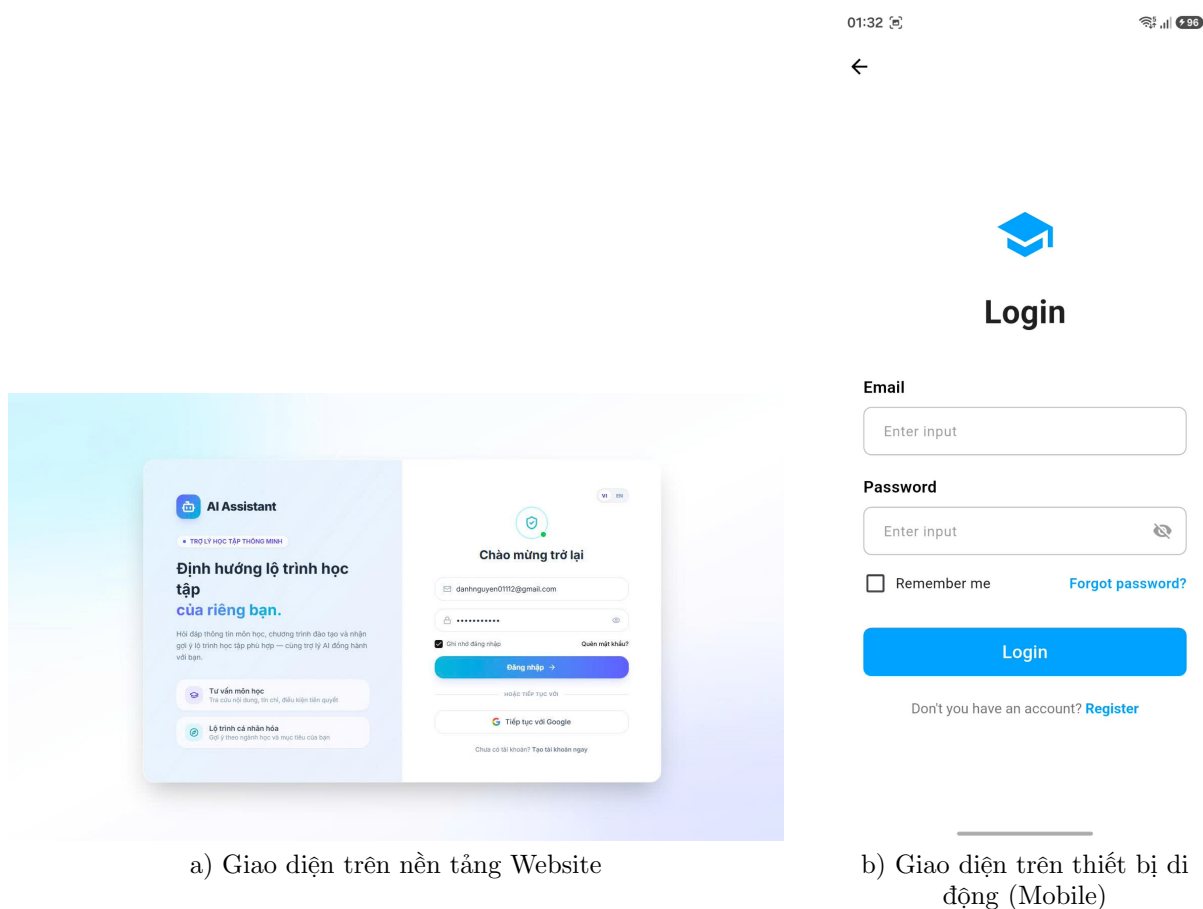
1. Các trường nhập thông tin: Họ và tên (Full Name), Mã sinh viên (Student Code), địa chỉ Email học thuật, Mật khẩu (Password) và Xác nhận mật khẩu (Confirm Password). Khung nhập mật khẩu trên ứng dụng di động được tích hợp thêm nút ẩn/hiện mật khẩu để thuận tiện cho thao tác cảm ứng.
2. Nút hành động: "Tạo tài khoản" (trên Website) hoặc "Sign up" (trên di động) để thực hiện gửi thông tin đăng ký lên hệ thống.

3. Điều hướng nhanh: Liên kết chuyển hướng sang giao diện Đăng nhập dành cho người dùng đã sở hữu tài khoản, cùng với nút quay lại (mũi tên ở góc trái) trên thiết bị di động.

Cách sử dụng:

1. Sinh viên điền đầy đủ và chính xác các thông tin được yêu cầu vào biểu mẫu.
2. Nhấn chọn nút "Tạo tài khoản" hoặc "Sign up".
3. Hệ thống kiểm tra các ràng buộc dữ liệu (mật khẩu xác nhận trùng khớp, đúng định dạng email sinh viên, không bỏ trống thông tin). Nếu hợp lệ, hệ thống tạo tài khoản thành công trên cơ sở dữ liệu và tự động điều hướng sinh viên sang giao diện Đăng nhập.

4.7.6.2 Màn hình Đăng nhập hệ thống



Hình 4.11: Giao diện Màn hình Đăng nhập hệ thống trên hai nền tảng

Mục đích: Xác thực danh tính của sinh viên, đảm bảo bảo mật và cấp quyền truy cập vào các tính năng cá nhân hóa như lưu trữ lịch sử trò chuyện và đồng bộ hồ sơ học tập.

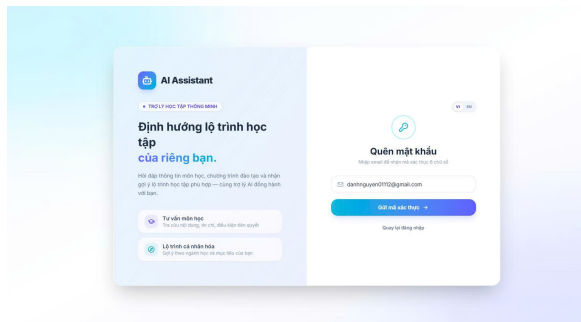
Thành phần giao diện:

1. Các trường nhập thông tin: Địa chỉ Email sinh viên và Mật khẩu (Password).
2. Tiện ích hỗ trợ: Hộp kiểm "Remember me" (Ghi nhớ phiên đăng nhập) giúp sinh viên không phải nhập lại mật khẩu trong những lần truy cập sau, và liên kết "Forgot password?" (Quên mật khẩu).
3. Nút hành động: "Login" hoặc "Đăng nhập" để gửi thông tin xác thực.
4. Điều hướng nhanh: Liên kết chuyển hướng nhanh đến biểu mẫu Đăng ký nếu chưa có tài khoản.

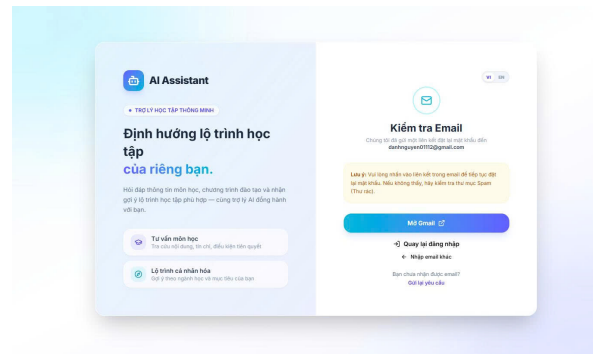
Cách sử dụng:

1. Sinh viên điền Email và Mật khẩu đăng nhập.
2. Tích chọn ô "Remember me" (nếu muốn hệ thống lưu phiên đăng nhập qua Token).
3. Nhấn nút "Login" hoặc "Đăng nhập" để hệ thống đối chiếu cơ sở dữ liệu. Nếu thông tin chính xác, sinh viên được chuyển hướng trực tiếp vào giao diện làm việc chính.
4. Trong trường hợp gặp sự cố về mật khẩu, sinh viên nhấn vào liên kết "Forgot password?" để bắt đầu quy trình khôi phục.

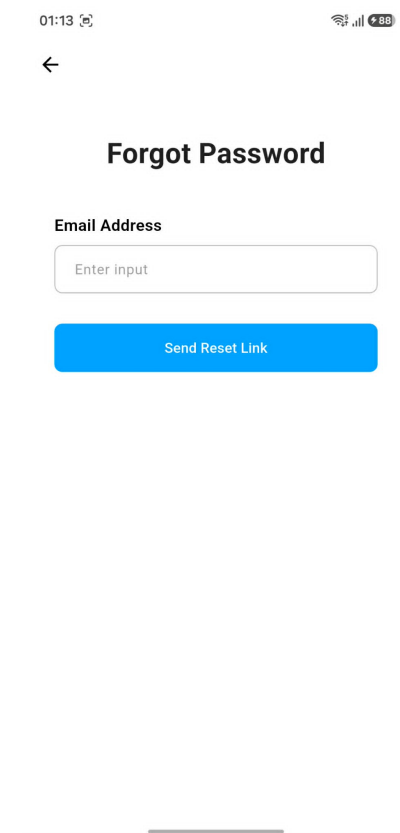
4.7.6.3 Màn hình Quên mật khẩu



a) Yêu cầu mã xác thực (Website)



b) Thông báo kiểm tra Email (Website)



c) Biểu mẫu trên di động (Mobile)

Hình 4.12: Giao diện Màn hình Quên mật khẩu trên hai nền tảng

Mục đích: Hỗ trợ sinh viên cấp lại mật khẩu mới thông qua quy trình xác minh hộp thư điện tử đã đăng ký trên hệ thống.

Thành phần giao diện:

1. Trường nhập Email để nhận liên kết khôi phục mật khẩu.
2. Nút hành động: "Gửi mã xác thực" (hoặc nút gửi trên giao diện di

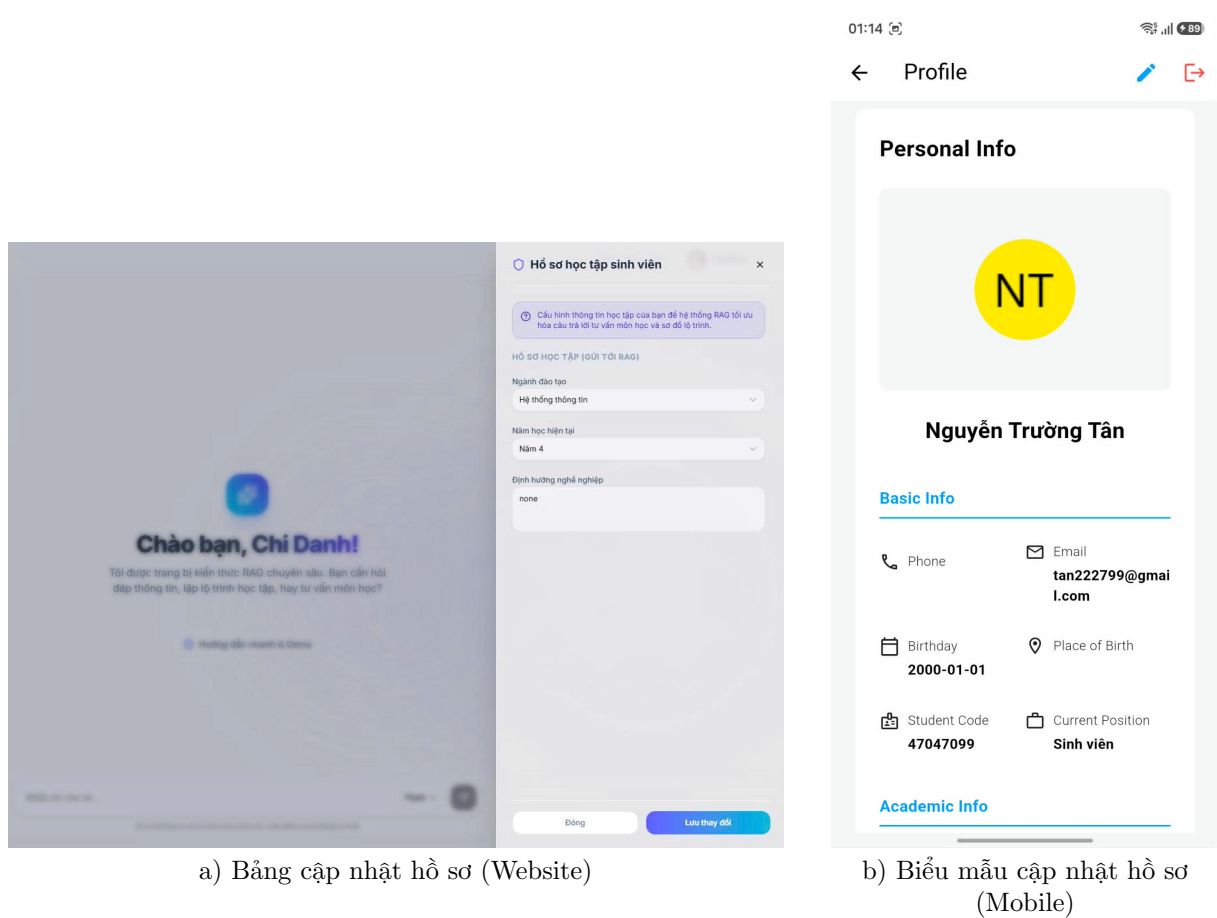
động).

3. Giao diện bước 2: Hộp thoại thông báo hướng dẫn người dùng kiểm tra hộp thư điện tử cùng nút "Mở Gmail" để truy cập nhanh.
4. Các nút điều hướng nhanh: "Quay lại đăng nhập", "Nhập email khác" hoặc "Gửi lại yêu cầu" trong trường hợp chưa nhận được email.

Cách sử dụng:

1. **Bước 1 - Gửi yêu cầu:** Sinh viên nhập Email đã đăng ký tài khoản vào ô dữ liệu, sau đó nhấn nút gửi. Hệ thống ghi nhận, tự động sinh mã Token khôi phục dùng một lần gửi tới email sinh viên, và chuyển giao diện sang Bước 2.
2. **Bước 2 - Xác thực email:** Sinh viên nhấn nút "Mở Gmail" (hoặc tự truy cập hộp thư cá nhân). Kiểm tra kỹ các thư mục (bao gồm cả thư mục Spam/Thư rác). Nhấn vào liên kết xác thực được đính kèm trong email để được chuyển hướng đến biểu mẫu thiết lập mật khẩu mới.

4.7.6.4 Màn hình Hồ sơ học tập cá nhân



Hình 4.13: Giao diện Màn hình Hồ sơ học tập cá nhân trên hai nền tảng

Mục đích: Cho phép sinh viên khai báo và cập nhật thông tin học tập hiện tại cùng định hướng cá nhân. Các dữ liệu này được lưu trữ vào bộ nhớ dài hạn (Long-term Memory) của AI Agent để phục vụ cho việc cá nhân hóa nội dung tư vấn môn học và lộ trình học tập.

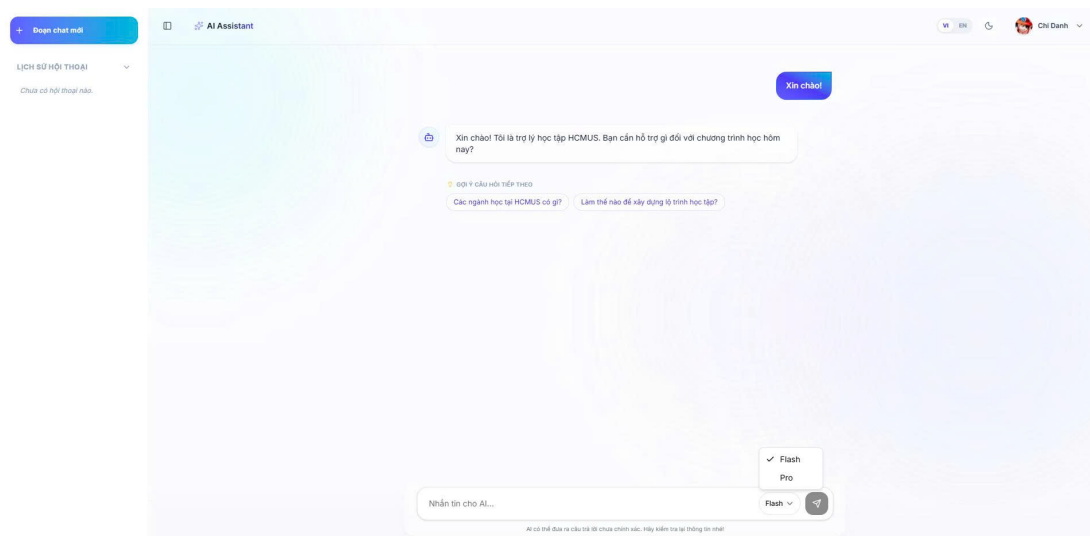
Thành phần giao diện:

1. Danh sách chọn (Dropdown): "Ngành đào tạo" (chọn ngành học hiện tại) và "Năm học hiện tại" (chọn năm học thứ nhất đến thứ tư).
2. Ô nhập văn bản tự do: "Định hướng nghề nghiệp" để sinh viên nhập các mục tiêu hoặc định hướng phát triển bản thân.
3. Các nút chức năng: "Lưu thay đổi" (hoặc "Save details" trên di động) và nút "Đóng" (hoặc biểu tượng "X" để hủy bỏ thao tác).

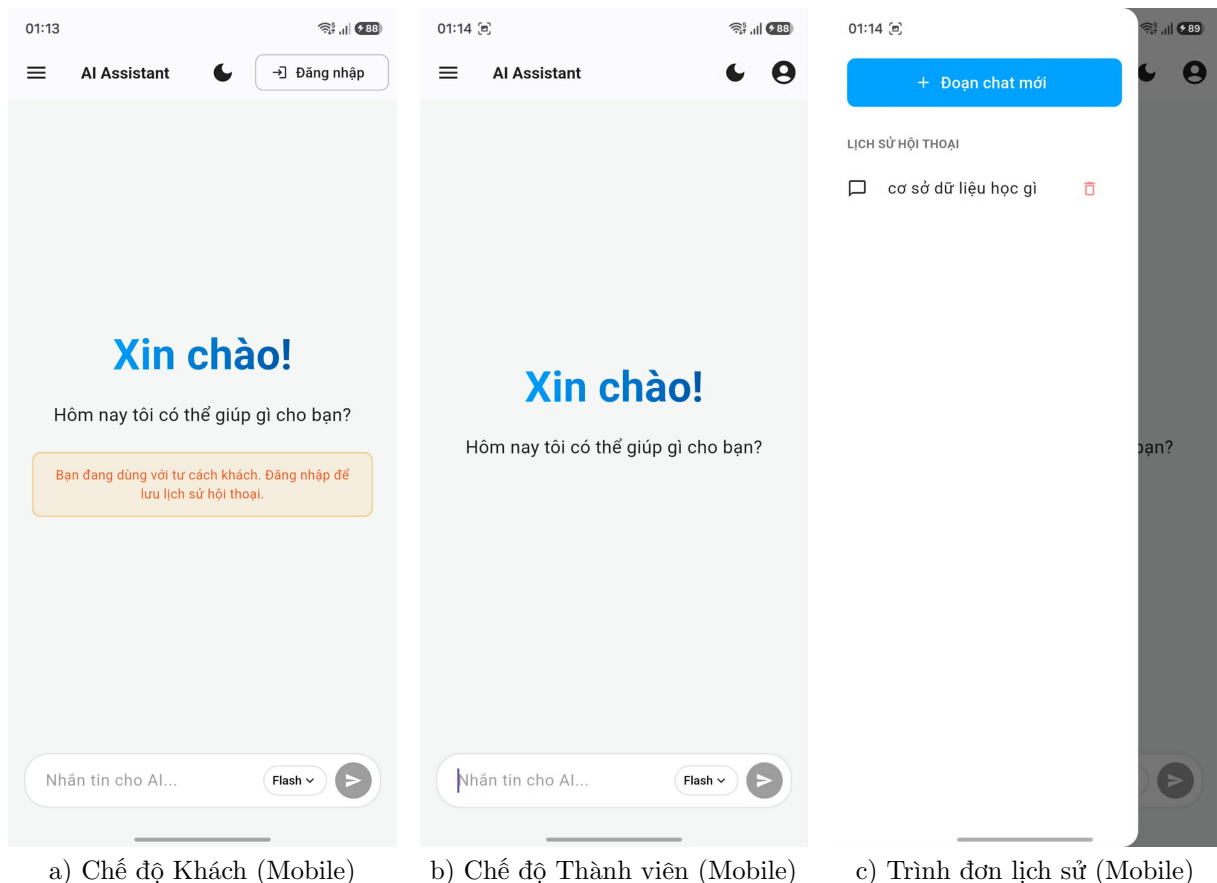
Cách sử dụng:

1. Sinh viên mở biểu mẫu Hồ sơ học tập từ thanh công cụ hoặc màn hình chat.
2. Lựa chọn đúng Ngành đào tạo và Năm học hiện tại từ các danh sách dropdown tương ứng.
3. Nhập định hướng nghề nghiệp của bản thân (ví dụ: "Muốn theo hướng Kỹ sư dữ liệu", "Định hướng lập trình Mobile").
4. Nhấn chọn nút "Lưu thay đổi" (hoặc "Save details"). Hệ thống sẽ đồng bộ thông tin này vào cơ sở dữ liệu để phục vụ cho các lượt truy xuất của AI Agent.

4.7.6.5 Màn hình Trò chuyện AI Chatbot



Hình 4.14: Giao diện trò chuyện AI Chat trên nền tảng Website



Hình 4.15: Giao diện Màn hình AI Chatbot trên thiết bị di động

Mục đích: Cung cấp không gian tương tác trực tiếp bằng ngôn ngữ tự nhiên giữa sinh viên và AI Agent. Để tối ưu hóa trải nghiệm người dùng, hệ thống hỗ trợ cả chế độ Khách (chưa đăng nhập) và chế độ Thành viên (đã đăng nhập).

Thành phần giao diện:

1. Khu vực lịch sử trò chuyện (Sidebar trên Website hoặc Drawer Menu trên Mobile): Nơi hiển thị danh sách các cuộc hội thoại cũ đã được AI tự động tóm tắt tiêu đề. Có nút "+ Đoạn chat mới" (New Chat) và nút xóa đoạn chat.
2. Khung hiển thị nội dung trò chuyện (ở trung tâm): Hiển thị luồng tin nhắn giữa sinh viên và AI. Đối với người dùng chưa đăng nhập (chế độ Khách), hệ thống sẽ hiển thị một khung cảnh báo màu cam nhắc nhở đăng nhập để lưu trữ lịch sử cuộc gọi lâu dài.
3. Khu vực nhập câu hỏi (phía dưới): Bao gồm ô nhập liệu văn bản,

danh sách thả xuống để chọn chế độ phản hồi (Mode) và nút gửi câu hỏi.

Cách sử dụng:

1. Trò chuyện với AI Agent: Sinh viên nhập câu hỏi vào ô dưới cùng, lựa chọn chế độ xử lý phù hợp và nhấn nút gửi.
 - Chế độ Flash: Tốc độ phản hồi nhanh, độ trễ thấp; phù hợp cho các câu hỏi đơn giản, hỏi đáp quy chế thông thường, sử dụng mô hình gọn nhẹ giúp tiết kiệm tài nguyên.
 - Chế độ Pro: Tốc độ phản hồi chậm hơn nhưng chất lượng câu trả lời cao hơn, thực hiện các bước suy luận sâu hơn để giải quyết các câu hỏi phức tạp hoặc có nhiều ràng buộc.
2. Chế độ Khách: Người dùng chưa đăng nhập có thể chat thử nghiệm trực tiếp, tuy nhiên lịch sử trò chuyện chỉ được lưu tạm trên bộ nhớ ứng dụng và sẽ mất đi khi đóng tab trình duyệt hoặc thoát ứng dụng di động.
3. Quản lý lịch sử hội thoại (Chỉ dành cho Thành viên): Sinh viên đã đăng nhập có thể xem lại, tải lại nội dung chat cũ từ MongoDB hoặc xóa vĩnh viễn các cuộc hội thoại không cần thiết thông qua thanh Sidebar/Drawer.

Chương 5

Kết luận

5.1 Kết quả đạt được

Đề tài đã nghiên cứu và xây dựng thành công "Hệ thống hỗ trợ thông tin học tập cho sinh viên sử dụng AI Agents". Bằng việc kết hợp sức mạnh của Đồ thị tri thức [13] và kiến trúc điều phối Đa tác nhân [19], hệ thống đã góp phần giải quyết bài toán phân mảnh thông tin trong giáo dục đại học [36].

Những đóng góp chính của đề tài bao gồm ba khía cạnh quan trọng. Thứ nhất, nghiên cứu đã số hóa và cấu trúc hóa thành công kho dữ liệu học vụ phức tạp thành Đồ thị tri thức (LightRAG) có khả năng suy luận nhanh chóng [10]. Thứ hai, hệ thống đã xây dựng một luồng xử lý Đa tác nhân thông minh, cho phép AI tự động chọn lọc công cụ tìm kiếm phù hợp với từng dạng câu hỏi của sinh viên. Cuối cùng, dự án triển khai hệ thống đánh giá tự động thông minh (LLM-as-a-Judge) [37] kết hợp RAGAS [8] giúp đo lường và tinh chỉnh chất lượng câu trả lời một cách khách quan. Nhìn chung, hệ thống đã giúp sinh viên dễ dàng tra cứu, thấu hiểu mối liên hệ giữa các môn học tiên quyết và lập kế hoạch học tập hiệu quả.

5.2 Hạn chế của hệ thống

Dù đạt được những kết quả khả quan, hệ thống vẫn còn tồn tại một số giới hạn về mặt kỹ thuật. Trở ngại lớn nhất hiện nay nằm ở độ trễ xử lý (Latency); do bản chất của việc duy trì hội thoại đa tác nhân [34] và suy luận phức tạp của mô hình LLM, tốc độ phản hồi của hệ thống vẫn còn tương đối chậm, đặc biệt khi có nhiều người dùng truy vấn đồng thời. Kế đến là những hạn chế trong việc số hóa tài liệu tiếng Việt, khi quá trình bóc tách cấu trúc bảng biểu phức tạp từ các file PDF quy chế đào tạo đôi

khi vẫn gặp lỗi nhận diện (OCR), dẫn đến việc Đồ thị tri thức bị khuyết dữ liệu ở một vài tiểu mục nhỏ.

5.3 Hướng phát triển trong tương lai

Qua quá trình nghiên cứu và thực hiện đề tài, nhóm đã đạt được các mục tiêu trọng tâm về lý thuyết, giải pháp và thực nghiệm. Về lý thuyết, nhóm xây dựng thành công quy trình đánh giá đồ thị tri thức đa tầng dựa trên các nghiên cứu về hệ thống RAG hiện đại [5], [11], [24]. Về giải pháp, việc đề xuất kiến trúc vùng đệm lưu trữ kết hợp với quy tắc SHACL đã tối ưu hóa cấu trúc đồ thị và đảm bảo tính liên kết chặt chẽ [32]. Thực nghiệm trên toàn bộ kho học liệu của khoa Công nghệ Thông tin đã minh chứng tính ổn định của hệ thống với kết quả vượt trội so với các kiến trúc truyền thống.

Để khắc phục các hạn chế hiện tại và đưa hệ thống vào ứng dụng thực tiễn ở quy mô toàn trường, định hướng nghiên cứu tiếp theo sẽ tập trung vào ba nhiệm vụ cốt lõi. Trước mắt, hệ thống cần nâng cấp luồng AI và năng lực xử lý tài liệu thông qua truy vấn đa phương tiện (Multimodal RAG). Sự cải tiến này giúp AI vượt khỏi giới hạn xử lý văn bản đơn thuần để trực tiếp phân tích và giải thích các biểu đồ, công thức phức tạp hay hình ảnh mô hình trong slide bài giảng nhờ tích hợp các luồng OCR tối ưu sâu. Nhờ đó, sinh viên có thể nhận được phản hồi chính xác ngay cả khi đặt câu hỏi liên quan đến các chi tiết trực quan trong học liệu. Song song với đó, bộ nhớ ngữ cảnh cũng sẽ được mở rộng bằng Knowledge Graph, khai thác sâu cấu trúc đồ thị nhằm liên kết khái niệm chéo giữa nhiều môn học khác nhau; chẳng hạn như việc tự động nhắc lại kiến thức nền tảng của môn Cấu trúc dữ liệu khi truy vấn tài liệu môn Cơ sở dữ liệu.

Bên cạnh việc hoàn thiện năng lực cốt lõi của mô hình, hệ thống sẽ được tích hợp thêm module tạo lộ trình học tập cá nhân hóa. Tính năng này đóng vai trò như một trợ lý định hướng, thiết kế các gợi ý học thuật riêng biệt dựa trên thông tin về chuyên ngành, năm học hiện tại cùng với vị trí và mục tiêu công việc thực tế mà sinh viên mong muốn theo đuổi trong tương lai.

Bước tiến quan trọng sau cùng là nâng cấp tính năng tương tác thành

một diễn đàn học tập đúng nghĩa, vượt ra ngoài khuôn khổ của một công cụ hỏi đáp để trở thành không gian chia sẻ tri thức cộng đồng. Diễn đàn này sẽ được trang bị công cụ AI Summarizer chuyên làm nhiệm vụ đọc, tổng hợp và tóm tắt lại các ý chính hoặc câu trả lời chuẩn xác nhất trong các chủ đề tranh luận dài, giúp người dùng mới dễ dàng nắm bắt trọn vẹn luồng thông tin mà không cần lướt đọc từ đầu. Thêm vào đó, cơ chế gợi ý nhóm học tập (Peer-matching) cũng sẽ được triển khai. Dựa trên lịch sử truy vấn, hành vi thảo luận và sự liên hệ giữa các môn học, hệ thống sẽ chủ động phát hiện những sinh viên có chung mối quan tâm hoặc đang cùng gặp khó khăn ở một chủ đề nhất định để gợi ý kết nối, từ đó thúc đẩy văn hóa học tập cộng tác và hỗ trợ lẫn nhau một cách hiệu quả nhất.

Tài liệu tham khảo

Tiếng Việt

- [1] Trường Đại học Kinh tế TP.HCM (UEH). “Ueh ai chatbot – hệ thống hỗ trợ tư vấn trực tuyến.” [Online]. Available: <https://aichat.ueh.edu.vn/>

Tiếng Anh

- [2] Alhammadi, S., Alhosani, K., and Alhashmi, H., “Knowledge graph-based academic advising system using Neo4j and large language models,” in *Proceedings of the UAE Graduate Students Research Conference*, 2024.
- [3] Bui, C. T. et al., “VMLU benchmarks: A comprehensive benchmark toolkit for Vietnamese LLMs,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vienna, Austria: Association for Computational Linguistics, 2025, pp. 11 495–11 515. DOI: 10 . 18653 / v1 / 2025 . acl - long . 563 [Online]. Available: <https://aclanthology.org/2025.acl-long.563>
- [4] Cheng, J., Lu, C., Yang, L., Chen, G., and Zhang, F., “EasyEA: Large language model is all you need in entity alignment between knowledge graphs,” in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025. [Online]. Available: <https://github.com/alusang/EasyEA-framework>
- [5] Dong, X. L. et al., “Knowledge-based trust: Estimating the trustworthiness of web sources,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1638–1649, 2015.

- [6] Edge, D. et al., *From local to global: A graph RAG approach to query-focused summarization*, 2024. arXiv: 2404.16130 [cs.CL].
- [7] Edge, D. et al., “From local to global: A graph RAG approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [8] Es, S., James, J., Espinosa-Anke, L., and Schockaert, S., “Ragas: Automated evaluation of retrieval augmented generation,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, 2024, pp. 150–158. DOI: 10.18653/v1/2024.eacl-demo.16
- [9] Franco, M. L. G., Silva, J. A. O. C. d., Santos, M. T. P. S. d., and Oliveira, R. G. d. S. B. d., “Feasibility of ontology-guided structuring of stress information: Creation and evaluation of the Mental Stress Ontology (MeSO),” *BMC Medical Informatics and Decision Making*, vol. 26, no. 1, p. 12, 2026. DOI: 10.1186/s12911-026-02755-x
- [10] Guo, Z., Xia, L., Yu, Y., Ao, T., and Huang, C., “LightRAG: Simple and fast retrieval-augmented generation,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 10 746–10 761. DOI: 10.18653/v1/2025.findings-emnlp.568
- [11] HKUDS. “LightRAG: Institutional memory and personal knowledge management with light-weight retrieval-augmented generation,” Accessed: Apr. 18, 2026. [Online]. Available: <https://github.com/HKUDS/LightRAG>
- [12] Hogan, A. et al., “Knowledge graphs,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021. [Online]. Available: <https://dl.acm.org/doi/10.1145/3447772>
- [13] Hogan, A. et al., “Knowledge graphs,” *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–37, 2021.
- [14] IEEE Xplore, “Optimal RAG system design for textbooks: A comprehensive evaluation and performance enhancement study,” *IEEE Transactions on Learning Technologies*, 2026.

- [15] Jang, Y., Kim, J., and Ko, Y., *HELEA: Hard-negative benchmark and LLM-based reranking for robust entity alignment*, 2024. arXiv: 2405.02534 [cs.CL].
- [16] Ji, Z. et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [17] Kasneci, E. et al., “ChatGPT for good? on opportunities and challenges of large language models in education,” *Learning and individual differences*, vol. 103, p. 102 274, 2023.
- [18] Labra Gayo, J. E. et al., “Formalizing and validating Wikidata’s property constraints using SHACL and SPARQL,” *ResearchGate*, 2024. [Online]. Available: https://www.researchgate.net/publication/383382328_Formalizing_and_validating_Wikidata's_property_constraints_using_SHACL_and_SPARQL
- [19] LangChain. “LangGraph: Multi-actor workflows with LLMs,” Accessed: May 1, 2024. [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [20] Lewis, P. et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [21] Li, J., Sun, A., Han, J., and Li, C., “A survey on deep learning for named entity recognition,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 50–70, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/9039685>
- [22] Little, J. D. C., “A proof for the queuing formula: $L = \lambda W$,” *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: 10.1287/opre.9.3.383
- [23] McGuinness, D. L. and Harmelen, F. van, “OWL web ontology language overview,” World Wide Web Consortium (W3C), W3C Recommendation, 2004. [Online]. Available: <https://www.w3.org/TR/2004/REC-owl-features-20040210/>

- [24] Microsoft. “GraphRAG: A modular graph-based retrieval-augmented generation system,” Accessed: Apr. 18, 2026. [Online]. Available: <https://github.com/microsoft/graphrag>
- [25] OpenDataLab. “MinerU: A high-quality tool for parsing mixed image and text documents,” Accessed: Apr. 18, 2026. [Online]. Available: <https://github.com/opendatalab/MinerU>
- [26] Page, L. C. and Gehlbach, H., “How an artificially intelligent virtual assistant helps students navigate the road to college,” *AERA Open*, vol. 3, no. 4, pp. 1–12, 2017. DOI: 10.1177/2332858417749220
- [27] Pham, L., Nguyen, M. N. T., et al., “VN-MTEB: Vietnamese massive text embedding benchmark,” in *Findings of the Association for Computational Linguistics: EACL 2026*, 2026.
- [28] Poveda-Villalón, M., Gómez-Pérez, A., and Suárez-Figueroa, M. C., “OOPS! (ontology pitfall scanner!): An on-line tool for evaluating ontologies,” *International Journal on Semantic Web and Information Systems (IJSWIS)*, 2014.
- [29] Qwen Team, *Qwen2.5 technical report*, 2024. arXiv: 2412.15115 [cs.CL].
- [30] Seo, J. et al., “Structural quality metrics for knowledge graphs,” *arXiv preprint arXiv:2211.10011*, 2022.
- [31] Shin, J., Park, C., Park, J., Seo, J., and Lim, H., “MultiDocFusion : Hierarchical and multimodal chunking pipeline for enhanced RAG on long industrial documents,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 20 985–21 004, ISBN: 979-8-89176-332-6. DOI: 10.18653/v1/2025.emnlp-main.1062 [Online]. Available: <https://aclanthology.org/2025.emnlp-main.1062/>
- [32] W3C. “Shapes constraint language (SHACL),” Accessed: Apr. 18, 2026. [Online]. Available: <https://www.w3.org/TR/shacl/>

- [33] Wang, L., Yang, N., Huang, X., et al., *Multilingual E5 text embeddings: A technical report*, Microsoft Research, 2024. arXiv: 2402 . 05672 [cs.CL].
- [34] Wu, Q. et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” in *First Conference on Language Modeling (COLM)*, 2024.
- [35] Xi, Z. et al., “The rise and potential of large language model based agents: A survey,” *Science China Information Sciences*, 2024.
- [36] Zawacki-Richter, O., Marín, V. I., Bond, M., and Gouverneur, F., “Systematic review of research on artificial intelligence applications in higher education—where are the educators?” *International Journal of Educational Technology in Higher Education*, vol. 16, no. 1, pp. 1–27, 2019.
- [37] Zheng, L. et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.